

Politecnico di Torino

Politecnico di Torino

Corso di Laurea Magistrale in Ingegneria Informatica

A.A. 2025/2026

Sessione di laurea Dicembre 2026

GroundOne: un decision model specialista per la verifica della groundedness nella RAG locale

Fine-tuning di modelli linguistici compatti come verificatori calibrati a singolo passaggio su Apple Silicon

Relatore: Prof. Paolo Garza

Candidato: Francesco Pio De Dominicis

Sommario

La retrieval-augmented generation (RAG) è ormai il modo standard per dare ai modelli linguistici accesso a documenti privati, e oggi l'intera pipeline può essere eseguita sul dispositivo dell'utente con hardware di fascia consumer, a vantaggio della privacy, dell'uso offline e dei costi. Il recupero dei documenti, tuttavia, non elimina le allucinazioni: le risposte generate contengono ancora frasi che il contesto recuperato non supporta o che lo contraddicono. Individuarle è un prerequisito per assistenti locali affidabili e, poiché il controllo va eseguito su ogni risposta, deve trovare posto sulla stessa macchina, dalle risorse limitate, che ospita il generatore.

La località è l'asse portante di questo lavoro e ha una conseguenza tecnica precisa. Su un singolo dispositivo leggere testo costa poco, scriverlo costa molto: il modello legge l'input in un unico passaggio limitato dal calcolo, ma genera l'output un token alla volta, e ogni token è limitato dalla banda di memoria necessaria a leggere tutti i pesi. Sul portatile usato in questa tesi scrivere un token costa da quattordici a sedici volte più che leggerne uno, e una rassegna dell'hardware locale attuale, dagli smartphone ai portatili fino a macchine dedicate come il Mac Studio di Apple e il DGX Spark di NVIDIA, mostra ovunque la stessa asimmetria, con un hardware nuovo che accelera la lettura molto più della scrittura. I piccoli modelli open-weight sono ormai indietro di pochi mesi rispetto alla frontiera chiusa, per cui pipeline locali capaci sono realistiche; la questione è come verificarle a un costo che il dispositivo possa sostenere.

La verifica segue oggi due paradigmi. I **verificatori basati su encoder**, come AlignScore, MiniCheck e HHEM, sono piccoli e veloci, ma restituiscono un unico punteggio fisso per ogni coppia di documento e frase. I **giudici generativi**, invece, sono flessibili, ma scrivono i verdetti token per token, per cui il loro costo cresce con la quantità di testo prodotto; i giudici piccoli violano spesso il formato di output, e la confidenza che esprimono è raramente utilizzabile come probabilità. Di recente è emersa una terza famiglia: i **decision model**, come il proprietario Jev e i modelli open SemIf e Rizzo-Flow, che rispondono a domande chiuse in un unico forward pass leggendo dai logit la probabilità di ciascuna opzione di risposta. Le evidenze a loro favore provengono per lo più dagli stessi autori, e le implementazioni open sono generaliste. Non era ancora stato studiato in modo controllato se un decision

model compatto, specializzato in un unico task, possa eguagliare i giudici generativi e gli encoder dedicati, da dove derivi il suo vantaggio e se le sue probabilità siano utilizzabili, il tutto su hardware consumer.

Questa tesi progetta, addestra e valuta **GroundOne**, un decision model specializzato in un'unica decisione: stabilire, per ogni frase di una risposta RAG, se il documento recuperato la supporta, la contraddice o non la menziona. Ogni frase della risposta è seguita da un marker, e i verdetti di tutte le K frasi si leggono dai logit di tre lettere di risposta in un unico forward pass, senza generare testo. Modelli Qwen2.5 da 0.5B e 3B parametri sono sottoposti a fine-tuning con LoRA sulle risposte di RAGTruth annotate manualmente, con una loss pesata per classe e ristretta alle tre lettere; una temperatura e un bias stimati sul dev set calibrano le probabilità, e una soglia scelta sugli stessi dati le traduce in etichette coerenti e in segnalazioni secondo la policy strict o lenient. Tutto, dalla costruzione del dataset all'addestramento, dalla valutazione alla messa in servizio, gira su un unico portatile Apple M4 con 16 GB di memoria, con MLX e senza servizi esterni, e ogni baseline è eseguita sulla stessa macchina. La valutazione usa tre test set costruiti e congelati prima dell'addestramento (la partizione di test di RAGTruth, LLM-AggreFact e VitaminC) e confronta GroundOne con indizi che non leggono mai il documento, con i modelli non addestrati, con giudici generativi della stessa dimensione, con e senza probabilità, con due decision model generalisti e con encoder specializzati, riportando intervalli calcolati con un bootstrap a cluster.

Sul test set di RAGTruth GroundOne raggiunge una balanced accuracy a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, contro 0.37 e 0.44 dei modelli non addestrati; la sua AUROC (0.905 e 0.940) supera quella degli encoder (circa 0.82), e il modello supera nettamente entrambi i decision model generalisti: RizzoFlow, otto volte più grande del 0.5B, e d1-3B di Liquid AI, della stessa dimensione del 3B (0.657), che a sua volta è molto più forte sul test fuori dominio VitaminC. Su LLM-AggreFact il 3B è alla pari con MiniCheck-RoBERTa (74.8 contro 74.0), sopra il modello 7B zero-shot pubblicato e sotto il decision model chiuso Jev, misurato in modo indipendente (78.6). Quando la risposta contiene quattordici o più frasi ($K \geq 14$), GroundOne risulta circa due volte più veloce di un giudice sintetico e sei volte più veloce di uno verboso della stessa dimensione e, a differenza dei giudici, produce sempre un output valido. Consuma anche meno energia (5.5 joule per risposta con il 0.5B, 33 con il 3B), ma il risparmio è minore di quello sul tempo, perché un prefill lungo assorbe più potenza del decoding. Combinando le specifiche dei produttori con la lunghezza misurata degli output dei giudici, un giudice da 3B deve spendere almeno 1.8 secondi soltanto per scrivere i verdetti di una risposta lunga sul portatile della tesi, e 0.12 secondi persino sulla scheda grafica consumer più veloce: un costo che GroundOne non paga su nessun hardware. Quando si chiede loro lo stesso output, i giudici generativi forniscono probabilità che distinguono a stento le decisioni corrette da quelle errate, mentre la calibrazione

porta l'Expected Calibration Error di GroundOne sul test set a 0.019. Anche il modo di leggere la decisione conta: a parità di modello addestrato, confrontare soltanto le tre lettere ammesse è più accurato che lasciare al modello generare liberamente il proprio verdetto, con un guadagno di balanced accuracy di 0.25 per il 0.5B e di 0.07 per il 3B. Restano alcuni aspetti da migliorare, che indicano direzioni naturali per il lavoro futuro: le prestazioni calano su domini lontani da quello di addestramento, come il fact checking di VitaminC; le allucinazioni più sottili, nascoste in frasi che per il resto ricalcano fedelmente la fonte, vengono individuate solo in parte; infine, l'addestramento non è ancora del tutto stabile al variare del seed, un aspetto che semplici modifiche all'ottimizzatore potrebbero attenuare.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Indice

Elenco delle figure x

1 Introduzione 1
- 1.1 Motivazione . . . 1
  - 1.1.1 La RAG sul dispositivo dell'utente . . . 1
  - 1.1.2 Le allucinazioni e la necessità di verificare . . . 2
  - 1.1.3 Il costo di verificare una risposta su un laptop . . . 3
- 1.2 Decision Model . . . 3
- 1.3 Approccio e domande di ricerca . . . 4
- 1.4 Ambito . . . 6
- 1.5 Contributi . . . 7
- 1.6 Risultati principali . . . 7
- 1.7 Struttura della tesi . . . 8

2 Inferenza locale dei modelli linguistici 10
- 2.1 Dai modelli remoti ai modelli locali . . . 10
- 2.2 Modelli open-weight e small language model . . . 14
  - 2.2.1 Open-weight non significa open source . . . 14
  - 2.2.2 Il divario con i modelli chiusi si sta chiudendo . . . 15
  - 2.2.3 Small language model . . . 16
- 2.3 Il costo dell'inferenza su un singolo dispositivo . . . 17
  - 2.3.1 Prefill e decoding . . . 17
  - 2.3.2 Il modello roofline . . . 17
  - 2.3.3 Un limite inferiore alla generazione . . . 19
- 2.4 L'hardware per l'inferenza locale . . . 20
  - 2.4.1 Classi di hardware . . . 20
  - 2.4.2 L'asimmetria vale per ogni classe . . . 25
- 2.5 Runtime e quantizzazione . . . 25
  - 2.5.1 Runtime . . . 25
  - 2.5.2 Motori di inferenza specializzati . . . 27
  - 2.5.3 Quantizzazione . . . 28

2.5.4 Generare più in fretta senza generare meno 29
2.6 Energia 30
2.7 Conseguenze per la verifica sul dispositivo 30

3 Fondamenti e stato dell'arte 32
3.1 Decision Model 32
3.2 Retrieval-augmented generation e allucinazioni 35
3.3 Verifica della groundedness 35
3.4 Calibrazione e selective prediction 36
3.5 Parameter-efficient fine-tuning 38
3.6 Lacuna nella letteratura 39

4 GroundOne: metodo e implementazione 41
4.1 Task e decisione 41
4.2 Tutti i verdetti in un solo forward pass 42
4.3 Obiettivo di addestramento 45
4.4 Calibrazione 46
4.5 Software 48

5 Dati 50
5.1 Task ed etichette 50
5.2 Dataset di partenza 51
5.3 Costruzione 53
5.4 Il dataset risultante 55
5.5 Affidabilità dei test set 56
5.6 Analisi esplorativa 58
5.7 Controllo del leakage 60

6 Impostazione sperimentale 62
6.1 Modelli 62
6.2 Baseline 63
6.3 Metriche 66
6.4 Protocollo statistico 67
6.5 Latenza, energia e hardware 68

7 Risultati 70
7.1 Addestramento 70
7.2 Variabilità tra seed 72
7.3 Accuratezza sul test set RAG (T1) 73
7.4 Confronto con le baseline 75
7.5 Trasferimento a un dominio diverso (T3) 78
7.6 Readout e generazione a confronto 79

7.7 Calibrazione e selective prediction . . . 80
7.8 Probabilità a parità di output . . . 82
7.9 Latenza in funzione di K . . . 83
7.10 Energia per verifica . . . 84
7.11 Località: esecuzione offline, quantizzazione e co-residenza . . . 85
7.12 Analisi degli errori . . . 86

8 Discussione, limiti e minacce alla validità 88
8.1 Quando conviene un Decision Model specialista . . . 88
8.2 Limiti . . . 92
8.3 Minacce alla validità . . . 93

9 Conclusioni e sviluppi futuri 95
9.1 Sintesi dei risultati . . . 95
9.2 Sviluppi futuri . . . 96

A Output di ogni sistema su una stessa risposta 98
A.1 Verdetti frase per frase . . . 99
A.2 Output grezzi . . . 100

Bibliografia 102

Elenco delle figure

1.1 Due modi di verificare le K frasi di una risposta. Parallelogrammi: input e output; rettangoli blu: componenti del modello; rettangolo giallo: elaborazione esterna al modello; rombi: decisioni. Il decision model legge documento e risposta una sola volta e, per ogni frase, sceglie fra tre risposte ammesse, restituendo anche la probabilità. L'LLM generativo ripassa dal modello per ogni token scritto (ciclo in rosso), e i verdetti vanno poi ricavati dal testo, che può non rispettare il formato richiesto. 5

2.1 Capacità del miglior modello chiuso e del miglior modello open-weight nel tempo, sull'indice aggregato di capacità di Epoch AI (punti: singoli modelli; linee: migliore fino a quel momento). Grafico ricavato dalla tabella dati pubblicata da Epoch AI [34]. 15

2.2 Tempo per token quando il modello legge l'input (prefill) e quando scrive l'output (decoding), misurato sulla macchina della tesi (Apple M4, 16 GB, MLX a 8 bit, una richiesta alla volta). Prefill: mediana su contesti di 500, 1000 e 2000 token. 18

2.3 Modello roofline della macchina della tesi [41]. Tetto inclinato: banda di memoria dichiarata da Apple (120 GB/s). Tetto piatto: throughput raggiunto nel prefill, limite inferiore del picco non pubblicato. Intensità calcolate per uno strato feed-forward di Qwen2.5 con pesi a 8 bit e attivazioni a 16 bit; l'attenzione è trascurata. . . . 19

2.4 Memoria necessaria per i pesi di un modello in funzione della sua dimensione e della precisione (numero di parametri moltiplicato per i bit per peso, compreso mezzo bit di scale per gruppo nei formati quantizzati). Linee tratteggiate: tagli di memoria comuni nell'hardware locale. La KV cache, le attivazioni e gli altri modelli in esecuzione si aggiungono a questo valore. 20

2.5 Le due architetture dell'hardware per l'inferenza locale. Rettangoli blu: unità di calcolo; rettangoli verdi: memoria; rettangolo arancione: i pesi del modello; spessore delle linee: banda indicativa. Valori di capacità e banda dalla Tabella 2.2. In un sistema discreto un modello che non entra nella VRAM deve passare dal collegamento PCIe, molto più lento; in un sistema a memoria unificata il modello può occupare quasi tutta la memoria della macchina, ma ogni token generato la legge a una velocità inferiore a quella di una scheda grafica di fascia alta. 21

2.6 Alcuni dei sistemi per l'inferenza locale confrontati in questa sezione (immagini non in scala). Fonti: (a), (c) Apple; (b) LoMit, Wikimedia Commons, CC0; (d), (e) NVIDIA. Le immagini dei produttori sono materiale promozionale, riprodotto a scopo di documentazione scientifica. 22

2.7 L'hardware per l'inferenza locale sui due assi che stabiliscono che cosa può girare: capacità di memoria (quali modelli entrano) e banda di memoria (quanto velocemente si generano i token). Specifiche dei produttori: marcatori in corrispondenza della configurazione con più memoria, barre ombreggiate sull'intervallo di tagli di memoria offerti; la macchina della tesi è rappresentata con i suoi 16 GB effettivi. 24

2.8 Limiti inferiori al costo della generazione di testo su ciascuna classe di hardware locale, calcolati come byte dei pesi divisi per la banda dichiarata dal produttore (una richiesta alla volta; KV cache trascurata). (a) Tempo per token generato con i due modelli base di GroundOne a 8 bit; rombi: tempo di decoding misurato sulla macchina della tesi. (b) Tempo minimo che un giudice generativo da 3B impiega soltanto per scrivere i verdetti di una risposta di quattordici o più frasi (mediana dei token generati nel protocollo di latenza). Un decision model non genera alcun token, quindi per esso entrambi i costi sono nulli su ogni macchina. 26

2.9 Lo streaming degli esperti dall'SSD in un motore come Colibri. In memoria restano la parte del modello usata da ogni token (attenzione, router, esperti condivisi) e una cache degli esperti usati di recente; tutti gli esperti stanno sull'SSD. Per ogni token il router sceglie pochi esperti: quelli già in cache si usano subito, quelli mancanti si leggono dal disco, con una banda di pochi gigabyte al secondo. Per esempio, GLM-5.2 tiene residenti circa 10 GB e lascia sul disco circa 429 GB di pesi [60]. Lo schema non è in scala. 28

3.1 Tre modi di verificare le K frasi di una risposta. Un encoder specializzato valuta ogni coppia (documento, frase) in un passaggio separato e restituisce solo una probabilità di supporto. Un giudice generativo legge tutto, ma scrive il proprio verdetto token per token (ciclo rosso), quindi il costo cresce con il testo prodotto e l'output va interpretato da un parser. Un decision model specialista legge documento e risposta in un solo prefill e ricava da ogni marker tre probabilità, senza generare alcun token. 37

3.2 Schema di una curva risk–coverage (forma illustrativa, non un risultato). Le decisioni sono ordinate dalla più sicura alla meno sicura; per ogni quota decisa (coverage) si riporta il tasso di errore (rischio) delle decisioni accettate. A coverage 1 il rischio coincide con l'errore complessivo; rinunciando alle decisioni meno sicure l'errore scende. L'area sotto la curva (AURC) riassume la curva in un numero: più è piccola, meglio la confidenza separa le decisioni corrette da quelle errate. 38

3.3 Low-rank adaptation applicata a uno strato lineare. La matrice dei pesi W resta congelata (in GroundOne a 8 bit); si addestrano soltanto le due matrici piccole A e B, il cui prodotto, scalato di α, si somma all'output dello strato. Con rango r l'adapter ha $r(d + k)$ parametri invece dei dk di W. 39

4.1 Il prompt a marker di GroundOne (design b). Le frasi della risposta sono numerate e ogni frase valutata è seguita dal marker “ `Verdict:`”; una frase senza contenuto verificabile (qui un'intestazione) resta come contesto, senza marker. Il modello elabora l'intera sequenza in un solo forward pass, e la testa di output, ristretta alle lettere A, B e C, si applica soltanto nelle posizioni dei marker. Per l'attenzione causale il verdetto della frase i dipende dal documento e dalle frasi da 1 a i. 42

4.2 Costo stimato della verifica di una risposta di $S = 800$ token in funzione di K, calcolato con le sole costanti misurate sull'M4: il tempo per token del prefill e del decoding e il numero medio di token che ciascun giudice genera per verdetto sul sottoinsieme di latenza (K medio 8.7). È un modello analitico, non una misura: le latenze misurate sono nella Sezione 7.9. Il costo del decision model cresce di soli tre token di prefill per decisione, quello dei giudici di alcuni token di decoding per decisione. 44

4.3 Le due soluzioni per ottenere K verdetti da una sola lettura. (b) Il readout a marker, adottato come predefinito, inserisce un marker dopo ogni frase e legge tutti i verdetti nello stesso passaggio. (a) La soluzione di riserva legge una volta il prefisso condiviso e vi accoda una breve domanda per ogni frase, valutate in batch; richiede però K copie della KV cache del prefisso, ed è il formato naturale per un modello non addestrato. 44
4.4 Dai logit alle etichette restituite. La decisione grezza, cioè l'argmax dei logit, serve alle metriche di classificazione del Capitolo 7. Le probabilità calibrate, ottenute con temperatura e bias stimati sul dev set, alimentano invece l'unica regola dell'output restituito: supportata se $P(\text{supported}) \geq \tau$, altrimenti la più probabile fra contraddetta e non menzionata. Da qui derivano i flag strict e lenient. 48
4.5 Architettura del software. In alto la pipeline sperimentale, dai dataset pubblici ai test congelati, eseguita sotto il memory guard; in basso il servizio di verifica, in cui riga di comando e server HTTP (endpoint nativo e compatibile) condividono lo stesso Engine, che serializza le richieste ed esegue ogni chiamata MLX su un unico worker thread. L'adapter e il file di calibrazione prodotti dalla pipeline sono gli unici artefatti che il servizio legge. 49

5.1 Costruzione del dataset. Le risposte di RAGTruth sono segmentate, filtrate dalle frasi senza contenuto verificabile, etichettate a partire dagli span e ripulite dalle esclusioni; lo split ufficiale di test diventa T1, mentre le fonti di addestramento sono divise per fonte in training set e dev set. LLM-AggreFact e VitaminC forniscono soltanto test set (T2 e T3). Un controllo automatico del leakage accompagna ogni costruzione, e i test set sono congelati con un hash prima di qualsiasi addestramento. 54
5.2 Regola con cui le annotazioni a livello di span di RAGTruth diventano etichette di frase. Le frasi senza contenuto verificabile restano nel prompt come contesto ma non sono decisioni; fra le altre, il conflitto prevale sull'informazione infondata. 55
5.3 Distribuzione delle classi fra le frasi etichettate di ciascuno split. Negli split di RAGTruth (train, dev e T1) circa l'88–91 % delle frasi è supportato, mentre T3 è campionato con 1,000 coppie per classe. . 56
5.4 Distribuzione di K, il numero di frasi valutate per risposta, nel training set e in T1. Le linee tratteggiate delimitano le fasce di K usate per riportare i risultati. 57
5.5 Quota di frasi contraddette o non menzionate in base al modello che ha scritto la risposta, nel training set e in T1. 59

5.6 Sovrapposizione lessicale tra una frase e il suo documento, cioè quota delle parole di contenuto della frase presenti nel documento, per classe, nel training set, in T1 e in T3. Tra parentesi è indicata la media. 60

5.7 Quota di frasi contraddette o non menzionate in base alla posizione della frase nella risposta. 61

6.1 Mappa delle baseline. Ciascuna differisce da GroundOne per un aspetto preciso e risponde quindi a una domanda diversa: quanto conta l'addestramento (B1), quanto il readout ristretto rispetto al token generato (B2), quanto il modo di produrre il verdetto (giudici B0a, B0b, B0d), quanto la specializzazione rispetto a un decision model generalista (B3, B3b), quanto l'architettura (encoder B4) e quanto basta senza leggere il documento (segnali elementari). 64

6.2 Bootstrap appaiato a cluster. Le frasi del test set sono raggruppate per fonte; a ogni ripetizione si estraggono le fonti con reinserimento, si calcola la metrica dei due sistemi sulle stesse fonti estratte e se ne registra la differenza. Dopo $B = 2{,}000$ ripetizioni, i percentili 2.5 e 97.5 delle differenze danno l'intervallo al 95 %, e la quota di differenze di segno opposto a quello osservato dà il p-value. 68

7.1 Tutte le differenze appaiate tra sistemi sui test set congelati, con l'intervallo bootstrap a cluster al 95 %. In blu le differenze a favore del primo sistema (A), in arancione quelle a favore del secondo (B), in grigio quelle non significative, il cui intervallo contiene lo zero. . . 71

7.2 Balanced accuracy a tre classi sul dev set durante l'addestramento del modello 0.5B: valore complessivo (a sinistra) e valore per task (a destra). La linea punteggiata indica il livello casuale. Il 3B mostra lo stesso andamento sul proprio sottoinsieme di 200 risposte del dev set. 72

7.3 Balanced accuracy a tre classi sul dev set durante l'addestramento del modello 0.5B, con la stessa procedura e tre seed diversi. Due esecuzioni convergono allo stesso livello; la terza è collassata su un predittore costante ed è stata interrotta. 73

7.4 AUROC di P(supported) su T1 per task (question answering, riassunto, data-to-text) e per sistema. Il vantaggio di GroundOne si concentra nel data-to-text; nella summarization MiniCheck-RoBERTa supera il 0.5B ed è alla pari con il 3B. 75

7.5 T1: AUROC di P(supported) per ogni sistema (a sinistra) e balanced accuracy a tre classi per i sistemi che hanno un output a tre classi (a destra). Rizzo-Flow è valutato su 300 risposte e il 3B non addestrato su 600; tutti gli altri sistemi, compreso d1-3B, sono valutati sull'intero test set. 76

7.6 Confronto dei sistemi su T3 (VitaminC): a sinistra l'AUROC di P(supported), a destra la balanced accuracy a tre classi per i sistemi con un output a tre classi. Rizzo-Flow è valutato su un sottocampione di 600 coppie; il tratteggio indica il livello casuale. Per MiniCheck-RoBERTa VitaminC fa parte della genealogia dei dati di addestramento. 78

7.7 Controllo B2 sull'intero T1. Con gli stessi pesi e lo stesso forward pass, il verdetto è letto dalle tre lettere di risposta (GroundOne) oppure preso come il token che emetterebbe la generazione greedy; gli errori di formato sono contati come errori. 79

7.8 Reliability diagram di P(supported) del 3B, prima e dopo la calibrazione stimata sul dev set, per il dev set, T1 e T3; ogni punto è una fascia di probabilità con almeno 20 frasi. Come per il 0.5B, la calibrazione si trasferisce a T1 ma non a T3. 80

7.9 Reliability diagram di P(supported) del 0.5B, prima della calibrazione (grezza) e dopo la calibrazione stimata sul dev set, per il dev set, T1 e T3. Ogni punto rappresenta una fascia di probabilità che contiene almeno 20 frasi. 80

7.10 Curve risk–coverage della decisione binaria del 0.5B con probabilità calibrate: tasso di errore al crescere del numero di frasi meno sicure che vengono rinviate. 81

7.11 Confronto a parità di output sulle 200 risposte del sottoinsieme di latenza. Ai giudici generativi della stessa dimensione si chiede l'output di GroundOne, con le probabilità lette dai logit della lettera generata oppure scritte in JSON; tutti i sistemi sono calibrati sul dev set con la stessa mappa. Sono considerate soltanto le frasi con un verdetto valido. 82

7.12 Tempo mediano per risposta in funzione di K sulle stesse 200 risposte di T1 (MLX in-process, una risposta alla volta, M4 con 16 GB). Sono confrontati GroundOne, il decision model con una domanda per frase e i giudici generativi sintetico e verboso. Scala logaritmica. 83

7.13 Giudici generativi sul sottoinsieme di latenza: balanced accuracy a tre classi, con gli errori di formato contati come errori, e quota di frasi il cui verdetto non è stato possibile interpretare. 84

7.14 Energia per risposta verificata su 100 risposte del sottoinsieme di latenza, misurata con powermetrics su CPU, GPU e Neural Engine al netto della potenza a riposo, insieme alla potenza netta media durante l'esecuzione. 85

8.1 Da dove viene l'accuratezza di GroundOne su T1: lo stesso modello base non addestrato (readout, design a), addestrato ma con la decisione presa dal token che genererebbe (controllo B2, errori di formato contati come errori) e addestrato con il readout ristretto. Tra parentesi il guadagno rispetto al passaggio precedente. Il 3B non addestrato è valutato su 600 risposte; il tratteggio indica il livello casuale. 90

8.2 Quando conviene un decision model specialista, secondo i risultati di questa tesi. È una sintesi qualitativa: i tempi si riferiscono al portatile M4 e alle risposte con 14 o più frasi, e il trasferimento fuori dominio è quello osservato su LLM-AggreFact e VitaminC. 92

Capitolo 1

Introduzione

1.1 Motivazione

1.1.1 La RAG sul dispositivo dell'utente

La retrieval-augmented generation (RAG) è ormai l'approccio standard per far rispondere un modello linguistico a domande su documenti che non ha mai visto in addestramento. Un retriever seleziona i passaggi pertinenti alla domanda, e il modello scrive la risposta avendoli nel proprio contesto [1]. La stessa ricetta serve a un'azienda che consulta i propri report interni, a un avvocato che esamina un contratto e a uno studente che interroga i propri appunti, e in tutti questi casi il valore della risposta dipende dalla sua fedeltà ai documenti.

Fino a poco tempo fa una pipeline di questo tipo richiedeva un modello ospitato in remoto, raggiungibile via rete. Oggi non è più così. Modelli open-weight compatti e runtime efficienti girano su hardware consumer [2, 3], e i produttori di piattaforme integrano direttamente nei propri dispositivi modelli da pochi miliardi di parametri [4]. Il divario con i migliori modelli chiusi è ridotto e continua a ridursi: i modelli che entrano in una singola scheda grafica consumer raggiungono le prestazioni della frontiera del momento nel giro di sei-dodici mesi [5], e su una nota leaderboard il vantaggio del miglior modello chiuso sul miglior modello open-weight è sceso in un solo anno dall'8 % a meno del 2 % [6]. L'intera pipeline, dal recupero dei documenti alla risposta, può quindi restare sulla macchina dell'utente. Questo conta per quattro ragioni. Quando i documenti sono riservati o soggetti a regolamentazione, non lasciano mai il dispositivo; la privacy diventa così una proprietà tecnica del sistema, anziché una promessa del fornitore che l'utente non ha modo di verificare. Una recente disputa sulla paternità di un risultato tra OpenAI e due matematici, che avevano scritto bozze non ancora pubblicate all'interno di uno dei prodotti dell'azienda, ha mostrato quanto la questione sia concreta: lo stesso fornitore ha dichiarato di non poter escludere che dati derivati

da quell'utilizzo abbiano contribuito a migliorare i suoi modelli [7]. Quando manca la rete, l'assistente continua a funzionare. Quando il comportamento del sistema deve restare stabile, i pesi cambiano solo se è l'utente a cambiarli, invece di seguire gli aggiornamenti di un fornitore. Infine, il costo diventa quello dell'hardware e della sua energia, invece di un prezzo per token. I modelli locali, inoltre, espongono i propri logit, che le interfacce remote spesso nascondono o limitano: una proprietà che si rivela essenziale per l'approccio studiato in questa tesi.

La località, però, è anche un vincolo. Un laptop ha memoria, banda e batteria limitate, e ogni componente dell'assistente deve rientrarvi nello stesso momento. Questa tesi tratta la località come un requisito e non come un dettaglio: ogni fase del lavoro, dalla costruzione del dataset all'addestramento, dalla valutazione alla messa in servizio, gira su un unico laptop consumer con 16 GB di memoria, e nessuna dipende da un servizio esterno. Poiché la località è l'asse portante del lavoro, il Capitolo 2 è interamente dedicato a essa: perché conta, quali modelli e quale hardware la rendono possibile e quanto costa.

1.1.2 Le allucinazioni e la necessità di verificare

Il recupero dei documenti non elimina le allucinazioni. Anche con i passaggi giusti nel contesto, le risposte generate contengono ancora frasi che i passaggi non supportano, o che li contraddicono [8, 9]. Un esempio tratto dai dati usati in questa tesi mostra quanto il problema possa essere sottile. Chiamato a descrivere un ristorante a partire da una scheda strutturata, un modello scrive sei frasi scorrevoli. Quattro sono corrette. Una afferma che il ristorante è aperto sette giorni su sette, mentre la scheda indica che il lunedì è chiuso: la frase contraddice la fonte. Un'altra afferma che il ristorante ha una valutazione di 4.5 stelle su Google, mentre la scheda riporta la valutazione senza indicarne la fonte: la frase aggiunge un dettaglio che la fonte non menziona (Appendice A).

Da questo esempio seguono due osservazioni. In primo luogo, l'errore si trova in una singola frase di una risposta per il resto corretta, quindi un verdetto sull'intera risposta serve a poco: l'utente ha bisogno di sapere di quale frase diffidare. In secondo luogo, le due frasi sbagliate lo sono in modi diversi. Una contraddizione è un errore fattuale rispetto alla fonte, mentre un'aggiunta non supportata può essere un innocuo dettaglio di contorno oppure un'invenzione, e un'applicazione può voler trattare i due casi in modo diverso. Un verificatore utile, quindi, stabilisce per ogni frase se il documento la supporta, la contraddice o non la menziona. E poiché non si può pretendere che l'utente confronti ogni frase con la fonte, il verificatore deve essere eseguito su ogni risposta, in modo automatico, sulla stessa macchina che l'ha generata.

1.1.3 Il costo di verificare una risposta su un laptop

Un verificatore eseguito accanto al generatore si contende con esso la stessa memoria, lo stesso tempo e la stessa batteria. La soluzione più comune, cioè chiedere a un modello linguistico di giudicare la risposta, è costosa proprio dove un laptop è più debole. Un giudice scrive i propri verdetti token per token, e su hardware locale ogni token generato costa molto più di ogni token letto. La lettura del prompt è limitata dalla capacità di calcolo, che la GPU di un laptop gestisce bene, mentre la scrittura dell'output è limitata dalla banda di memoria, perché ogni nuovo token richiede di leggere tutti i pesi [10]. Sul laptop usato in questa tesi, leggere un token richiede circa 0.37 millisecondi con un modello da 0.5B parametri, mentre scriverne uno ne richiede circa 5.9, sedici volte di più, e l'asimmetria vale per ogni classe di hardware locale, dagli smartphone alle macchine dedicate all'IA (Capitolo 2). Un giudice che spiega il proprio ragionamento prima di ogni verdetto moltiplica questo costo per la lunghezza della spiegazione, e il costo cresce ancora con il numero di frasi della risposta. I giudici piccoli, inoltre, non rispettano il formato di output richiesto, e la confidenza che dichiarano è raramente utilizzabile come probabilità.

I verificatori basati su encoder evitano del tutto la generazione. Modelli come AlignScore, MiniCheck e HHEM leggono una coppia di documento e frase e restituiscono un punteggio di supporto [11, 12, 13]. Sono piccoli e veloci, ma forniscono un unico punteggio binario e fisso per ogni coppia: non distinguono una contraddizione da un'omissione, leggono ogni frase senza il resto della risposta, e adattarli a una nuova decisione richiede un nuovo modello.

1.2 Decision Model

Un modo diverso di usare un modello linguistico per prendere decisioni ha attirato di recente l'attenzione: i *decision model*. La categoria è stata introdotta con il nome di *System One model* [14], che richiama il modo di pensare rapido e intuitivo contrapposto al ragionamento deliberato; questa tesi adotta il termine decision model, adottato in seguito dall'ecosistema. Invece di generare una risposta, un modello di questo tipo legge una sola volta la domanda e il contesto e restituisce una probabilità per ciascuna risposta ammissibile, per esempio leggendo i logit delle lettere delle opzioni alla fine del prompt (e.g., A: 90%, B: 5%, C: 5%) [15, 16]. Ne seguono direttamente tre proprietà. Non viene generato alcun testo, quindi la decisione costa solo la lettura dell'input. L'output è sempre una delle risposte ammissibili, quindi è sempre valido. Molte domande sullo stesso contesto possono condividere un unico passaggio su di esso, quindi ogni domanda aggiuntiva costa poco.

L'idea si presta bene alla verifica. Una risposta di K frasi pone K domande chiuse sullo stesso documento, ciascuna con tre risposte ammissibili. Un decision

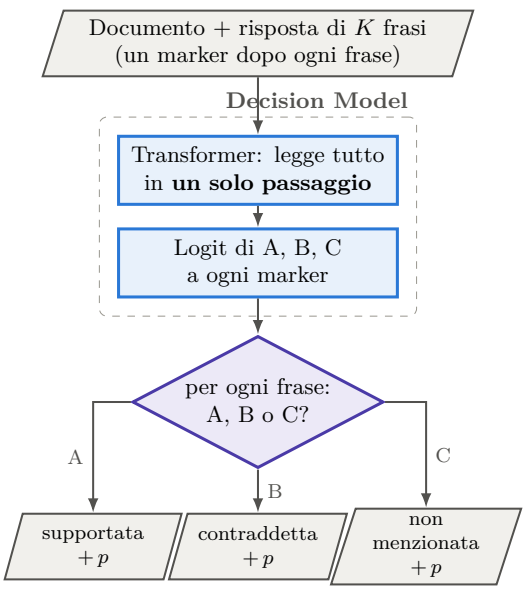
model può rispondere a tutte leggendo il documento una sola volta, e alla probabilità restituita per ogni frase si possono applicare una calibrazione e una soglia scelta dall'applicazione. Un LLM tradizionale, invece, produce la risposta in modo autoregressivo: dopo aver letto il documento deve generare i verdetti un token alla volta, con un nuovo passaggio nel modello per ogni token scritto, e solo alla fine il testo prodotto può essere interpretato come una decisione (Figura 1.1).

Il campo è giovane e si muove in fretta. Dopo il primo modello ospitato sono arrivate le implementazioni open [15, 16], e il 9 ottobre 2026 l'hub di Hugging Face ha introdotto ufficialmente un tag per i decision model, che il giorno successivo raccoglieva già circa 1500 modelli, più della metà dei quali pubblicati nelle settimane precedenti [17]. Anche organizzazioni affermate hanno rilasciato i propri, come d1-3B di Liquid AI e Clef di Cloudflare [18, 19]. Le evidenze, però, sono in ritardo rispetto ai rilasci. Provengono per lo più dagli autori di ciascun sistema, le valutazioni indipendenti sono recenti e spesso meno favorevoli [20, 21], e una leaderboard comune rileva che i decision model generalisti con meno di un miliardo di parametri superano a stento la baseline banale nel rilevamento delle allucinazioni nelle risposte RAG [22]. Tutte le implementazioni open, inoltre, sono sistemi generalisti in grado di rispondere a qualunque domanda tipizzata.

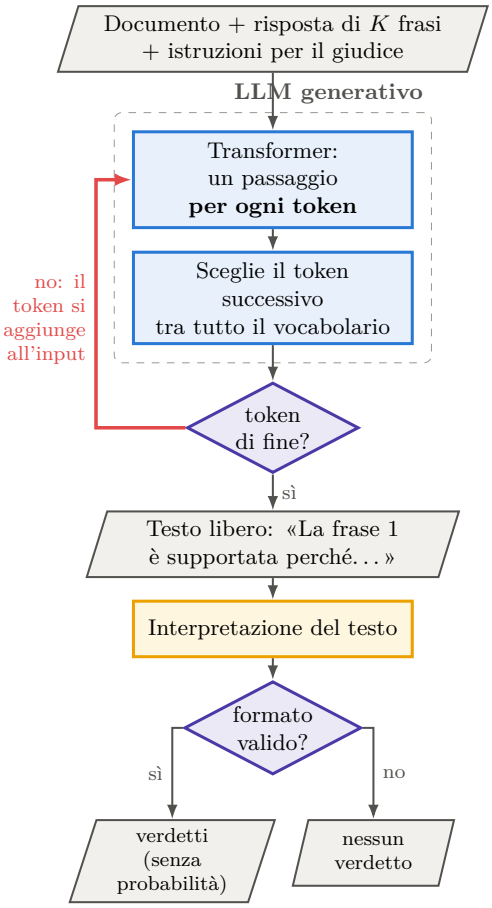
Restano quindi aperti tre problemi. Il primo è capire se, per un singolo task, convenga usare un decision model piccolo e specializzato. Il secondo è capire come un modello di questo tipo si confronti con le alternative: un decision model generalista, un giudice generativo e un verificatore dedicato. Il terzo è capire da dove derivi l'eventuale vantaggio: dalla specializzazione, dal modo in cui la decisione viene letta oppure, più semplicemente, dalla dimensione del modello.

1.3 Approccio e domande di ricerca

Questa tesi progetta, addestra e valuta GroundOne, un decision model compatto specializzato in un'unica decisione: stabilire, per ogni frase di una risposta RAG, se il documento recuperato la supporta, la contraddice o non la menziona. La risposta viene scritta con un breve marker dopo ogni frase, e i verdetti di tutte le frasi si leggono in un unico forward pass, dai logit di tre lettere di risposta in corrispondenza dei marker, senza generare testo. Modelli Qwen2.5 da 0.5B e 3B parametri sono sottoposti a fine-tuning con LoRA sulle risposte di RAGTruth annotate manualmente [23, 24, 8], e le loro probabilità sono calibrate sul dev set. Tre test set sono stati costruiti e congelati prima di addestrare qualunque modello, così che nessuna scelta progettuale potesse essere ottimizzata su di essi. I tre problemi aperti diventano cinque domande di ricerca: le prime due li riformulano in modo verificabile, le altre tre aggiungono ciò che conta per un uso reale sul



(a) Decision model: una sola lettura e una decisione fra tre risposte ammesse per ogni frase.



(b) LLM tradizionale: un passaggio nel modello per ogni token, finché il testo non è completo.

Figura 1.1: Due modi di verificare le K frasi di una risposta. Parallelogrammi: input e output; rettangoli blu: componenti del modello; rettangolo giallo: elaborazione esterna al modello; rombi: decisioni. Il decision model legge documento e risposta una sola volta e, per ogni frase, sceglie fra tre risposte ammesse, restituendo anche la probabilità. L'LLM generativo ripassa dal modello per ogni token scritto (ciclo in rosso), e i verdetti vanno poi ricavati dal testo, che può non rispettare il formato richiesto.

dispositivo, cioè l'affidabilità delle probabilità, il costo e il valore di una decisione a tre classi.

RQ1, specializzazione. Un decision model da 0.5B o 3B parametri, addestrato con fine-tuning sul task, riesce a stabilire se ogni frase è fedele al documento?

E lo fa bene quanto modelli più grandi usati zero-shot[1] e quanto gli encoder specializzati in questa verifica?

RQ2, readout contro generazione. Quanto contribuisce il meccanismo del decision model, cioè il readout[2] della decisione dai logit, a parità di dimensione del modello? Il confronto è duplice: con giudici generativi che scrivono i propri verdetti e con lo stesso modello lasciato libero di generare la risposta. E come cambia la differenza quando aumenta il numero di frasi da verificare?

RQ3, calibrazione. Le probabilità restituite dal modello sono affidabili come misura di confidenza? In particolare, si possono usare per fissare una soglia di decisione e per rinviare all'utente i casi incerti? E sono migliori di quelle che si possono ottenere da un giudice generativo?

RQ4, costo sul dispositivo. Il verificatore rientra nei limiti di memoria, tempo ed energia di un laptop da 16 GB? E quanto costa rispetto a un giudice generativo eseguito sulla stessa macchina?

RQ5, oltre il punteggio binario. Che cosa guadagna chi usa il verificatore da una decisione a tre classi per ogni frase, rispetto al semplice punteggio di un verificatore binario?

1.4 Ambito

La tesi studia la fase di verifica in modo isolato. Il documento recuperato è dato, quindi la qualità del recupero non viene valutata, e una risposta è giudicata solo rispetto al documento da cui è stata generata, non rispetto alla conoscenza generale. GroundOne segnala le frasi ma non le corregge, e restituisce verdetti e probabilità anziché spiegazioni scritte. I dati sono in inglese. I documenti più lunghi del contesto usato in addestramento vengono divisi in parti, come descritto nel Capitolo 6. Infine, gli esperimenti riguardano una sola classe di hardware, un laptop consumer con memoria unificata, e tutti i sistemi, baseline comprese, girano sulla stessa macchina.

[1] Un modello è usato *zero-shot* quando affronta un task senza alcun addestramento specifico e senza esempi nel prompt: riceve soltanto un'istruzione che descrive il compito.

[2] Con *readout* si indica la lettura della decisione direttamente dai logit del modello: invece di lasciargli scrivere una risposta, si confrontano i punteggi che assegna alle sole risposte ammesse (qui le lettere A, B e C) e si sceglie la più probabile.

1.5 Contributi

La tesi offre quattro contributi.

1. **GroundOne**, un decision model specialista e open per la verifica della groundedness a livello di frase, eseguito interamente su un laptop. Grazie a un marker inserito dopo ogni frase, legge tutti i verdetti di una risposta in un solo forward pass. È rilasciato come pacchetto installabile, con un'interfaccia a riga di comando e un'API HTTP locale che comprende anche un'interfaccia compatibile con l'interfaccia System One di Jev, già usata da altri client.

2. **Un dataset a livello di frase derivato da annotazioni pubbliche.** Nessuna etichetta è stata annotata ex novo: le annotazioni umane di RAGTruth, che marcano porzioni di testo, sono state convertite in un'etichetta a tre classi per ogni frase, con regole fissate in anticipo. I tre test set provengono da benchmark pubblici, sono stati verificati con controlli automatici, per esempio sulla segmentazione in frasi, e congelati prima di addestrare qualunque modello, con il leakage controllato a livello di documento.

3. **Una scomposizione controllata dei guadagni.** Il confronto isola il contributo della specializzazione, del readout ristretto e della dimensione del modello. GroundOne è confrontato con indizi che non verificano mai una frase, modelli non addestrati, giudici generativi con e senza probabilità, due decision model generalisti (Rizzo-Flow, da 4B parametri, e d1-3B di Liquid AI, della stessa dimensione del 3B) ed encoder specializzati, tutti sullo stesso hardware e con intervalli ottenuti tramite cluster bootstrap.

4. **Uno studio di calibrazione, tempo ed energia**: come le probabilità possano essere calibrate e tradotte in etichette coerenti, quanto aiutino a rinviare le decisioni incerte e come tempo ed energia di una verifica crescano con la lunghezza della risposta. Lo studio si inserisce in un'analisi dell'inferenza locale che combina le specifiche dei produttori con misure proprie per stabilire, per ogni classe di hardware consumer, il costo minimo che un verificatore che genera testo deve pagare e che un decision model non paga.

1.6 Risultati principali

I risultati mostrano che la specializzazione funziona nel dominio per cui è stata pensata. Sul test set di RAGTruth GroundOne raggiunge una balanced accuracy a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, contro 0.37 e 0.44 degli stessi modelli prima dell'addestramento. Ordina le frasi supportate meglio degli encoder specializzati e supera nettamente entrambi i decision model generalisti:

Rizzo-Flow, otto volte più grande del 0.5B, e d1-3B, della stessa dimensione del 3B. Su LLM-AggreFact, un benchmark costruito da fonti diverse, il 3B è alla pari con il miglior encoder di piccole dimensioni.

Il meccanismo del decision model ripaga in velocità e in accuratezza. Quando una risposta contiene quattordici o più frasi, GroundOne la verifica circa due volte più velocemente di un giudice sintetico e sei volte più velocemente di un giudice verboso della stessa dimensione, con un output sempre valido e consumando meno energia. Anche il modo di leggere la decisione conta: con lo stesso modello addestrato, scegliere la più probabile fra le sole tre lettere ammesse è più accurato che lasciare al modello generare liberamente il proprio verdetto, con un guadagno di balanced accuracy di 0.25 per il 0.5B e di 0.07 per il 3B. Dopo la calibrazione, le probabilità di GroundOne sono vicine alle frequenze osservate, mentre quelle dei giudici generativi a cui si chiede lo stesso output distinguono a stento le decisioni corrette da quelle errate.

Restano alcuni aspetti da migliorare, che indicano direzioni naturali per il lavoro futuro. Le prestazioni calano su domini lontani da quello di addestramento, dove il decision model generalista d1-3B risulta più forte; le allucinazioni più sottili, nascoste in frasi che per il resto ricalcano fedelmente la fonte, vengono individuate solo in parte; infine, l'addestramento non è ancora del tutto stabile al variare del seed[3].

1.7 Struttura della tesi

Il Capitolo 2 è dedicato all'inferenza locale: le ragioni per mantenere i modelli sul dispositivo, i modelli open-weight e i modelli piccoli, il modello di costo dell'inferenza su un singolo dispositivo, il panorama dell'hardware, i runtime e la quantizzazione, l'energia e le conseguenze per la verifica. Il Capitolo 3 passa in rassegna i decision model e l'ecosistema dei decision model, le allucinazioni nella retrieval-augmented generation, i metodi di verifica e la calibrazione. Il Capitolo 4 descrive GroundOne: la decisione, il readout in un solo passaggio, l'obiettivo di addestramento, la calibrazione e l'output esposto dal servizio. Il Capitolo 5 definisce le etichette e descrive come sono stati costruiti, controllati e congelati il training set e i test set. Il Capitolo 6 presenta i modelli addestrati, le baseline, le metriche e il protocollo di misura. Il Capitolo 7 riporta i risultati, il Capitolo 8 ne discute il significato e i

[3]Il *seed* è il numero che inizializza il generatore di numeri casuali usato durante l'addestramento, per esempio per inizializzare i pesi aggiunti e per mescolare l'ordine dei dati. Ripetere l'addestramento con seed diversi produce modelli leggermente diversi: se i risultati cambiano molto da un seed all'altro, la procedura non è robusta.

limiti e il Capitolo 9 trae le conclusioni. L'Appendice A mostra l'output di tutti i sistemi sulla risposta di esempio descritta nella Sezione 1.1.2.

Capitolo 2

Inferenza locale dei modelli linguistici

Questa tesi costruisce un verificatore che gira interamente sulla macchina dell'utente, e questa scelta condiziona tutte le altre: la dimensione dei modelli, il modo in cui se ne leggono le decisioni, le misure e i confronti. Il capitolo spiega perché l'inferenza locale conta, che cosa la rende possibile oggi e quanto costa. Si parte dalle ragioni per mantenere un modello sul dispositivo e dai modelli open-weight che lo consentono. Si ricava poi un semplice modello di costo dell'inferenza su un singolo dispositivo, che mostra perché leggere un token costa molto meno che scriverlo, e si passa in rassegna l'hardware su cui girano i modelli locali, dal laptop usato in questo lavoro alle macchine desktop dedicate. Seguono il software e la quantizzazione, che portano i modelli alla portata di quell'hardware, e l'energia. Il capitolo si chiude traendo la conseguenza per la verifica, da cui discende il progetto di GroundOne descritto nei capitoli successivi.

2.1 Dai modelli remoti ai modelli locali

La maggior parte delle persone usa i modelli linguistici attraverso servizi remoti. L'input dell'utente viaggia in rete fino a un fornitore, un modello in un datacenter lo elabora e la risposta compie il percorso inverso. Il modello, l'hardware e le regole che governano i dati appartengono tutti al fornitore. Eseguire il modello sul dispositivo dell'utente cambia tutto questo in un colpo solo. Ne derivano cinque proprietà, e ciascuna conta per un assistente RAG che lavora su documenti privati.

La privacy come garanzia tecnica. Con un modello remoto, ogni documento e ogni domanda vengono inviati a una terza parte, e ciò che accade dopo (per quanto tempo vengono conservati, chi vi può accedere, se vengono usati per migliorare

i modelli futuri) è regolato da contratti e policy. L'utente può leggere la policy, ma non può verificare che venga rispettata. Con un modello locale i documenti non lasciano mai la macchina, e la questione non si pone nemmeno: la privacy diventa una proprietà del sistema anziché una promessa. Le rassegne sull'edge intelligence, cioè sull'esecuzione dei modelli su dispositivi periferici vicini all'utente, lo affermano esplicitamente: l'inferenza sul dispositivo elimina le fughe di dati privati e la necessità di una connessione a Internet [25].

Il diritto europeo rende concreta la differenza. Secondo il Regolamento generale sulla protezione dei dati (GDPR), il trattamento dei dati personali comprende esplicitamente la loro "comunicazione mediante trasmissione" [26, Art. 4(2)]. Inviare a un modello remoto un documento che contiene dati personali rende quindi il fornitore un responsabile del trattamento[1], vincolato da un contratto con le garanzie specifiche dell'Articolo 28. Se poi il fornitore opera al di fuori dell'Unione europea, il trasferimento dei dati deve avere una base giuridica: una decisione di adeguatezza[2], garanzie adeguate oppure una delle deroghe previste dagli Articoli 44–49 [26]. Un verificatore eseguito sul dispositivo dell'utente non coinvolge alcun responsabile del trattamento né alcun trasferimento. Non elimina ogni obbligo, perché l'organizzazione che lo usa deve comunque rispettare i principi dell'Articolo 5 e proteggere il dispositivo, ma elimina gli obblighi che dipendono dal doversi fidare di qualcun altro. Molti documenti riservati, come ricerche inedite, contratti o codice sorgente, non contengono affatto dati personali; per essi il regolamento non si applica, e l'argomento diventa di riservatezza.

Un episodio recente mostra che cosa sia in gioco. Nel settembre 2026 OpenAI ha annunciato che un proprio sistema multi-agente interno aveva prodotto una dimostrazione di blow-up (la formazione di una singolarità in tempo finito) per le equazioni di Navier–Stokes forzate, uno dei problemi del Millennio, formalizzata nell'assistente di dimostrazione Lean [7]. Due matematici, Tristan Buckmaster e Levent Alpöge, avevano diffuso più o meno nello stesso periodo risultati affini per le equazioni di Eulero, e ne hanno rivendicato la paternità. Buckmaster aveva scritto le proprie bozze nell'assistente di programmazione della stessa OpenAI, e ha chiesto all'azienda "whether the model had been trained on, or had access to, our sessions in Codex, into which we had been putting all our drafts" [27] (se il modello fosse stato addestrato sulle sessioni in Codex in cui i due avevano inserito tutte le bozze, o avesse avuto accesso a esse). OpenAI ha negato che i propri ricercatori o agenti avessero visto il lavoro prima della pubblicazione, ma ha aggiunto: "While unlikely, we cannot rule out that de-identified data derived from their usage of our products

[1]Nel GDPR il *titolare* del trattamento decide perché e come trattare i dati; il *responsabile* li tratta per conto del titolare, per esempio fornendo un servizio di calcolo.

[2]Una decisione con cui la Commissione europea riconosce che un paese terzo garantisce un livello di protezione dei dati sostanzialmente equivalente a quello europeo.

helped improve our models" [7, 28] (per quanto improbabile, non è possibile escludere che dati de-identificati derivati dal loro utilizzo dei prodotti dell'azienda abbiano contribuito a migliorarne i modelli). Nulla di quanto è pubblicamente noto indica che le bozze siano state usate impropriamente, e la disputa riguarda soprattutto la paternità del risultato. L'episodio mostra però che, una volta che un lavoro inedito è entrato in un modello remoto, né l'utente né, per sua stessa ammissione, il fornitore possono escludere che abbia contribuito ai modelli del fornitore. Con un modello locale la domanda non ha nemmeno senso.

I fornitori sono consapevoli del problema. Apple descrive l'elaborazione sul dispositivo come la pietra angolare del proprio approccio alla privacy e, per le richieste che eccedono le capacità del dispositivo, ha costruito Private Cloud Compute, un cloud dedicato con server su misura, nessun accesso privilegiato per il personale, nessuna conservazione persistente delle richieste e un registro pubblico delle immagini software che vi vengono eseguite [29]. Il progetto è un tentativo serio di rendere verificabile l'inferenza remota, e la sua stessa complessità è un argomento a favore della località. Per garantire la privacy nel cloud servono silicio su misura, un sistema operativo irrobustito e un'attestazione crittografica, cioè una prova firmata di quale software sia in esecuzione sui server. Anche così, il sistema poggia sulla fiducia nell'hardware, nella filiera di fornitura e nell'attestazione stessa, che soltanto i ricercatori di sicurezza, e non gli utenti, sono in grado di ispezionare [29].

Disponibilità. Un modello locale funziona senza rete e senza dipendere dalla disponibilità di un fornitore. Per un assistente usato in treno, in un reparto ospedaliero con connettività limitata o in un ambiente air-gapped, cioè isolato fisicamente da qualunque rete, questo è un requisito, non una comodità.

Stabilità e riproducibilità. I pesi di un modello locale cambiano solo quando è l'utente a cambiarli. Un modello remoto può essere aggiornato, riaddestrato o ritirato in qualunque momento, spesso senza preavviso, così che la stessa richiesta produca risposte diverse da un mese all'altro. Per un verificatore le cui soglie sono calibrate sui dati di sviluppo (Capitolo 4), un cambiamento silenzioso del modello sottostante invaliderebbe la calibrazione senza alcun avviso.

Costo. Un modello remoto si paga a token, e un verificatore che controlla ogni risposta moltiplica il numero di token elaborati. Un modello locale si paga una volta sola, con l'hardware, e poi costa soltanto l'energia che consuma.

Accesso agli stati interni del modello. Un modello locale espone i propri logit, i punteggi da cui si calcola ogni probabilità. Le interfacce remote spesso li nascondono o li restituiscono solo per pochi token. L'approccio studiato in questa

tesi legge la decisione direttamente dai logit di poche lettere di risposta e li calibra, quindi richiede questo accesso; in questo senso è possibile soltanto con un modello locale.

Il prezzo della località. Questi vantaggi hanno un costo. Un laptop ospita solo modelli piccoli, e deve ospitarli accanto al sistema operativo, alle applicazioni dell'utente e a ogni altro componente dell'assistente. Dispone di meno banda di memoria e di meno capacità di calcolo di un acceleratore da datacenter, e funziona a batteria. Una parte della letteratura conclude che un'esecuzione puramente sul dispositivo è troppo limitata e propone di dividere il lavoro con un server edge, cioè un server vicino all'utente, o con il cloud [25, 30]: un modello locale piccolo gestisce le richieste facili e inoltra quelle difficili, oppure scrive una bozza di risposta che un modello remoto più grande controlla. L'abbinamento di Apple tra un modello sul dispositivo e Private Cloud Compute è l'esempio più visibile [4, 29]. Queste soluzioni ibride riducono il costo della località ma ne sacrificano una parte dei benefici, perché tutto ciò che viene inoltrato lascia il dispositivo. Questa tesi adotta la posizione più rigorosa: l'intera verifica resta sul dispositivo, e il resto del capitolo esamina che cosa ciò richieda.

La Tabella 2.1 riassume il confronto.

Tabella 2.1: Inferenza remota e inferenza locale a confronto sulle proprietà che contano per un verificatore di documenti privati.

Proprietà	Modello remoto	Modello locale
Privacy	Contrattuale: i documenti sono inviati a un responsabile del trattamento; l'utente non può verificare come vengano usati.	Tecnica: i documenti non lasciano mai il dispositivo.
Normativa (GDPR)	Contratto con il responsabile (Art. 28); trasferimenti verso paesi terzi (Artt. 44–49).	Nessun responsabile e nessun trasferimento; restano i principi dell'Art. 5 e la sicurezza del dispositivo.
Disponibilità	Richiede la rete e l'operatività del fornitore.	Funziona offline.
Stabilità	Il fornitore può aggiornare o ritirare il modello.	I pesi cambiano solo se è l'utente a cambiarli.
Costo	Prezzo per token.	Hardware una tantum, poi energia.
Logit	Spesso nascosti o limitati.	Pienamente accessibili: readout e calibrazione possibili.
Capacità	Modelli di frontiera.	Modelli piccoli, limitati da memoria, banda e batteria.

2.2 Modelli open-weight e small language model

L'inferenza locale richiede un modello i cui pesi si possano scaricare. Questa sezione descrive che cosa siano questi modelli, quanto distino dai migliori modelli chiusi e quali siano abbastanza piccoli per un dispositivo personale.

2.2.1 Open-weight non significa open source

Un modello si dice *open-weight* quando i suoi parametri addestrati sono pubblicati e chiunque può eseguirli e modificarli. Non è la stessa cosa che essere open source. La Open Source Initiative definisce un sistema di IA open source attraverso le quattro libertà della tradizione del software libero (usare, studiare, modificare e condividere il sistema) e richiede che sia disponibile la forma preferita per modificarlo: i parametri, il codice completo usato per addestrarlo ed eseguirlo, e informazioni sui dati di addestramento sufficienti perché una persona esperta possa costruire un sistema sostanzialmente equivalente [31]. I dati in sé non devono necessariamente essere rilasciati, ma i soli pesi non bastano. La maggior parte dei modelli comunemente definiti open pubblica i pesi e una ricetta per l'inferenza, ma né il codice di addestramento né una descrizione dettagliata dei dati. Essere open è quindi una questione di grado, non una proprietà binaria. Un indice recente valuta i modelli su cinque aspetti: la disponibilità dei pesi, la documentazione dei dati di pre-training e di post-training (le due fasi dell'addestramento, su testo generico e poi su istruzioni e preferenze), la metodologia e la licenza [32]. Nessun modello raggiunge il punteggio massimo; i modelli di ricerca interamente documentati, come OLMo, ottengono punteggi alti, mentre alcuni modelli open-weight molto diffusi pubblicano poco oltre ai pesi.

Le licenze aggiungono un'ulteriore distinzione. I due modelli sottoposti a fine-tuning in questa tesi appartengono alla stessa famiglia ma hanno licenze diverse: Qwen2.5-0.5B è rilasciato con la licenza permissiva Apache 2.0, mentre Qwen2.5-3B è rilasciato con una licenza di ricerca che ne limita l'uso commerciale [23]. Nella terminologia appena introdotta entrambi sono modelli open-weight, ed è questo il termine che la tesi usa per indicarli. La distinzione conta anche sul piano normativo. L'AI Act europeo attenua alcuni obblighi di trasparenza per i modelli general-purpose rilasciati con licenze libere e open source, ma non per quelli che presentano un rischio sistemico. Un modello si presume a rischio sistemico quando il suo addestramento ha richiesto più di 10^{25} operazioni in virgola mobile (FLOP[3]) [33], molte più di quelle usate per i modelli di questa tesi.

[3]Un FLOP è una singola operazione aritmetica su numeri in virgola mobile, come una somma o una moltiplicazione. La velocità di calcolo si misura in FLOP al secondo: un TFLOP/s corrisponde a 10^{12} operazioni al secondo.

2.2.2 Il divario con i modelli chiusi si sta chiudendo

L'inferenza locale avrebbe un interesse limitato se i modelli open-weight fossero molto indietro rispetto a quelli chiusi. Non lo sono. Sulla leaderboard Chatbot Arena, dove gli utenti confrontano le risposte di due modelli anonimi, il vantaggio del miglior modello chiuso sul miglior modello open-weight è sceso dall'8.04 % del gennaio 2024 all'1.70 % del febbraio 2025 [6]. Sul capabilities index di Epoch AI, che aggrega molti benchmark, i migliori modelli open-weight sono rimasti indietro rispetto alla frontiera chiusa di circa quattro mesi in media tra gennaio e maggio 2026 (Figura 2.1) [34].

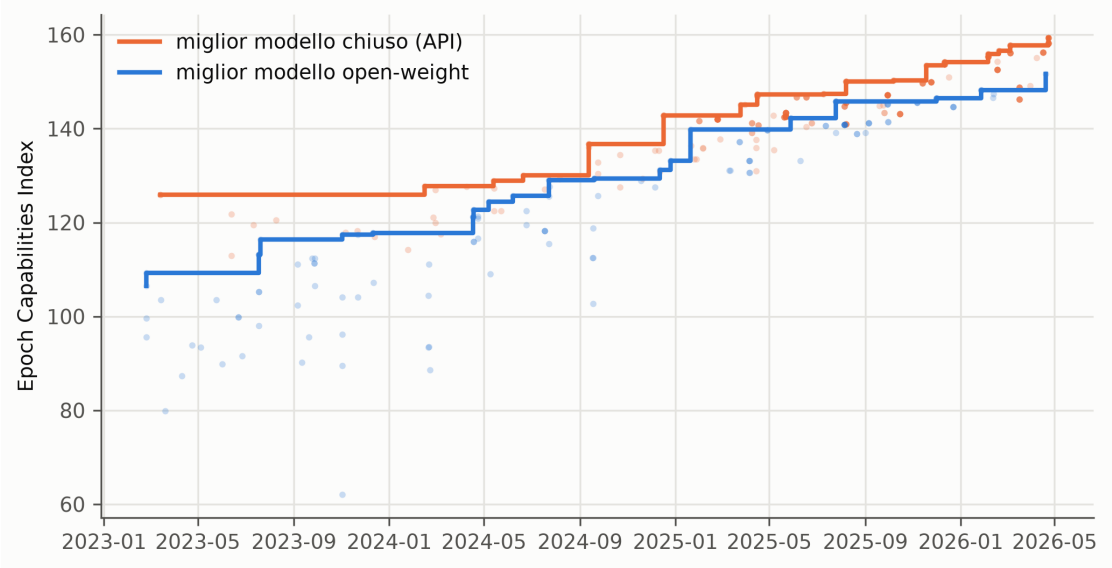


Figura 2.1: Capacità del miglior modello chiuso e del miglior modello open-weight nel tempo, sull'indice aggregato di capacità di Epoch AI (punti: singoli modelli; linee: migliore fino a quel momento). Grafico ricavato dalla tabella dati pubblicata da Epoch AI [34].

Per un dispositivo personale non conta il miglior modello open-weight, che può avere centinaia di miliardi di parametri, ma il miglior modello che entra in un hardware consumer. Epoch AI misura direttamente questa grandezza. Considerando i modelli abbastanza piccoli da girare su una singola scheda grafica consumer, al massimo circa 28 miliardi di parametri per una scheda da 24 GB e circa 40 miliardi per una da 32 GB, rileva che su quattro benchmark essi raggiungono le prestazioni del modello di frontiera del momento dopo sei-dodici mesi [5]. Tre forze trainano il recupero: gli sviluppatori di modelli open-weight aumentano il calcolo di addestramento più o meno alla stessa velocità di quelli chiusi, la distillazione, cioè l'addestramento di un modello piccolo a imitare le risposte di uno grande, ne

trasferisce gran parte delle capacità, e ogni nuova generazione di schede grafiche aumenta la dimensione dei modelli che vi trovano posto [35]. La stessa tendenza emerge dal lato opposto: il modello più piccolo con un punteggio superiore al 60 % sul benchmark MMLU, un insieme di domande a scelta multipla su 57 materie, aveva 540 miliardi di parametri nel 2022 e 3.8 miliardi nel 2024, una riduzione di 142 volte [6].

2.2.3 Small language model

All'estremo inferiore della scala si trovano gli *small language model*. Le definizioni variano; una rassegna che ne misura diverse decine sui dispositivi colloca l'intervallo tra 100 milioni e 5 miliardi di parametri, con architettura decoder-only, cioè quella dei comuni LLM che generano il testo da sinistra a destra [36]. I modelli piccoli vengono addestrati su molti più dati di quanto converrebbe se si volesse soltanto minimizzare il costo dell'addestramento. Il motivo è che per essi il costo che conta è l'inferenza, ripetuta milioni di volte dopo l'addestramento: conviene quindi spendere di più una volta sola per ottenere un modello piccolo ma capace. Un modello da mezzo miliardo di parametri può così vedere migliaia di miliardi di token [36]. Misurati su dispositivi edge con pesi a 4 bit, i modelli tra 0.1 e 3 miliardi di parametri richiedono tra 0.3 e 2.5 GB di memoria, e con contesti lunghi la maggior parte della memoria è occupata dalla KV cache, che conserva il contesto già letto (Sezione 2.3.1), e dai buffer di calcolo [36]. Le ricerche sui modelli sotto il miliardo di parametri sostengono che, sugli smartphone, un'applicazione non dovrebbe usare più di circa un decimo della memoria del dispositivo, e mostrano che un modello da 350 milioni di parametri progettato con cura eguaglia un modello generalista da 7 miliardi su un task ristretto come l'invocazione di un'interfaccia applicativa [37].

I produttori di piattaforme hanno adottato la stessa idea. Il foundation model che Apple impiega sui propri dispositivi ha circa tre miliardi di parametri, è quantizzato a meno di quattro bit per peso ed è specializzato per ciascuna funzionalità con piccoli adapter low-rank, moduli aggiuntivi con pochi parametri addestrati sul singolo compito (Sezione 3.5) [4]; la famiglia Gemma 3n di Google è progettata per girare su smartphone e laptop con un'impronta di memoria ridotta, ed è promossa esplicitamente per la privacy e l'uso offline [38]. GroundOne segue lo stesso schema: un modello open-weight piccolo, quantizzato e specializzato da un adapter per un unico task.

2.3 Il costo dell'inferenza su un singolo dispositivo

Le sezioni precedenti spiegano perché un modello dovrebbe girare in locale e quali modelli possono farlo. Questa sezione spiega quanto costa eseguirne uno, e perché il costo dipende così tanto da ciò che gli si chiede di fare.

2.3.1 Prefill e decoding

Un modello linguistico decoder-only elabora una richiesta in due fasi. Nella fase di *prefill* legge l'intero prompt in una volta. Per farlo usa il meccanismo di attenzione, con cui ogni token raccoglie informazioni dai token che lo precedono: ogni strato calcola, per ciascun token di input, due vettori (una *chiave* e un *valore*) che i token successivi consulteranno, e li memorizza nella *KV cache* (dall'inglese key-value). Nella fase di *decoding* produce l'output un token alla volta; ogni nuovo token consulta la cache attraverso l'attenzione e la estende di una posizione [10]. Il prefill elabora molti token in parallelo, il decoding uno solo. I sistemi che servono i modelli a molti utenti sfruttano questa differenza in vari modi, per esempio gestendo la cache a pagine per accogliere più richieste concorrenti [39] oppure eseguendo le due fasi su macchine diverse, ciascuna adatta alla propria [40].

La conseguenza è visibile sul laptop usato in questa tesi (Figura 2.2). Con il modello da 0.5B, leggere un token del prompt richiede meno di 0.4 millisecondi, mentre scrivere un token di output ne richiede 5.9, sedici volte di più. Con il modello da 3B i valori sono 2.2 e 31.8 millisecondi, quattordici volte di più. Un modello che scrive un breve paragrafo impiega quindi più tempo a scriverlo che a leggere un documento di diverse migliaia di token.

2.3.2 Il modello roofline

La ragione di questa asimmetria è colta dal modello roofline, un semplice modello delle prestazioni introdotto per i processori multicore [41]. Una computazione è caratterizzata dalla sua *intensità aritmetica*, cioè dal numero di operazioni in virgola mobile che esegue per ogni byte letto dalla memoria. Un processore ha due limiti: una velocità di calcolo di picco e una banda di memoria di picco. La velocità raggiungibile è il minimo tra il calcolo di picco e il prodotto tra banda e intensità. Una computazione a bassa intensità è limitata dalla banda, una ad alta intensità dal calcolo; l'intensità alla quale i due limiti si incontrano si chiama punto di cresta (ridge point) [41].

Applicato a un transformer, il modello separa nettamente le due fasi [42]. Ogni matrice di pesi di uno strato lineare deve essere letta dalla memoria una volta per forward pass. Durante il prefill la stessa matrice moltiplica gli stati nascosti

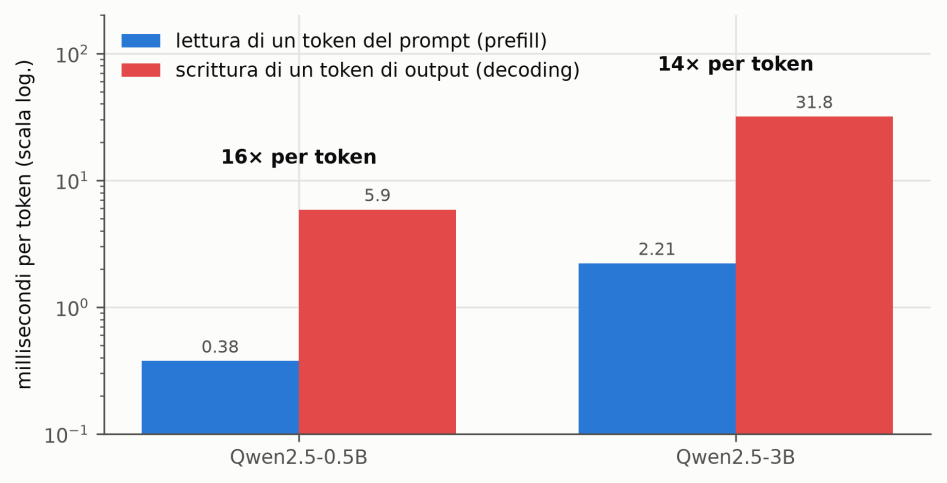


Figura 2.2: Tempo per token quando il modello legge l'input (prefill) e quando scrive l'output (decoding), misurato sulla macchina della tesi (Apple M4, 16 GB, MLX a 8 bit, una richiesta alla volta). Prefill: mediana su contesti di 500, 1000 e 2000 token.

di centinaia o migliaia di token, quindi ogni byte di pesi serve per centinaia di operazioni: l'intensità è alta e il limite è il calcolo. Durante il decoding la matrice moltiplica un singolo vettore, quindi ogni peso serve per circa due operazioni: l'intensità è vicina a uno e il limite è la banda. Per un modello da 7 miliardi di parametri a 16 bit, gli strati lineari raggiungono intensità superiori a mille operazioni per byte nel prefill e di circa una nel decoding [42]. Nei datacenter questo squilibrio si attenua: il decoding di molte richieste contemporanee viene raggruppato in batch, così che ogni peso letto dalla memoria serve a generare più token e l'intensità sale [10]. Un dispositivo personale serve invece un solo utente, una richiesta alla volta, e non può contare su questo rimedio: è qui che l'asimmetria tra le due fasi pesa di più.

La Figura 2.3 traccia il roofline della macchina della tesi. Apple pubblica la banda di memoria, 120 GB/s, ma non la velocità di calcolo di picco [43]; il tetto piatto della figura è quindi la velocità effettivamente raggiunta durante il prefill, circa 2.8 TFLOP/s, che costituisce un limite inferiore del picco. Il ridge point si trova a circa 23 operazioni per byte. La generazione di un token si colloca a circa due operazioni per byte, un ordine di grandezza sotto il ridge point, mentre la lettura di un documento di centinaia di token si colloca un ordine di grandezza sopra. Le due fasi dello stesso modello, sulla stessa macchina, vivono in regimi diversi.

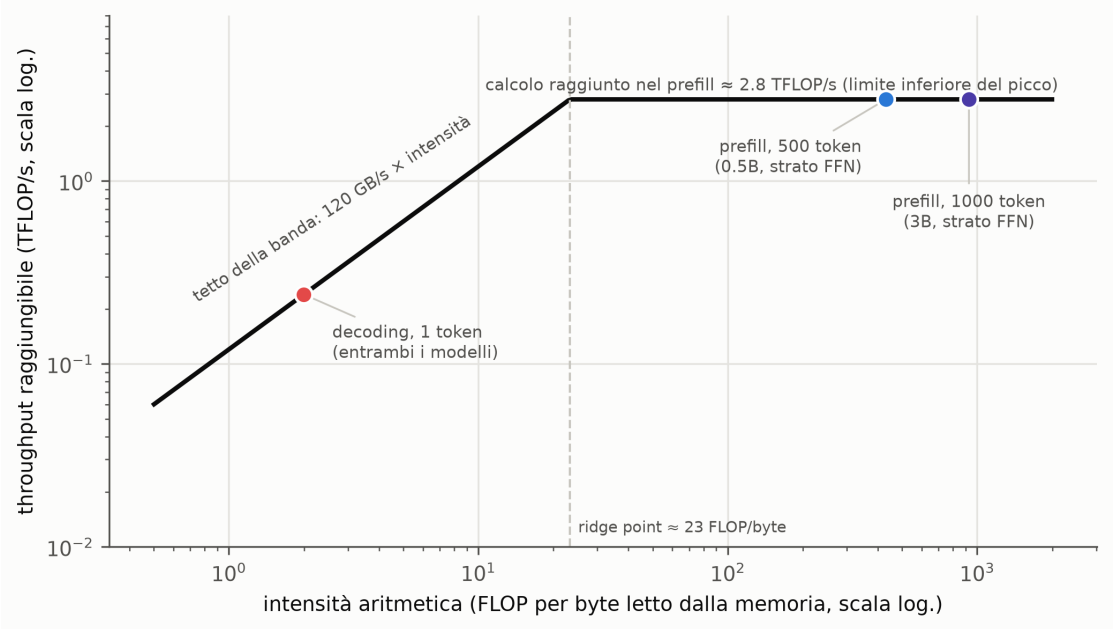


Figura 2.3: Modello roofline della macchina della tesi [41]. Tetto inclinato: banda di memoria dichiarata da Apple (120 GB/s). Tetto piatto: throughput raggiunto nel prefill, limite inferiore del picco non pubblicato. Intensità calcolate per uno strato feed-forward di Qwen2.5 con pesi a 8 bit e attivazioni a 16 bit; l'attenzione è trascurata.

2.3.3 Un limite inferiore alla generazione

Poiché il decoding legge ogni peso una volta per ogni token generato, il suo tempo per token non può scendere sotto la dimensione dei pesi divisa per la banda di memoria, qualunque sia l'hardware. Per il modello da 3B a 8 bit, i cui pesi occupano circa 3.3 GB, questo limite vale 27 millisecondi per token su una macchina da 120 GB/s; i 31.8 millisecondi misurati lo superano solo di circa un sesto. Il limite dipende da due soli numeri, la dimensione del modello e la banda della macchina, e si applica a ogni modello che genera testo. È la chiave per confrontare l'hardware nella Sezione 2.4 e per la conseguenza tratta nella Sezione 2.7.

La capacità di memoria impone un secondo limite, ancora più rigido: i pesi devono entrare in memoria. Un modello richiede circa due byte per parametro nel formato a 16 bit usato in addestramento, uno a 8 bit e mezzo a 4 bit, più alcuni piccoli fattori di scala per ogni gruppo di pesi, necessari a ricostruirne il valore (Figura 2.4). Un modello da 7 miliardi di parametri a 16 bit richiede 14 GB per i soli pesi, più di quanto la maggior parte dei dispositivi personali possa mettere a disposizione [44]; a 4 bit ne richiede circa 4. A questi si aggiungono la KV cache, le attivazioni, il sistema operativo e ogni altro modello in esecuzione nello stesso

momento. Un laptop da 16 GB può quindi ospitare un generatore da 7–8 miliardi di parametri a 4 bit accanto a un piccolo verificatore, ma non molto di più.

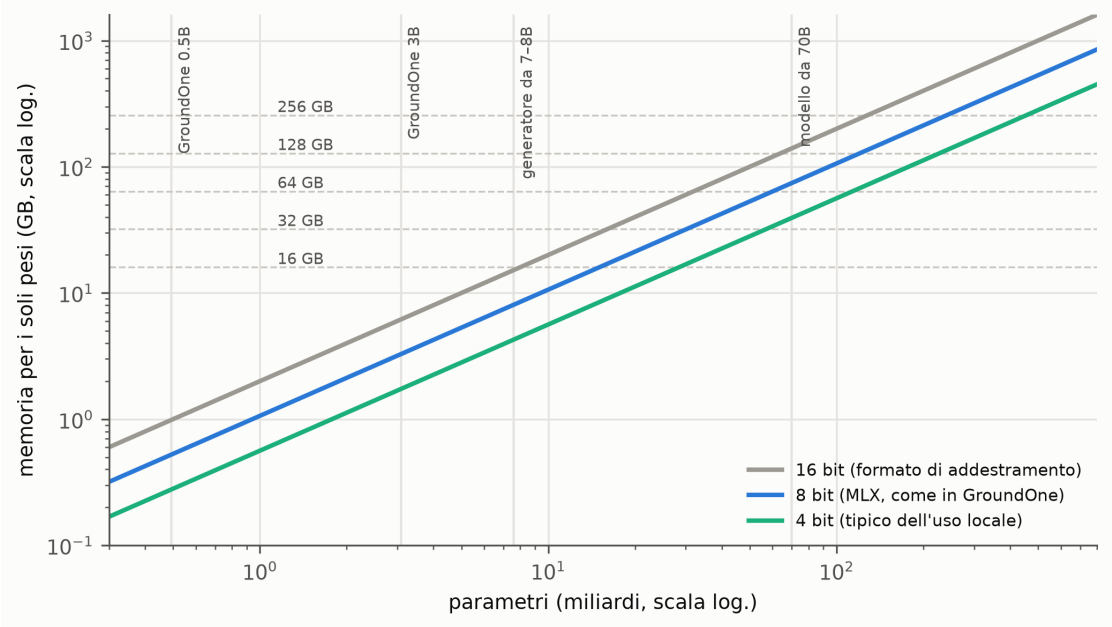


Figura 2.4: Memoria necessaria per i pesi di un modello in funzione della sua dimensione e della precisione (numero di parametri moltiplicato per i bit per peso, compreso mezzo bit di scale per gruppo nei formati quantizzati). Linee tratteggiate: tagli di memoria comuni nell'hardware locale. La KV cache, le attivazioni e gli altri modelli in esecuzione si aggiungono a questo valore.

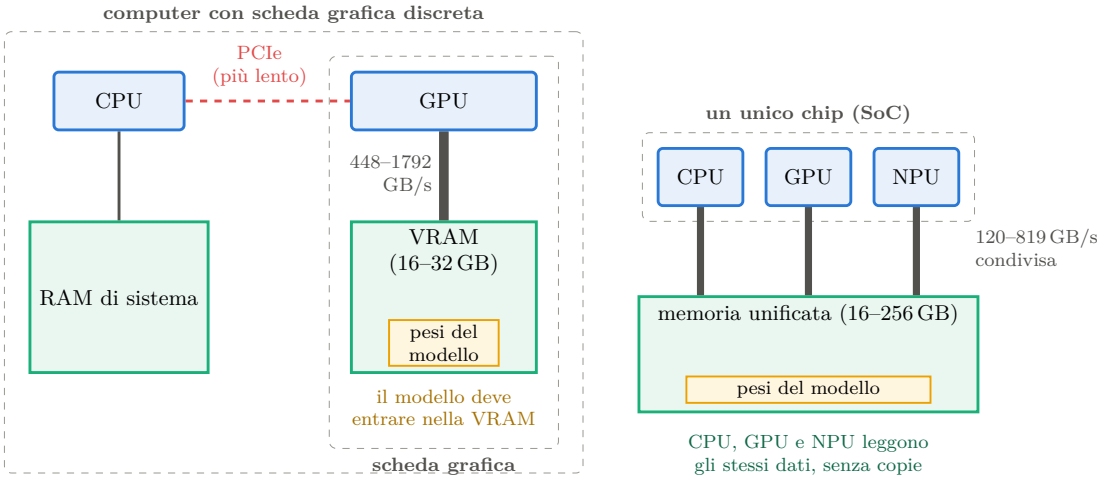
2.4 L'hardware per l'inferenza locale

Il modello di costo riduce la scelta dell'hardware per l'inferenza locale a tre grandezze. La *capacità di memoria* stabilisce quali modelli entrano. La *banda di memoria* stabilisce quanto velocemente possono generare. La *capacità di calcolo* stabilisce quanto velocemente possono leggere. Potenza e prezzo completano il quadro. Questa sezione passa in rassegna l'hardware disponibile a ottobre 2026 lungo questi assi, sulla base delle specifiche pubblicate dai produttori.

2.4.1 Classi di hardware

Dominano due architetture. In un sistema *discreto* la scheda grafica ha una propria memoria, veloce ma piccola, e il modello deve entrare in essa. In un sistema a *memoria unificata* il processore e l'unità grafica condividono un unico, ampio blocco

di memoria, così che un modello può usarne la maggior parte, con una banda però inferiore a quella di una scheda grafica di fascia alta. I processori di Apple, il DGX Spark di NVIDIA e il Ryzen AI Max (Strix Halo) di AMD seguono tutti questa seconda architettura. La Figura 2.5 mette a confronto le due architetture.



(a) Sistema discreto: la GPU legge i pesi dalla propria VRAM, molto veloce ma limitata a 16–32 GB; il resto del sistema è raggiungibile solo attraverso il PCIe.

(b) Memoria unificata: CPU, GPU e NPU sullo stesso chip condividono un unico blocco di memoria, più ampio ma con una banda inferiore.

Figura 2.5: Le due architetture dell'hardware per l'inferenza locale. Rettangoli blu: unità di calcolo; rettangoli verdi: memoria; rettangolo arancione: i pesi del modello; spessore delle linee: banda indicativa. Valori di capacità e banda dalla Tabella 2.2. In un sistema discreto un modello che non entra nella VRAM deve passare dal collegamento PCIe, molto più lento; in un sistema a memoria unificata il modello può occupare quasi tutta la memoria della macchina, ma ogni token generato la legge a una velocità inferiore a quella di una scheda grafica di fascia alta.

La Tabella 2.2 e la Figura 2.7 collocano le principali opzioni sui due assi della memoria. La Figura 2.6 mostra alcuni dei sistemi citati, per dare un'idea concreta di come si presentano.

Laptop. I laptop con 16–32 GB e 120–150 GB/s, come il MacBook Pro usato in questa tesi, sono i computer personali più diffusi su cui girano modelli locali. Ospitano senza difficoltà i modelli piccoli, e quelli da 7–8 miliardi di parametri a 4 bit. La macchina della tesi si trova in basso a sinistra nella Figura 2.7: se un verificatore funziona bene lì, funziona su tutto ciò che sta sopra.



(a) MacBook Pro da 14 pollici con M4, la macchina della tesi.



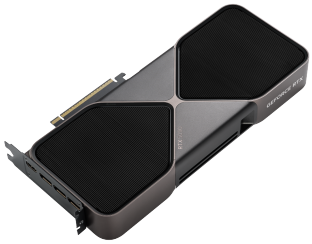
(b) Mac mini (2024, M4).



(c) Mac Studio.



(d) NVIDIA DGX Spark.



(e) Scheda grafica GeForce RTX 5090 Founders Edition.

Figura 2.6: Alcuni dei sistemi per l'inferenza locale confrontati in questa sezione (immagini non in scala). Fonti: (a), (c) Apple; (b) LoMit, Wikimedia Commons, CC0; (d), (e) NVIDIA. Le immagini dei produttori sono materiale promozionale, riprodotto a scopo di documentazione scientifica.

Macchine desktop a memoria unificata. Una seconda classe, entro o vicino a un budget di circa tremila dollari, offre molta più memoria con una banda moderata. Il processore Strix Halo di AMD, venduto per esempio nel Framework Desktop, consente all'unità grafica di usare fino a 96 GB dei suoi 128 GB di memoria; al lancio, nell'agosto 2025, la configurazione da 128 GB costava 1999 dollari, con una potenza del processore di 55 W [49]. Il Mac mini di Apple con M5 Pro arriva a 64 GB a 307 GB/s [46]. Macchine di questo tipo possono ospitare modelli da decine di miliardi di parametri o modelli mixture-of-experts, composti da molte sottoreti (gli esperti) di cui ogni token ne attiva solo una parte, così che il decoding legge soltanto i pesi degli esperti attivi, al prezzo di una generazione più lenta che su una scheda grafica.

Macchine dedicate all'IA. Oltre questo budget, il Mac Studio con M3 Ultra unisce fino a 256 GB a 819 GB/s [47], mentre il DGX Spark di NVIDIA è un piccolo box desktop, con alimentatore da 240 W, che contiene 128 GB di memoria unificata

Tabella 2.2: Hardware per l'inferenza locale: specifiche pubblicate dai produttori (consultate l'11 ottobre 2026). Memoria: intervallo delle configurazioni offerte; banda: relativa a quelle configurazioni; n.d.: non dichiarato dalla fonte.

Sistema	Memoria (GB)	Banda (GB/s)	Tipo di memoria	Fonte
MacBook Pro, M4 (macchina della tesi: 16 GB)	16–32	120	unificata	[43]
MacBook Pro / Mac mini, M5	16–32	153	unificata	[45, 46]
Mac mini, M5 Pro	24–64	307	unificata	[46]
MacBook Pro, M5 Max	36–128	460–614	unificata	[45]
Mac Studio, M4 Max	36–128	410–546	unificata	[47]
Mac Studio, M3 Ultra	96–256	819	unificata	[47]
NVIDIA DGX Spark (GB10)	128	273	unificata	[48]
Framework Desktop, AMD Strix Halo	32–128 (96 alla GPU)	n.d.	unificata	[49]
NVIDIA RTX 5090	32	1792	discreta	[50]
NVIDIA RTX 5080	16	960	discreta	[50]
NVIDIA RTX 5060 Ti	16	448	discreta	[50]

con 273 GB/s di banda e un'unità grafica Blackwell da un petaflop (10^{15} operazioni al secondo) in aritmetica a 4 bit, valore che sfrutta la sparsità, cioè il salto dei pesi nulli [48]. Le due macchine compiono scelte opposte, e un confronto diretto mostra il modello di costo all'opera: su un modello da 8 miliardi di parametri con un prompt di 8000 token, il DGX Spark completa il prefill 3.8 volte più velocemente del Mac Studio, grazie al calcolo più elevato, mentre il Mac Studio genera 3.4 volte più velocemente, grazie a una banda tre volte superiore [51]. Lo stesso studio combina le due macchine, eseguendo il prefill sullo Spark e il decoding sul Mac: è la disaggregazione delle due fasi [40] portata su una scrivania.

Schede grafiche. Le schede dedicate offrono di gran lunga la banda più alta, fino a 1792 GB/s per la RTX 5090, ma al massimo 32 GB di memoria, con una

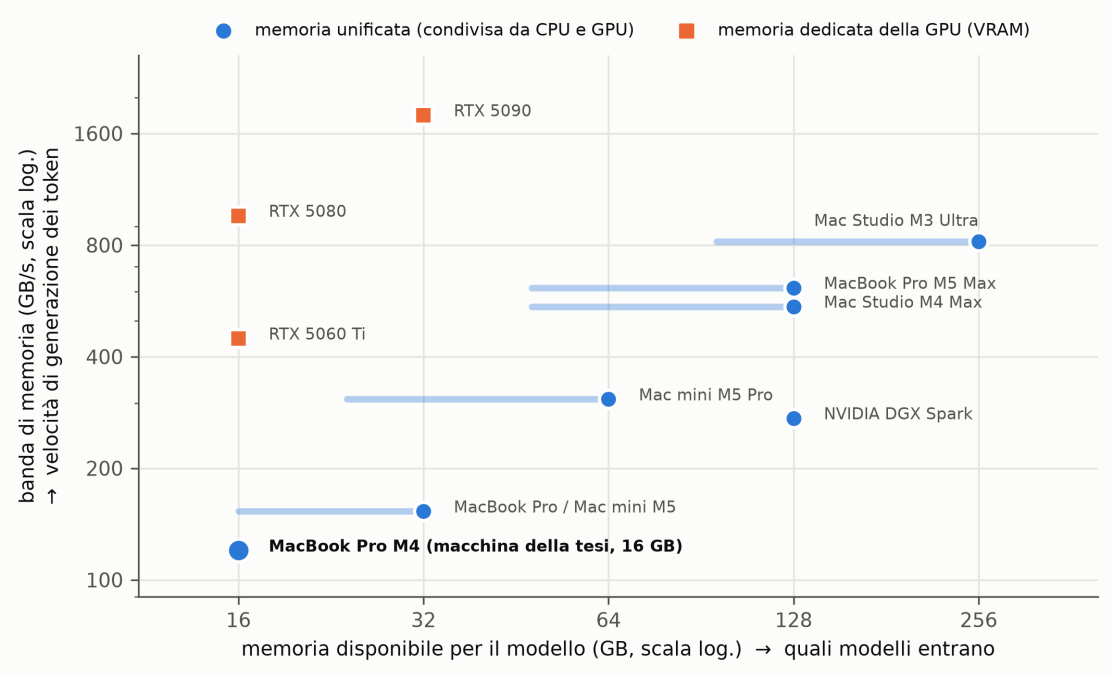


Figura 2.7: L'hardware per l'inferenza locale sui due assi che stabiliscono che cosa può girare: capacità di memoria (quali modelli entrano) e banda di memoria (quanto velocemente si generano i token). Specifiche dei produttori: marcatori in corrispondenza della configurazione con più memoria, barre ombreggiate sull'intervallo di tagli di memoria offerti; la macchina della tesi è rappresentata con i suoi 16 GB effettivi.

potenza di diverse centinaia di watt [50]. Sono l'opzione più veloce per qualunque modello vi entri, e inutilizzabili per qualunque modello non vi entri.

Smartphone e processori mobili. All'estremo opposto, gli smartphone hanno da 6 a 16 GB di memoria e una banda di qualche decina di gigabyte al secondo, condivisa da tutti i processori [30, 52]. Le loro unità di elaborazione neurale (NPU) sono veloci nelle moltiplicazioni di matrici a 8 bit, e quindi nel prefill: su uno smartphone, un modello da 7 miliardi di parametri a 4 bit legge il prompt a circa 770 token al secondo sulla NPU, contro 37 sull'unità grafica e 8 sui core del processore [52]. I sistemi di ricerca ne approfittano eseguendo il prefill sulla NPU e il decoding altrove [53].

2.4.2 L'asimmetria vale per ogni classe

Le due fasi dipendono da proprietà diverse dell'hardware, e le generazioni più recenti lo rendono esplicito. Apple riporta che il processore M5, che aggiunge unità matriciali dedicate a ogni core grafico, riduce di 3.3–4.1 volte rispetto all'M4 il tempo al primo token, cioè l'attesa prima che il modello inizi a rispondere, dominata dal prefill, mentre la generazione diventa soltanto 1.19–1.27 volte più veloce, in linea con una banda di memoria passata da 120 a 153 GB/s [54]. È la stessa Apple ad attribuire il primo dato al calcolo e il secondo alla banda. Sui dispositivi edge, uno studio che confronta 18 coppie di configurazioni rileva che i rallentamenti si accumulano nel decoding, mentre il tempo di valutazione del prompt varia al massimo del 16 % [55]. Sugli smartphone, per i task con output breve, il prefill occupa la maggior parte della latenza [53]. L'hardware si sta quindi evolvendo per leggere più velocemente, molto più che per scrivere più velocemente.

La Figura 2.8a applica il limite inferiore della Sezione 2.3.3 all'hardware della Tabella 2.2. Sulla scheda più veloce il modello da 3B potrebbe in linea di principio generare un token in 1.8 millisecondi; sulla macchina della tesi il limite è di 27 millisecondi, e il tempo misurato vi si avvicina. Un hardware migliore abbassa il limite, ma nessun hardware lo elimina.

2.5 Runtime e quantizzazione

2.5.1 Runtime

Il software per l'inferenza locale è maturato rapidamente. llama.cpp offre inferenza in C e C++ senza dipendenze esterne, con quantizzazione e supporto per la maggior parte degli acceleratori consumer [2]. Ha inoltre definito il formato GGUF, che raccoglie in un unico file il modello quantizzato e tutto ciò che serve per usarlo, come il tokenizer e il formato delle conversazioni. Strumenti come Ollama e LM Studio si basano su di esso e riducono a un solo comando lo scaricamento e l'avvio di un modello. MLX è un framework di calcolo progettato da Apple per la memoria unificata dei propri processori, con un'interfaccia vicina a NumPy [3]. Il suo pacchetto per modelli linguistici supporta la quantizzazione e il fine-tuning con LoRA direttamente sul dispositivo [56], ed è il runtime usato in tutta questa tesi.

Il runtime conta quanto il chip. Un confronto tra cinque runtime sulla stessa macchina Apple riporta velocità di decoding che differiscono di oltre un ordine di grandezza, con MLX e MLC-LLM in cima e il backend Metal generico di PyTorch in fondo [57]; i numeri dello studio non sono del tutto coerenti tra loro e vanno letti come indicativi, ma la dispersione è abbastanza ampia da resistere all'incertezza. Sugli smartphone, una valutazione di diversi runtime e dispositivi rileva grandi

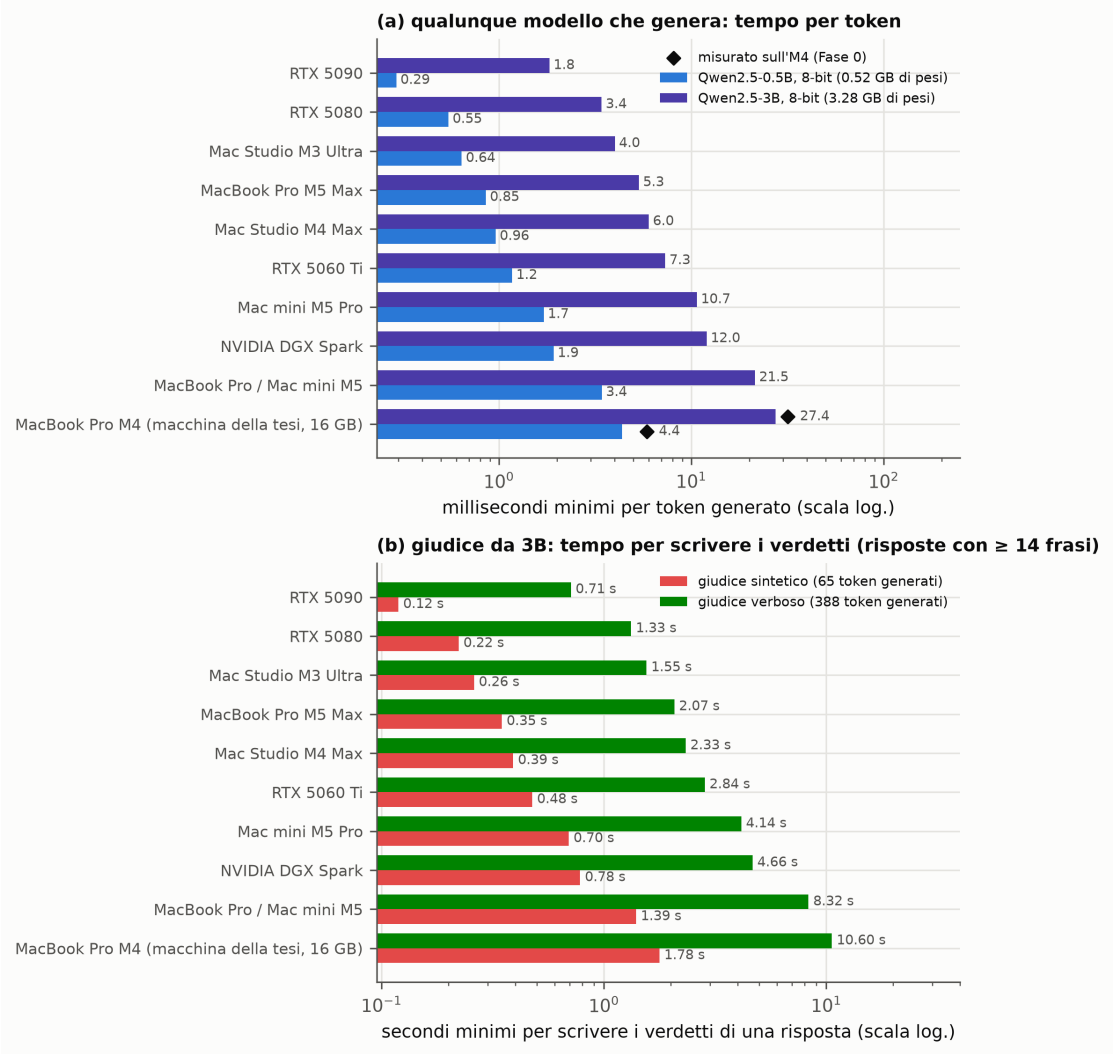


Figura 2.8: Limiti inferiori al costo della generazione di testo su ciascuna classe di hardware locale, calcolati come byte dei pesi divisi per la banda dichiarata dal produttore (una richiesta alla volta; KV cache trascurata). (a) Tempo per token generato con i due modelli base di GroundOne a 8 bit; rombi: tempo di decoding misurato sulla macchina della tesi. (b) Tempo minimo che un giudice generativo da 3B impiega soltanto per scrivere i verdetti di una risposta di quattordici o più frasi (mediana dei token generati nel protocollo di latenza). Un decision model non genera alcun token, quindi per esso entrambi i costi sono nulli su ogni macchina.

differenze di velocità, energia e comportamento termico sia tra runtime sia tra dispositivi [58].

2.5.2 Motori di inferenza specializzati

Accanto ai runtime generali è comparsa, nel 2026, una seconda famiglia di software: motori di inferenza scritti per pochi modelli, che rinunciano alla generalità per spingere più in là i limiti di memoria descritti nella Sezione 2.3. DwarfStar (ds4), sviluppato da Salvatore Sanfilippo, è un motore nativo ottimizzato per alcuni grandi modelli open-weight, a partire da DeepSeek V4 Flash, e pensato per Mac con almeno 96 GB di memoria, per il DGX Spark e per le macchine Strix Halo [59]. Due sue scelte riguardano da vicino questa tesi. La prima è una KV cache salvata su disco: una conversazione già letta si può riprendere senza rifare il prefill, che su modelli di queste dimensioni è la parte più costosa di una richiesta lunga. La seconda è la lettura dei pesi direttamente dall'SSD, che consente di eseguire modelli più grandi della memoria disponibile, al prezzo della velocità.

Colibri porta questa seconda idea all'estremo [60]. Sfrutta il fatto che un modello mixture-of-experts usa per ogni token solo una piccola parte dei propri pesi: tiene in memoria la parte sempre attiva e legge gli esperti dal disco quando il router li richiede. In questo modo GLM-5.2, un modello da 744 miliardi di parametri, gira su un laptop con 25 GB di memoria. Il prezzo è quello previsto dal modello di costo: la banda di un SSD è di pochi gigabyte al secondo, molto inferiore a quella della memoria, e secondo gli autori la generazione scende a 0.05–0.1 token al secondo a freddo su quel laptop e a 1.83 token al secondo su una macchina Strix Halo da 128 GB [60]. È la stessa idea di LLM in a flash [44], applicata ai modelli a esperti: la capacità di memoria smette di essere un limite rigido e si trasforma in un limite di banda, ancora più severo. La Figura 2.9 illustra il meccanismo.

Colibri interessa questa tesi anche per un secondo motivo. Espone la stessa interfaccia System One di Jev, con la quale qualunque modello che esegue risponde a domande chiuse leggendo la probabilità di ciascuna opzione invece di generare la risposta, e include in modo nativo tre decision model: Laya, GLiNER2.5-Decide e Clef [60]. Gli autori riportano che, con Qwen3.6, quattro domande sullo stesso documento ricevono risposta 5.7 volte più velocemente che generando le stesse risposte sulla stessa CPU. Si tratta di una misura dichiarata dal progetto e non verificata in questa tesi, ottenuta con modello e hardware diversi, ma va nella stessa direzione dei risultati del Capitolo 7. Che un motore pensato per eseguire modelli enormi su hardware modesto adotti la lettura delle decisioni come modalità di prima classe conferma che l'idea del decision model si sta diffondendo proprio dove generare costa di più.

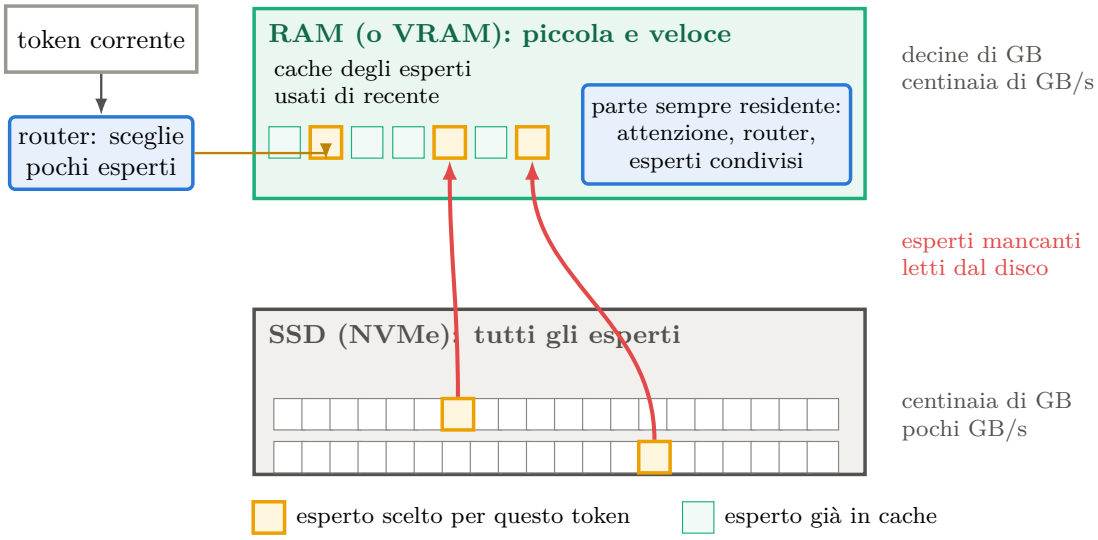


Figura 2.9: Lo streaming degli esperti dall'SSD in un motore come Colibri. In memoria restano la parte del modello usata da ogni token (attenzione, router, esperti condivisi) e una cache degli esperti usati di recente; tutti gli esperti stanno sull'SSD. Per ogni token il router sceglie pochi esperti: quelli già in cache si usano subito, quelli mancanti si leggono dal disco, con una banda di pochi gigabyte al secondo. Per esempio, GLM-5.2 tiene residenti circa 10 GB e lascia sul disco circa 429 GB di pesi [60]. Lo schema non è in scala.

2.5.3 Quantizzazione

La quantizzazione memorizza i pesi con meno bit dei 16 usati in addestramento. Sull'hardware locale serve a due scopi contemporaneamente: il modello occupa meno memoria e il decoding diventa più veloce, perché il suo tempo è dominato dalla lettura dei pesi. La Tabella 2.3 riassume i metodi che hanno segnato il campo.

Due risultati di questa letteratura contano per la tesi. Il primo è che la quantizzazione aiuta la fase limitata dalla banda. GPTQ attribuisce interamente i propri speed-up ai prodotti matrice-vettore del decoding e dichiara di non ottenere alcuna riduzione del calcolo [62]; AWQ illustra lo stesso punto con un roofline: su una scheda grafica desktop, un prefill di 200 token richiede 10 millisecondi, mentre generarne 20 ne richiede 310 [63]. Ridurre i bit dei pesi accelera quindi un modello che genera molto più di uno che si limita a leggere. Il secondo è che i modelli piccoli sono più fragili. GPTQ perde molta più accuratezza a 4 bit su un modello da 1.3 miliardi di parametri che su uno da 175 miliardi, e sugli smartphone i modelli da tre miliardi di parametri o meno diventano inutilizzabili a 3 bit [62, 58]. LLM.int8() mostra che un pugno di feature outlier controlla una quota rilevante della massa di probabilità dell'attenzione [61]. Per un modello il cui output è una probabilità da

Tabella 2.3: Metodi di quantizzazione dei pesi per i modelli linguistici.

Metodo	Idea e risultato principale	Fonte
LLM.int8()	Pesi e attivazioni a 8 bit; le poche dimensioni con valori anomali molto grandi (outlier) restano a 16 bit. Nessuna perdita fino a 175B parametri; fa risparmiare memoria ma non accelera i modelli piccoli	[61]
GPTQ	Pesi a 3–4 bit, quantizzati strato per strato stimando quanto ogni peso influisca sull'errore. OPT-175B a 4 bit perde 0.03 di perplexity[a]; i modelli piccoli perdono di più; decoding 3.2–4.5× più veloce grazie alla banda	[62]
AWQ	Pesi a 4 bit; l'1 % di canali più importanti, individuato dall'ampiezza delle attivazioni, è protetto tramite scalatura. Robusto rispetto ai dati di calibrazione; pensato per l'uso sul dispositivo	[63]
GGUF k-quant	Formati a blocchi da 2 a 8 bit di llama.cpp, standard di fatto per la distribuzione consumer	[2]
MLX affine	Pesi a 4 o 8 bit con una scala e un bias per gruppo di 64; supporta l'addestramento LoRA sul modello quantizzato (usato in questa tesi)	[56]
QLoRA	Modello base a 4 bit in formato normal float con adapter a 16 bit, per il fine-tuning con memoria limitata	[64]

[a] La *perplexity* misura quanto un modello predice bene un testo: più è bassa, meno il modello è "sorpreso" dalle parole che incontra.

calibrare, e non un testo che deve soltanto risultare scorrevole, queste osservazioni suggeriscono una precisione prudente: GroundOne usa pesi a 8 bit. Una misura condotta in questa tesi conferma che, a questa precisione, la quantizzazione non costa accuratezza: applicando lo stesso adapter del modello 0.5B ai pesi originali a 16 bit, la balanced accuracy a tre classi su T1 resta praticamente identica (0.768 contro 0.769), le probabilità cambiano in media di 0.011 e la classe predetta coincide nel 98 % delle frasi (Sezione 7.11).

2.5.4 Generare più in fretta senza generare meno

Una diversa famiglia di metodi accelera la generazione senza modificare il modello. Nello speculative decoding un piccolo modello bozza propone diversi token, e il modello grande li verifica tutti in un unico forward pass, accettando il prefisso più lungo che avrebbe prodotto esso stesso; l'output è identico a quello del modello grande, e lo speed-up arriva a due-tre volte [65]. Il metodo è utile per qualunque sistema debba generare, ma non cambia la natura del costo: ogni token accettato richiede comunque un forward pass del modello grande, e la quantità totale di calcolo aumenta [65]. Rende la scrittura più economica, non gratuita.

2.6 Energia

Su un dispositivo a batteria l'energia è una proprietà del metodo e non soltanto dell'hardware. Misure su hardware da datacenter mostrano che l'energia di una richiesta cresce con il numero di token generati e dipende fortemente dalla dimensione del modello e dal task [66, 67]. Sugli smartphone, un modello da 3 miliardi di parametri a 4 bit spende tra 0.58 e 0.76 joule per token generato, con una potenza sostenuta intorno ai 14 W e temperature superficiali vicine ai 48 °C [58]; una regola empirica tratta dai lavori sui modelli piccoli stima il costo in circa 0.1 joule per token generato per ogni miliardo di parametri [37]. Spostare il prefill sulla NPU di uno smartphone ne riduce l'energia di 1.9–4.3 volte rispetto all'unità grafica [53].

La distinzione tra le due fasi vale anche per la potenza. Il prefill tiene occupate le unità aritmetiche e assorbe più potenza; il decoding attende la memoria e assorbe meno potenza, ma per molto più tempo [40]. Un metodo che fa risparmiare tempo fa quindi risparmiare anche energia, ma in misura minore, e il Capitolo 7 misura proprio questo effetto: GroundOne spende 5.5 joule per risposta lunga con il modello da 0.5B e 33 con il 3B, meno di qualunque giudice generativo della stessa dimensione, ma con un vantaggio inferiore a quello sul tempo (Sezione 7.10).

2.7 Conseguenze per la verifica sul dispositivo

Il capitolo si può ora riassumere in un'unica osservazione: su un dispositivo locale, che serve una richiesta alla volta, leggere un token costa molto meno che scriverne uno, quattordici-sedici volte meno sulla macchina della tesi (Sezioni 2.3.1–2.3.3), e le generazioni più recenti di hardware accelerano la lettura molto più della scrittura (Sezione 2.4.2). Ne discende una conseguenza diretta per un verificatore che gira accanto al generatore.

Un giudice generativo verifica una risposta leggendo il documento e la risposta e poi scrivendo i propri verdetti. Paga un prefill, come ogni metodo che legge il documento, e poi un passo di decoding per ogni token che scrive. Nel protocollo di latenza di questa tesi (Sezione 7.9), per una risposta di quattordici o più frasi un giudice sintetico da 3B scrive in mediana 65 token, e uno verboso, che motiva ogni verdetto, ne scrive 388. Il limite inferiore della Sezione 2.3.3 traduce questi conteggi in un tempo minimo che nessuna implementazione può battere (Figura 2.8b): sulla macchina della tesi, almeno 1.8 secondi per il giudice sintetico e 10.6 per quello verboso, oltre al prefill; persino sulla scheda grafica più veloce, 0.12 e 0.71 secondi.

Un decision model elimina del tutto questo termine. Legge il documento e la risposta una sola volta e prende ogni decisione dai logit calcolati durante quell'unica lettura, quindi il suo costo coincide con il solo prefill: la fase limitata dal calcolo, quella che l'hardware locale esegue bene e che il nuovo hardware accelera di più.

Il suo vantaggio su un giudice non è quindi una proprietà di una particolare macchina, ma della struttura dei costi dell'inferenza locale, ed è massimo proprio dove l'inferenza è locale. La località, che a prima vista è soltanto un vincolo sulla dimensione del modello, diventa la ragione per cui il progetto conta.

Da questo capitolo discendono tre requisiti per il verificatore, che guidano il metodo del Capitolo 4. Non deve generare alcun token, così che il suo costo sia un singolo prefill. Deve usare un modello abbastanza piccolo da condividere una macchina da 16 GB con un generatore e un retriever, quantizzato in modo prudente perché il suo output è una probabilità. E deve leggere le proprie decisioni dai logit, che solo un modello locale espone, così che possano essere calibrate.

Capitolo 3

Fondamenti e stato dell'arte

Questo capitolo passa in rassegna i lavori su cui GroundOne si fonda e quelli con cui viene confrontato. Il costo dell'inferenza su un dispositivo locale, da cui discende il progetto, è già stato trattato nel Capitolo 2. Si parte dai decision model, cioè dall'idea di leggere le decisioni di un modello linguistico invece di farle generare, e dal loro ecosistema, in rapida crescita. Seguono il problema delle allucinazioni nella retrieval-augmented generation (RAG) e i metodi che verificano le risposte rispetto alle fonti. Il capitolo si chiude con la calibrazione, con il parameter-efficient fine-tuning e con la lacuna che questa tesi intende colmare.

3.1 Decision Model

Il termine *System One model* è stato introdotto da TypeSafe con Jev, un modello offerto come servizio remoto. Jev risponde a domande chiuse su uno stato dato, cioè sul contesto fornito nella richiesta, e restituisce output tipizzati, come un booleano, una scelta tra opzioni o un punteggio, senza generare testo [14]. Il nome richiama il pensiero veloce e intuitivo descritto da Kahneman, contrapposto al ragionamento deliberato. Il fornitore dichiara latenze comprese tra le decine e le centinaia di millisecondi e forti risparmi rispetto ai giudici generativi. Queste cifre, però, restano dichiarazioni del fornitore, e l'architettura non è resa pubblica.

Le implementazioni open arrivate in seguito si distinguono per il modo in cui producono le probabilità, e se ne possono individuare tre famiglie. La più diffusa presenta la domanda come una scelta multipla e conserva la head del modello linguistico, cioè lo strato finale che assegna un punteggio a ogni token del vocabolario: la decisione si legge dai logit delle lettere delle opzioni, nella posizione in cui il modello scriverebbe la risposta. SemIf applica questo readout a un modello da 4B parametri lasciato congelato, cioè senza modificarne i pesi, e riporta che 21 criteri relativi a un documento ottengono risposta in circa un secondo; la migliore

baseline generativa capace di produrre JSON valido impiega invece più di cinque secondi [15]. Rizzo-Flow, a differenza di SemIf, addestra con LoRA un modello da 4B su decisioni tipizzate ed espone la stessa interfaccia di Jev. I suoi autori riportano che il fine-tuning ne migliora sia l'accuratezza sia la calibrazione [16]. È questo il decision model generalista usato come baseline in questa tesi.

La seconda famiglia codifica separatamente lo stato e ciascuna risposta candidata, e ne confronta gli embedding, cioè i vettori numerici che rappresentano ciascun testo. I Contrastive Language Models, che appartengono a questa famiglia, sono molto veloci quando i candidati sono numerosi e riutilizzabili. Nel confronto riportato dagli stessi autori, però, concordano con le decisioni di Jev molto meno di un modello che fa interagire stato e candidato [68]. La terza famiglia usa piccoli encoder[1] con una head decisionale dedicata. Laya, per esempio, si basa su un encoder ModernBERT da circa 400M parametri e assegna un punteggio a ogni opzione in corrispondenza del relativo token di maschera. Le domande di una richiesta sono valutate in un'unica chiamata, raggruppate in un batch, ma ciascuna viene codificata insieme a una propria copia del contesto [69, 70].

Le evidenze a supporto di questi sistemi sono disomogenee. La maggior parte proviene dagli autori stessi, spesso su una sola macchina e su un solo benchmark. Le valutazioni indipendenti sono recenti e meno favorevoli ai piccoli modelli open. Una valutazione di Jev su 37 dataset trova ben calibrate le probabilità che il modello assegna alle scelte fra opzioni. Le probabilità binarie, invece, ordinano bene gli esempi ma risultano mal posizionate rispetto a una soglia fissa di 0.5. Lo stesso studio riporta per Jev una balanced accuracy[2] media di 78.6 su LLM-AggreFact [20]. Una riproduzione indipendente di Laya osserva una calibrazione diversa da quella dichiarata e una forte dipendenza da una temperatura di calibrazione stimata sui dati di destinazione [21]. Un benchmark costruito su coppie contrastive, cioè su decisioni quasi identiche che differiscono per un solo dettaglio decisivo, colloca i piccoli modelli open molto al di sotto di Jev [71]. Infine, è stato mostrato che, quando l'etichetta di un'opzione e la sua definizione sono in conflitto, i decision model tipizzati seguono l'etichetta anziché la definizione [70].

L'ecosistema, intanto, cresce rapidamente. Il 9 ottobre 2026 l'hub di Hugging Face ha introdotto ufficialmente un tag dedicato a questi *decision model* [17]. Il

[1] Un *encoder* è un transformer in cui ogni token vede tutti gli altri, sia quelli che lo precedono sia quelli che lo seguono. Produce una rappresentazione del testo invece di generarlo, come BERT e i modelli che ne derivano. Il *token di maschera* è un token speciale che, durante il preaddestramento, prende il posto delle parole che il modello deve ricostruire.

[2] La *balanced accuracy* è la media, calcolata sulle classi, della recall, cioè della frazione di esempi di ciascuna classe classificati correttamente. A differenza dell'accuratezza semplice, non premia un modello che predice sempre la classe più frequente: un predittore costante ottiene 0.5 nel caso binario (50 se espressa in percentuale, come qui) e circa 0.33 con tre classi.

giorno successivo il tag raccoglieva già circa 1500 modelli, pubblicati da circa 600 autori e di qualità molto variabile. Questa tesi usa il termine *decision model*, adottato dall'ecosistema; *System One* resta il nome con cui TypeSafe ha introdotto la categoria, e compare nel seguito soltanto come nome dell'interfaccia compatibile con Jev.

Poche settimane dopo il rilascio di Jev, anche organizzazioni affermate hanno pubblicato i propri decision model open. Clef di Cloudflare aggiunge a un decoder una head decisionale dedicata e risponde a tutte le domande con un solo passaggio sullo stato: è l'architettura che per Jev era stata soltanto ipotizzata [19]. d1-3B di Liquid AI legge la decisione da un insieme ristretto di lettere di risposta, con un modello da 3B il cui stato è condiviso da tutte le domande [18]. Perplexity, infine, applica il fine-tuning a un modello da 27B con una head sulle lettere [72]. Una leaderboard della comunità, il Decision Index, valuta questi modelli su una batteria comune di benchmark pubblici, tra cui RAGTruth valutato a livello di risposta intera [22]. I risultati tracciano una linea netta. I decision model generalisti con meno di un miliardo di parametri superano a stento la baseline banale su RAGTruth, e soltanto i modelli da decine di miliardi di parametri individuano in modo affidabile le risposte allucinate.

Gli studi su Jev aggiungono tre osservazioni. Le sue probabilità sono calibrate se aggregate su molti task, ma non sul singolo task. Soglie e temperature vanno stimate sul carico di lavoro di destinazione [73, 74]. Scegliere la soglia tramite un limite inferiore di confidenza, infine, rende più sicuro mettere in produzione un giudice che restituisce soltanto decisioni [74]. Anche i runtime si sono adeguati: llama.cpp supporta ora nativamente l'interfaccia System One [2]. Nessuno di questi modelli, tuttavia, è specializzato nella groundedness, cioè nell'aderenza della risposta al documento. Nessuno, inoltre, riporta risultati di verifica a tre classi a livello di frase sui benchmark usati in questa tesi.

Il readout sulle lettere è stato studiato anche al di fuori della letteratura sui decision model, nelle domande a scelta multipla. Qui si possono confrontare due modi di ottenere la risposta di un modello addestrato a seguire istruzioni (instruction-tuned): leggere quale lettera ha la probabilità più alta come primo token, oppure lasciare che il modello scriva la risposta per esteso. Le due risposte spesso non coincidono [75]. Il readout, inoltre, privilegia particolari identificatori di opzione, come certe lettere, e questo bias si può stimare e rimuovere [76]. Entrambi i risultati riguardano modelli usati zero-shot. Suggeriscono che un readout sulle lettere, se deve servire da decisione, vada addestrato, calibrato o entrambe le cose. Nessun decision model open, inoltre, è stato specializzato nella verifica della groundedness e poi confrontato, in condizioni controllate e sullo stesso hardware, con giudici generativi, con verificatori specializzati e con decision model generalisti, di cui uno della stessa dimensione.

3.2 Retrieval-augmented generation e allucinazioni

La retrieval-augmented generation (RAG) dà a un modello linguistico accesso a documenti su cui non è stato addestrato: un retriever seleziona i passaggi pertinenti alla domanda e il modello scrive la risposta con questi passaggi nel contesto [1]. È l'approccio standard per costruire assistenti su documenti privati e può girare interamente su una macchina locale. Non elimina però le allucinazioni. Le rassegne della letteratura distinguono due tipi di errore. Gli errori di *fattualità* (factuality) si misurano rispetto alla conoscenza del mondo, quelli di *fedeltà* (faithfulness) rispetto all'input o al contesto forniti al modello [9]. Una risposta RAG, in particolare, può essere infedele proprio ai passaggi che ha ricevuto, contraddicendoli o aggiungendo informazioni che essi non contengono.

RAGTruth documenta quanto ciò sia frequente. Il dataset raccoglie circa 18,000 risposte, scritte da sei modelli linguistici in tre task di tipo RAG, e vi annota con precisione al carattere le porzioni di testo in conflitto con il contesto o non supportate da esso [8]. La distinzione di RAGTruth tra contenuto in conflitto e contenuto privo di fondamento corrisponde alle classi *contraddetta* e *non menzionata* adottate in questa tesi.

3.3 Verifica della groundedness

Verificare una risposta rispetto alla fonte è un problema strettamente legato alla natural language inference (NLI). Nella NLI si stabilisce se una premessa implica un'ipotesi (entailment), la contraddice o è neutrale rispetto a essa. Grandi dataset di NLI forniscono dati di addestramento: MultiNLI [77] e ANLI [78], quest'ultimo costruito in modo avversariale, cioè raccogliendo esempi scritti apposta per ingannare i modelli del momento. VitaminC [79] aggiunge coppie contrastive, in cui una revisione minima di una frase di Wikipedia ribalta il verdetto: il modello è così costretto a guardare al fatto preciso e non alla sovrapposizione di parole.

Applicare questi modelli a testi generati lunghi richiede di scomporli. SummaC suddivide in frasi sia il documento sia il riassunto e aggrega i punteggi di inferenza calcolati frase per frase [80]. FActScore, invece, scompone le generazioni lunghe in fatti atomici e li verifica uno per uno [81]. La verifica a livello di frase, adottata da GroundOne, è una via di mezzo tra un unico punteggio per l'intera risposta e la scomposizione in fatti atomici. Le frasi sono le unità naturali di una risposta, l'utente può intervenire su una frase segnalata e il numero di decisioni cresce con la lunghezza della risposta.

Nella pratica prevalgono due famiglie di verificatori. La prima è quella degli encoder specializzati, piccoli e veloci. AlignScore addestra un'unica funzione

di allineamento su un'ampia miscela di task [11]. MiniCheck addestra piccoli verificatori su dati sintetici, costruiti in modo da richiedere un ragionamento su più frasi, ed eguaglia modelli molto più grandi sui benchmark di grounding [12]. HHEM è un modello di valutazione delle allucinazioni dello stesso tipo, rilasciato come modello open [13]. Questi modelli valutano ogni coppia documento–frase in modo indipendente e restituiscono un'unica probabilità di supporto.

La seconda famiglia usa un modello linguistico come giudice, secondo lo schema LLM-as-a-judge[3] [82]. Su giudici di questo tipo si basano framework di valutazione per RAG come RAGAS [83] e ARES [84]. Lynx addestra con fine-tuning un giudice dedicato al rilevamento delle allucinazioni [85], mentre Self-RAG addestra il generatore stesso a emettere token di critica, cioè token speciali con cui il modello valuta i passaggi recuperati e il proprio output [86]. I giudici sono flessibili, ma il loro costo cresce con la quantità di testo che scrivono. I modelli piccoli usati come giudici, inoltre, violano spesso il formato di output richiesto. Per garantire una risposta che il parser sappia interpretare serve allora un decoding vincolato da una grammatica, che a ogni passo ammette soltanto i token compatibili con il formato richiesto [87]. Il benchmark di riferimento per entrambe le famiglie è LLM-AggreFact, che riunisce undici dataset di grounding annotati manualmente e li riconduce a un protocollo binario comune [12, 88]. La Figura 3.1 mette a confronto le due famiglie con il decision model specialista studiato in questa tesi.

3.4 Calibrazione e selective prediction

Una probabilità è calibrata quando, tra le predizioni fatte con una certa confidenza, la frazione di quelle corrette coincide con tale confidenza: in un modello calibrato, per esempio, circa l'80 % delle predizioni fatte con confidenza 0.8 risulta corretto. Le reti neurali moderne sono spesso mal calibrate, e una correzione semplice ed efficace è il temperature scaling: i logit vengono divisi per un'unica costante, la temperatura[4], stimata su dati tenuti da parte [89]. La misura abituale della calibrazione è l'Expected Calibration Error (ECE), che confronta confidenza e frequenza di predizioni corrette all'interno di fasce di confidenza [90]. Per i modelli linguistici interrogati tramite prompt può servire anche un bias per classe, che corregga una preferenza sistematica verso alcune risposte [91].

I modelli linguistici sanno, entro certi limiti, quello che sanno: sulle domande chiuse, le probabilità dei loro token sono spesso informative sulla correttezza della

[3]Nello schema *LLM-as-a-judge* un modello linguistico riceve tramite prompt il testo da valutare e i criteri di valutazione, e scrive il proprio giudizio, a volte preceduto da una motivazione.

[4]Con una temperatura $T > 1$ le probabilità diventano meno estreme, con $T < 1$ più estreme. Poiché tutti i logit sono divisi per la stessa costante, la classe più probabile non cambia.

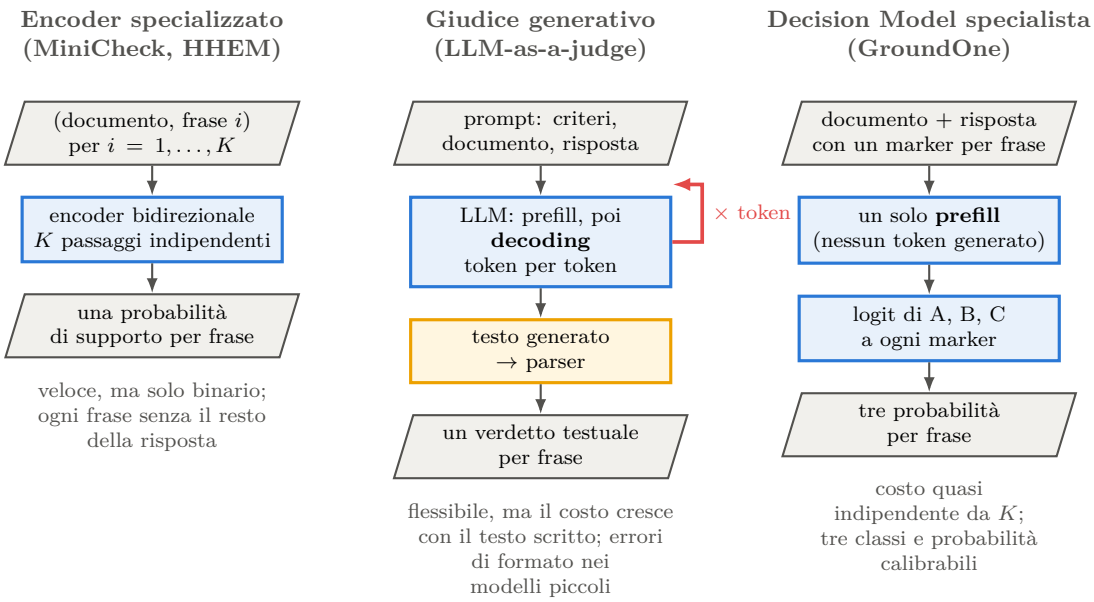


Figura 3.1: Tre modi di verificare le K frasi di una risposta. Un encoder specializzato valuta ogni coppia (documento, frase) in un passaggio separato e restituisce solo una probabilità di supporto. Un giudice generativo legge tutto, ma scrive il proprio verdetto token per token (ciclo rosso), quindi il costo cresce con il testo prodotto e l'output va interpretato da un parser. Un decision model specialista legge documento e risposta in un solo prefill e ricava da ogni marker tre probabilità, senza generare alcun token.

risposta [92]. Quando le probabilità non sono accessibili, si può chiedere al modello di esprimere la propria confidenza a parole. Nei modelli grandi sottoposti a fine-tuning con feedback umano, questa confidenza verbalizzata può risultare calibrata meglio delle probabilità stimate per campionamento [93]. Valutazioni sistematiche, però, la trovano grossolana e troppo sicura, con valori concentrati su pochi numeri tondi [94].

Una probabilità calibrata diventa utile grazie alla selective prediction: il sistema agisce sulle decisioni di cui è sicuro e rinvia le altre. La curva risk–coverage descrive questo compromesso. Per ogni quota di decisioni accettate (la coverage) riporta il tasso di errore su di esse (il risk), e mostra così come il tasso di errore scenda man mano che si rinviano più decisioni [95]. La Figura 3.2 ne illustra la forma tipica.

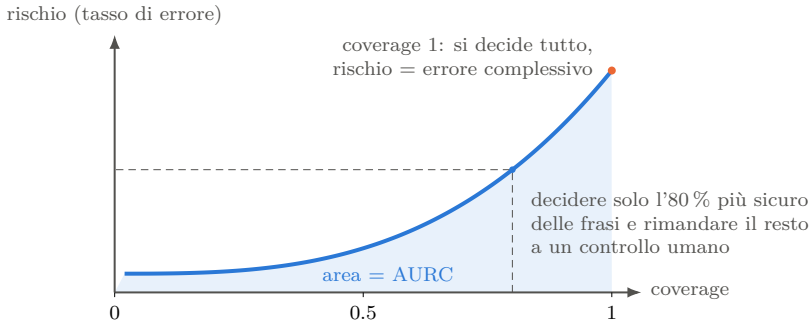


Figura 3.2: Schema di una curva risk–coverage (forma illustrativa, non un risultato). Le decisioni sono ordinate dalla più sicura alla meno sicura; per ogni quota decisa (coverage) si riporta il tasso di errore (rischio) delle decisioni accettate. A coverage 1 il rischio coincide con l'errore complessivo; rinunciando alle decisioni meno sicure l'errore scende. L'area sotto la curva (AURC) riassume la curva in un numero: più è piccola, meglio la confidenza separa le decisioni corrette da quelle errate.

3.5 Parameter-efficient fine-tuning

Specializzare un modello preaddestrato non richiede di aggiornarne tutti i pesi. I metodi di parameter-efficient fine-tuning (PEFT) addestrano soltanto una piccola quota di parametri e lasciano invariato il resto del modello. La low-rank adaptation (LoRA) congela il modello e, per alcune matrici di pesi, addestra soltanto un aggiornamento a basso rango[5]. Al termine dell'addestramento, l'aggiornamento si può fondere nei pesi oppure mantenere come piccolo adapter separato [24]. Applicare LoRA a tutti gli strati lineari conta più che usare un rango elevato, e la tecnica si combina con pesi di base quantizzati, così che anche modelli grandi stiano in poca memoria [64]. La famiglia Qwen2.5 offre modelli open a partire da 0.5B parametri con un tokenizer condiviso, e permette quindi di confrontare dimensioni diverse con la stessa ricetta [23]. La Figura 3.3 schematizza un adapter LoRA applicato a uno strato lineare.

Il fine-tuning su dataset piccoli, però, è notoriamente instabile. I risultati variano con il seed casuale, che determina sia l'inizializzazione dei nuovi parametri sia

[5]Il *rango* di una matrice è il numero massimo di sue righe (o colonne) linearmente indipendenti. LoRA impone che la correzione appresa per una matrice di pesi $d \times k$ abbia un rango r piccolo. Una matrice di questo tipo si può scrivere come prodotto di due matrici strette, una $d \times r$ e una $r \times k$, e basta quindi memorizzare e addestrare queste due: $r(d+k)$ parametri invece dei dk della matrice completa. In GroundOne 3B, per esempio, una matrice 2048×2048 ha circa 4.2 milioni di pesi, mentre la sua correzione di rango 16 ne richiede circa 65 mila, l'1.6 %.

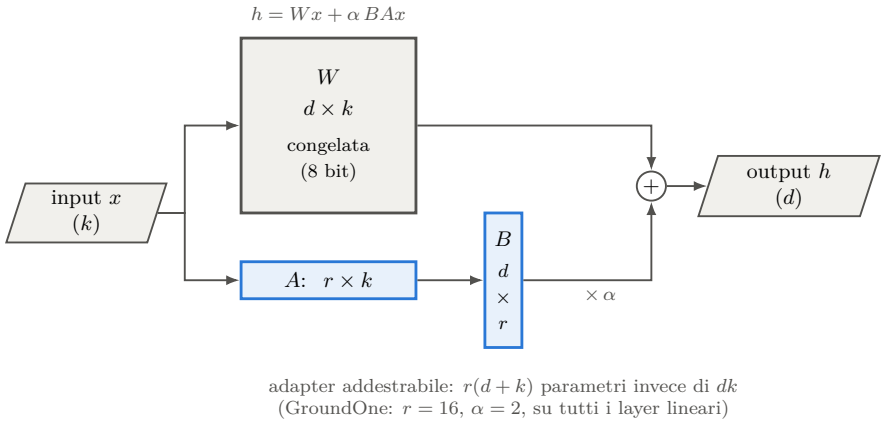


Figura 3.3: Low-rank adaptation applicata a uno strato lineare. La matrice dei pesi W resta congelata (in GroundOne a 8 bit); si addestrano soltanto le due matrici piccole A e B, il cui prodotto, scalato di α, si somma all'output dello strato. Con rango r l'adapter ha $r(d+k)$ parametri invece dei dk di W.

l'ordine dei dati di addestramento [96]. Alcuni addestramenti falliscono del tutto, e questo comportamento è legato a dettagli dell'ottimizzazione come l'assenza della bias correction in Adam[6] [97]. Il gradient clipping, che limita la norma del gradiente, è infine la difesa standard contro occasionali aggiornamenti esplosivi [98].

3.6 Lacuna nella letteratura

Gli elementi necessari a un verificatore locale, veloce e calibrato esistono già, ma separatamente: modelli open compatti e runtime che li eseguono su un laptop, l'idea dei decision model di leggere le decisioni anziché generarle, verificatori specializzati e calibrazione post-hoc. Manca uno studio indipendente e controllato che li metta insieme su un unico task. In particolare, nessun lavoro ha misurato se un decision model compatto, specializzato nella verifica della groundedness, possa eguagliare su questo task i giudici generativi e gli encoder dedicati. Non è noto nemmeno da dove provenga il suo vantaggio: dalla specializzazione, dal readout o dalla dimensione del modello. Resta infine da stabilire se le sue probabilità siano abbastanza buone da fissare soglie e rinviare decisioni, e come il suo tempo e la sua energia si confrontino

[6] Adam è un ottimizzatore che aggiorna ogni peso usando due medie mobili, una del gradiente e una del suo quadrato, dette stime dei momenti. Poiché queste medie partono da zero, nei primi passi sottostimano i valori veri; la *bias correction* le riscala per compensare. AdamW è la variante di Adam che applica il weight decay separatamente dal gradiente.

con quelli di un giudice sullo stesso hardware consumer. Il resto della tesi affronta queste domande.

Capitolo 4

GroundOne: metodo e implementazione

GroundOne è un modello linguistico compatto usato come decision model. Legge una sola volta il documento e la risposta e, senza generare testo, emette un verdetto per ciascuna frase valutata della risposta, ricavandolo dai logit di poche lettere di risposta. Il capitolo descrive anzitutto la decisione che il modello prende e il prompt che raccoglie in un unico forward pass tutte le decisioni relative a una risposta. Presenta poi l'obiettivo di addestramento, la calibrazione delle probabilità in uscita e la regola che converte le probabilità nelle etichette restituite, e si chiude con il software che implementa il tutto.

4.1 Task e decisione

Per ogni frase di una risposta, GroundOne sceglie fra tre classi, definite nella Sezione 5.1: rispetto al documento, la frase è *supportata*, *contraddetta* o *non menzionata*. Alle tre classi corrispondono le lettere A, B e C, e il modello restituisce una probabilità per ciascuna. Il punteggio binario di groundedness e le due policy di segnalazione, strict (che segnala ogni frase non supportata) e lenient (che segnala soltanto le contraddette), non sono domande distinte poste al modello. Vengono calcolati a partire da queste tre probabilità, e quindi un unico forward pass basta per ogni utilizzo. Interrogare il modello direttamente sulle policy non porterebbe nulla, perché ciascuna è una funzione deterministica della classe a tre vie. Domande separate moltiplicherebbero le decisioni senza aggiungere informazione, e potrebbero inoltre fornire al modello obiettivi di addestramento incoerenti per la stessa frase.

4.2 Tutti i verdetti in un solo forward pass

Il prompt contiene una breve istruzione, il documento e la risposta. Le frasi della risposta sono numerate, e ogni frase a cui è associata una decisione è seguita da un marker, cioè da una breve stringa fissa che indica il punto in cui leggere il verdetto: la stringa “ `Verdict:`”. Le frasi prive di contenuto verificabile (Sezione 5.3) restano nel testo come contesto, ma senza marker. Il modello elabora l'intera sequenza una sola volta. In corrispondenza dell'ultimo token di ciascun marker si leggono i logit delle lettere A, B e C, che vengono normalizzati con una softmax[1] ristretta a queste tre lettere. In un decoder ogni posizione vede soltanto quelle precedenti, quindi il verdetto sulla frase i dipende dal documento e dalle frasi $1, \ldots, i$. È esattamente ciò che serve alla verifica, perché i riferimenti, come i pronomi, puntano all'indietro. Quanto segue l'ultimo marker non influisce su alcun verdetto, quindi la sequenza si chiude lì. La Figura 4.1 mostra la struttura del prompt e i punti in cui avviene il readout.

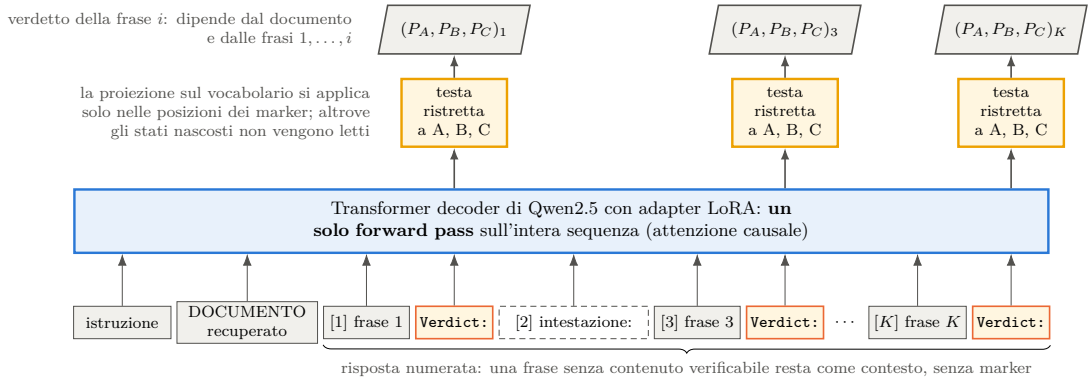


Figura 4.1: Il prompt a marker di GroundOne (design b). Le frasi della risposta sono numerate e ogni frase valutata è seguita dal marker “ `Verdict:`”; una frase senza contenuto verificabile (qui un'intestazione) resta come contesto, senza marker. Il modello elabora l'intera sequenza in un solo forward pass, e la testa di output, ristretta alle lettere A, B e C, si applica soltanto nelle posizioni dei marker. Per l'attenzione causale il verdetto della frase i dipende dal documento e dalle frasi da 1 a i.

Due accorgimenti rendono il readout esatto ed efficiente. Il primo riguarda l'esattezza: il prompt viene costruito tokenizzando separatamente le sue parti e poi

[1] La *softmax* trasforma un vettore di punteggi in una distribuzione di probabilità: ogni punteggio z_i diventa $e^{z_i} / \sum_j e^{z_j}$. Restringerla a tre lettere significa calcolarla soltanto sui loro tre logit, ignorando il resto del vocabolario.

concatenandole, così che la posizione di ogni marker sia nota con esattezza e resti identica in addestramento e in inferenza. Il secondo riguarda l'efficienza: la proiezione di output sul vocabolario si applica solo nelle posizioni dei marker. Il corpo del transformer elabora l'intera sequenza, ma la grande matrice del vocabolario viene moltiplicata soltanto per i pochi stati nascosti effettivamente letti, cioè per i vettori che il modello calcola nelle posizioni dei marker.

La lettura a marker determina anche il costo di una verifica. Si indichino con S la lunghezza complessiva di documento e risposta, con K il numero di decisioni e con m la lunghezza in token di un marker. Il decision model richiede un solo prefill di $S + Km$ token. Un giudice generativo, invece, deve anche produrre i propri verdetti in fase di decoding. Se ogni verdetto richiede n token e ogni token in decoding richiede un tempo t_{dec}, il costo del giudice è all'incirca $\mathrm{prefill}(S) + K\,n\,t_{\mathrm{dec}}$. La lunghezza m di un marker è di soli tre token, mentre sull'hardware di riferimento il decoding è molto più lento del prefill a parità di token; il divario fra i due costi, quindi, si allarga al crescere di K. Sull'M4 usato in questa tesi, con il modello 0.5B il prefill costa circa 0.37 ms per token e il decoding 5.9 ms per token generato; con il 3B i due valori sono 2.2 e 31.8 ms. Un token in decoding costa quindi quanto quattordici-sedici token in prefill. Le due fasi, inoltre, sollecitano l'hardware in modo diverso. Il prefill elabora molti token insieme ed è limitato dalla capacità di calcolo, mentre il decoding produce un token alla volta ed è limitato dalla banda necessaria a leggere i pesi [10, 40]. La differenza riemerge nelle misure di energia del Capitolo 7. La Figura 4.2 traduce il modello di costo in secondi per risposta, usando soltanto queste costanti misurate.

Esiste anche una seconda soluzione, che legge una sola volta il prefisso condiviso e vi accoda poi una breve domanda per ogni frase, valutando le domande in batch. A differenza della soluzione a marker, che si accontenta di un solo passaggio, questa richiede però una KV cache replicata K volte. È mantenuta come alternativa di riserva ed è il formato naturale per un modello non addestrato, che comprende una domanda esplicita ma non ha motivo di interpretare un marker come una richiesta di verdetto. Per questo è il formato usato dalla baseline non addestrata (Sezione 6.2). La Figura 4.3 confronta le due soluzioni.

Un classificatore, non un generatore più veloce. Il readout restringe la decisione a tre lettere, e la restrizione non è un dettaglio. Dopo il marker, il modello sottoposto a fine-tuning non scriverebbe la lettera che viene letta. Il suo token successivo più probabile è infatti la stessa lettera preceduta da uno spazio, che per il tokenizer è un token diverso. In corrispondenza dei marker, le tre lettere del readout raccolgono insieme appena il 4–5 % circa della massa di probabilità. Una loss ristretta a queste lettere addestra i loro logit relativi, non la loro probabilità assoluta, e il readout si comporta quindi come un classificatore a tre classi innestato sul modello linguistico. È questo a definire un decision model: la risposta non passa

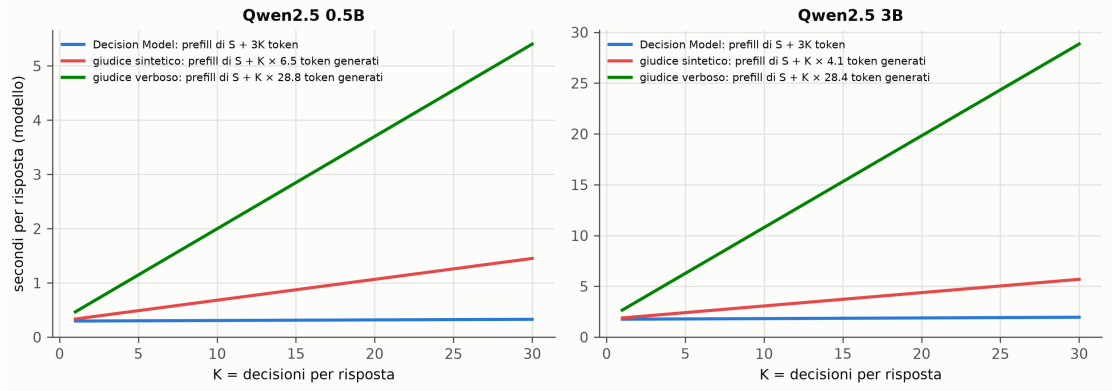


Figura 4.2: Costo stimato della verifica di una risposta di $S = 800$ token in funzione di K, calcolato con le sole costanti misurate sull'M4: il tempo per token del prefill e del decoding e il numero medio di token che ciascun giudice genera per verdetto sul sottoinsieme di latenza (K medio 8.7). È un modello analitico, non una misura: le latenze misurate sono nella Sezione 7.9. Il costo del decision model cresce di soli tre token di prefill per decisione, quello dei giudici di alcuni token di decoding per decisione.

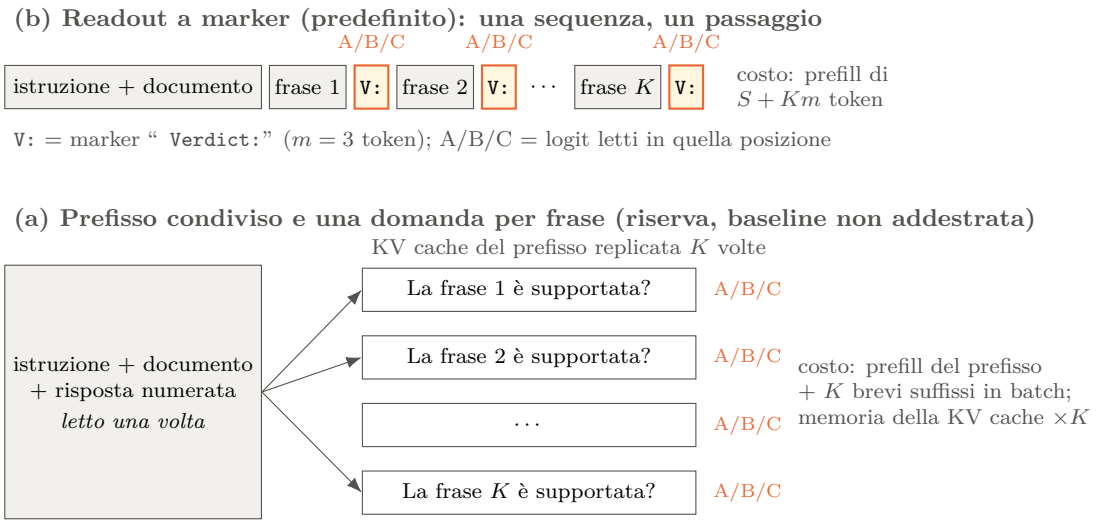


Figura 4.3: Le due soluzioni per ottenere K verdetti da una sola lettura. (b) Il readout a marker, adottato come predefinito, inserisce un marker dopo ogni frase e legge tutti i verdetti nello stesso passaggio. (a) La soluzione di riserva legge una volta il prefisso condiviso e vi accoda una breve domanda per ogni frase, valutate in batch; richiede però K copie della KV cache del prefisso, ed è il formato naturale per un modello non addestrato.

mai per il decoding, ma viene scelta fra le opzioni ammissibili. Il fenomeno richiama anche due risultati della letteratura: la probabilità del primo token di risposta e la risposta effettivamente scritta dal modello possono non coincidere [75], e la scelta fra lettere è soggetta a bias di selezione [76]. Un readout specializzato e calibrato deve assorbire entrambi gli effetti. Un esperimento di controllo ne quantifica la differenza rispetto alla generazione: mantiene gli stessi pesi e lo stesso passaggio, ma al posto del readout ristretto prende il token che la generazione emetterebbe (Sezione 7.6).

4.3 Obiettivo di addestramento

GroundOne si ottiene da Qwen2.5-0.5B-Instruct e Qwen2.5-3B-Instruct [23] tramite low-rank adaptation (LoRA) [24]. Gli adapter hanno rango 16, fattore di scala 2 e dropout 0.05[2]. Sono applicati a tutti i layer lineari di tutti i blocchi transformer, perché coprire ogni layer conta più che usare un rango elevato [64]. I pesi del modello base restano congelati in precisione a 8 bit. Ogni esempio di addestramento è una risposta con tutte le sue frasi valutate, e la loss viene calcolata su ciascuno dei suoi marker.

La loss è una cross-entropy[3] calcolata sulle sole tre lettere. Il prompt non viene mai supervisionato, quindi il segnale di addestramento proviene interamente dai verdetti. Circa l'88 % delle frasi del training set è supportato, per cui le classi sono pesate in modo che ciascuna contribuisca in egual misura alla loss. Con la distribuzione del training set si ottengono pesi di circa 0.38 per le frasi supportate, 7.5 per quelle contraddette e 4.4 per quelle non menzionate. Senza questi pesi, un modello può ottenere una loss bassa rispondendo quasi sempre "supportata". Ne discende una conseguenza utile: un modello che avesse appreso soltanto le frequenze delle classi resterebbe a una loss di $\ln 3 \approx 1.10$, quindi una loss inferiore a questo valore indica che il modello sfrutta effettivamente il testo.

L'ottimizzatore è AdamW, usato senza weight decay, cioè senza la penalità che a ogni passo riduce l'ampiezza dei pesi. Come nella configurazione predefinita di MLX, le stime dei momenti non ricevono la bias correction, e non si applica gradient clipping. L'addestramento dura un'epoca, cioè un solo passaggio sulle risposte del training set. Il learning rate sale fino a 10^{-4} con un warm-up lineare

[2]Il *fattore di scala* moltiplica l'aggiornamento calcolato dall'adapter prima di sommarlo all'output dello strato congelato. Il *dropout* azzera a caso, durante l'addestramento, una frazione degli input dell'adapter (qui il 5 %), così da ridurre il rischio che il modello memorizzi il training set.

[3]La *cross-entropy* di una predizione è il logaritmo negativo della probabilità che il modello assegna alla classe corretta: vale zero quando questa probabilità è uno e cresce senza limite quando il modello sbaglia con sicurezza.

sul primo 3 % dei passi e poi scende fino a zero con un cosine schedule[4]. Le risposte di lunghezza simile sono raggruppate in micro-batch, ciascuno entro un budget di token che comprende anche il padding, cioè i token di riempimento che uniformano le lunghezze. I gradienti di più micro-batch vengono poi accumulati, in modo che ogni passo dell'ottimizzatore veda circa 16 risposte, ovvero circa 110 decisioni. Così ogni passo contiene qualche frase contraddetta, nonostante la rarità della classe. Il gradient checkpointing, che durante il calcolo dei gradienti ricalcola alcune attivazioni invece di conservarle, mantiene l'occupazione di memoria entro i 16 GB del portatile. Durante l'addestramento il modello viene valutato periodicamente sul dev set, e si conserva il checkpoint, cioè la copia dei pesi, con la balanced accuracy a tre classi più alta, mediata sui tre task di RAGTruth (question answering, summarization e data-to-text). La Sezione 6.1 riporta le impostazioni di ciascun run.

4.4 Calibrazione

I pesi delle classi addestrano il modello come se le tre classi fossero ugualmente frequenti, per cui le sue probabilità grezze non rispecchiano le frequenze reali delle classi. Sul dev set si stima quindi una trasformazione post-hoc, applicata ai logit dopo l'addestramento e senza modificare i pesi. I tre logit vengono divisi per una temperatura T e traslati di un bias per classe, $\mathrm{softmax}(z/T + b)$, con il bias della classe supportata fissato a zero. I parametri T e b sono scelti minimizzando la negative log-likelihood delle etichette del dev set, cioè la cross-entropy media [89]. Il bias svolge il ruolo della correzione contestuale proposta per i modelli linguistici interrogati tramite prompt [91]; a differenza di quella, però, qui è stimato su dati etichettati. La sola temperatura cambia le probabilità ma non la classe più probabile né l'ordinamento delle frasi. Il bias, invece, può cambiare la classe più probabile e, con tre classi, può anche riordinare leggermente $P(\text{supported})$ fra una frase e l'altra. Per questo le metriche di classificazione e l'AUROC[5] sono calcolate sull'output grezzo. Le probabilità calibrate si usano invece dove una probabilità deve corrispondere a una frequenza: nelle metriche di calibrazione, nelle soglie e nella selective prediction (Sezione 7.7).

[4]Durante il *warm-up* il learning rate cresce gradualmente fino al valore scelto, così da evitare aggiornamenti bruschi all'inizio dell'addestramento. Il *cosine schedule* lo riduce poi seguendo mezzo periodo di un coseno: lentamente all'inizio, più in fretta a metà e di nuovo lentamente verso la fine.

[5]L'*AUROC* è l'area sotto la curva ROC e misura quanto bene un punteggio ordina gli esempi: equivale alla probabilità che un esempio positivo scelto a caso riceva un punteggio più alto di un esempio negativo scelto a caso. Vale 0.5 per un punteggio casuale e 1 per un ordinamento perfetto, e non dipende da alcuna soglia.

La calibrazione è misurata principalmente con l'Expected Calibration Error (ECE) [90, 89]. Le predizioni vengono ripartite in base alla confidenza in M intervalli di uguale ampiezza, con $M = 15$ in questa tesi. Per ogni intervallo B_m si confronta la confidenza media $\mathrm{conf}(B_m)$ con la frequenza osservata $\mathrm{acc}(B_m)$, e gli scarti vengono mediati con pesi proporzionali alla numerosità degli intervalli:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \Big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\Big|. \tag{4.1}$$

Qui n è il numero complessivo di predizioni. Nella variante binaria la confidenza è $P(\text{supported})$ e la frequenza osservata è la quota di frasi supportate nell'intervallo. Nella variante top-label a tre classi, invece, la confidenza è la probabilità massima e la frequenza osservata è la quota di predizioni corrette. L'ECE non confronta la distribuzione delle classi predette con quella del test set. Verifica piuttosto se, fra le frasi a cui il modello assegna una certa confidenza, quella confidenza corrisponde alla realtà. La misura ha però due limiti: dipende dalla scelta degli intervalli, e un predittore costante che restituisce sempre la frequenza di base ottiene un ECE vicino a zero pur non discriminando nulla. Per questi motivi l'ECE è sempre riportato insieme al Brier score[6], alla negative log-likelihood e ai reliability diagram, i grafici che riportano per ogni intervallo la frequenza osservata in funzione della confidenza media. La capacità delle probabilità di separare le decisioni corrette da quelle errate si misura invece a parte, con l'AUROC e con l'area sotto la risk–coverage curve.

Dalle probabilità alle etichette restituite. Chi usa il verificatore vede per ogni frase un'etichetta e una probabilità, che devono essere coerenti tra loro. Questa tesi valuta la decisione grezza, cioè la classe più probabile secondo l'output non calibrato. La decisione grezza, però, può contraddire la probabilità calibrata mostrata accanto: una frase può essere etichettata "non menzionata" pur avendo una $P(\text{supported})$ calibrata di poco superiore a un mezzo. L'output restituito ricava quindi ogni elemento dalle probabilità calibrate con un'unica regola. Una frase è etichettata come supportata se e solo se $P(\text{supported}) \geq \tau$; altrimenti riceve la più probabile fra le due classi restanti. La policy strict segnala ogni frase non supportata, quella lenient soltanto le contraddette. La soglia τ è stimata sul dev set in modo da massimizzare la balanced accuracy a tre classi. L'utente può modificarla a ogni richiesta, scegliendo così un diverso punto operativo senza riaddestrare il modello. La Figura 4.4 riassume il percorso dai logit grezzi alle etichette restituite.

[6] Il *Brier score* è l'errore quadratico medio fra la probabilità predetta e l'esito osservato, codificato come 1 se l'evento si verifica e 0 altrimenti: più è basso, migliori sono le probabilità.

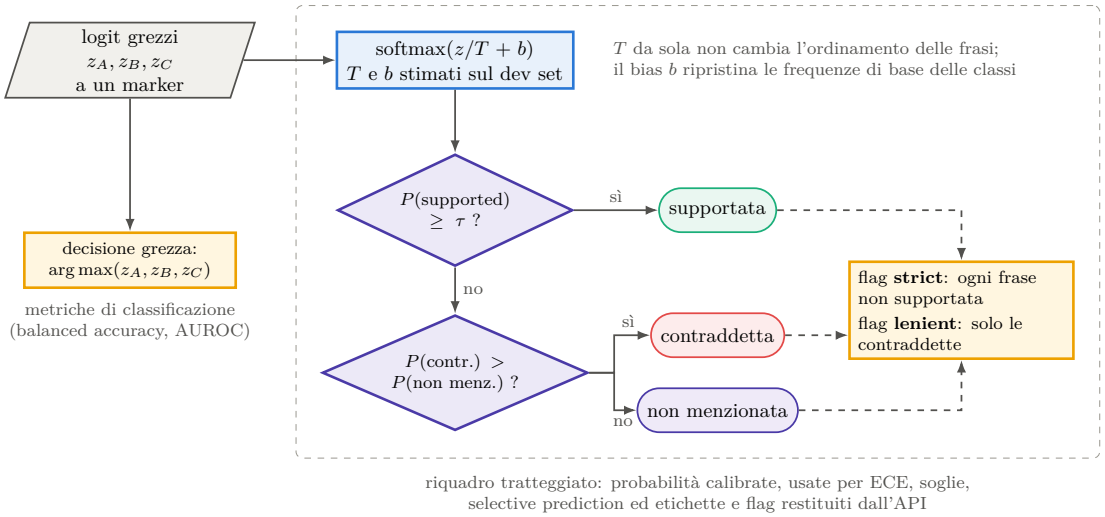


Figura 4.4: Dai logit alle etichette restituite. La decisione grezza, cioè l'argmax dei logit, serve alle metriche di classificazione del Capitolo 7. Le probabilità calibrate, ottenute con temperatura e bias stimati sul dev set, alimentano invece l'unica regola dell'output restituito: supportata se $P(\text{supported}) \geq \tau$, altrimenti la più probabile fra contraddetta e non menzionata. Da qui derivano i flag strict e lenient.

4.5 Software

GroundOne è implementato come pacchetto Python installabile basato su MLX [3, 56], il framework per array pensato per Apple silicon. Tutte le fasi di addestramento e di inferenza descritte finora vengono eseguite con MLX sullo stesso portatile. Il pacchetto comprende il costruttore del dataset con i relativi controlli, il ciclo di addestramento, la calibrazione, la valutazione sui test set congelati, le baseline e un meccanismo di protezione della memoria. Quest'ultimo esegue i processi lunghi uno alla volta e interrompe un processo prima che il sistema esaurisca la memoria.

Il verificatore è esposto tramite un'interfaccia a riga di comando e un server HTTP. Il server offre un endpoint nativo e un endpoint compatibile con l'interfaccia System One di Jev, già usata da altri decision model, così che i client esistenti possano adottare GroundOne cambiando soltanto l'indirizzo. L'endpoint nativo riceve un documento e una risposta. Segmenta la risposta con la stessa regola usata per costruire i dati ed esegue il singolo forward pass della Sezione 4.2. Per ogni frase riporta se è stata valutata, le tre probabilità calibrate, l'etichetta restituita e i flag delle policy strict e lenient, insieme alla regola di decisione e alla soglia che li hanno prodotti. Indica inoltre il numero di token in ingresso, il numero di token generati, sempre pari a zero, e il tempo impiegato. L'endpoint compatibile, invece, risponde a domande chiuse generiche su un contesto condiviso, con il formato a

una domanda per frase. Tutto il calcolo del modello avviene su un unico worker thread di lunga durata, perché il contesto Metal della GPU, cioè lo stato con cui il programma la usa attraverso l'interfaccia grafica di Apple, non deve spostarsi da un thread all'altro. Infine, nulla esce dalla macchina: modelli, dati e richieste restano in locale. La Figura 4.5 riassume l'architettura del pacchetto.

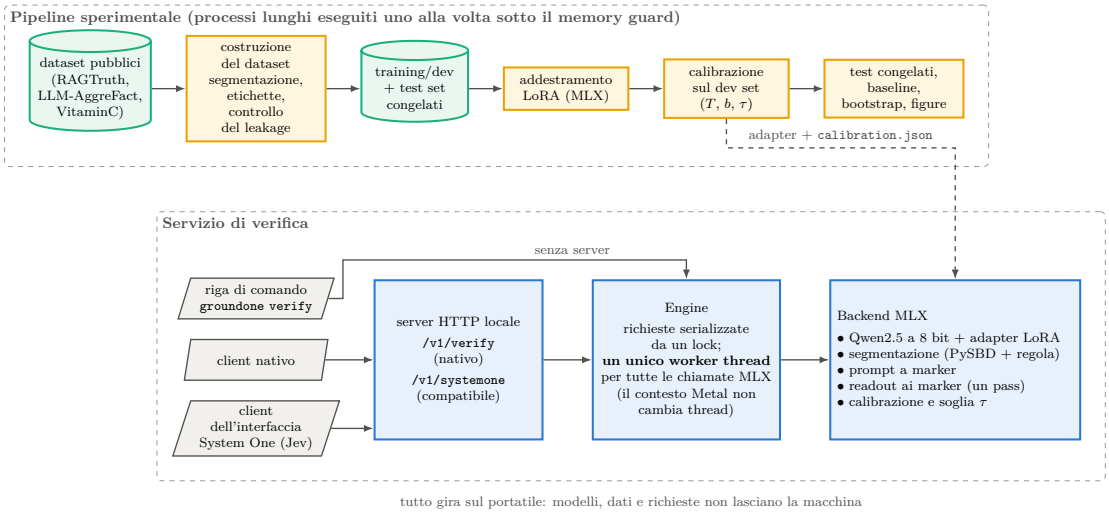


Figura 4.5: Architettura del software. In alto la pipeline sperimentale, dai dataset pubblici ai test congelati, eseguita sotto il memory guard; in basso il servizio di verifica, in cui riga di comando e server HTTP (endpoint nativo e compatibile) condividono lo stesso Engine, che serializza le richieste ed esegue ogni chiamata MLX su un unico worker thread. L'adapter e il file di calibrazione prodotti dalla pipeline sono gli unici artefatti che il servizio legge.

Capitolo 5

Dati

GroundOne apprende un'unica decisione, quindi i dati devono insegnare esattamente quella decisione e permettere di misurarla in modo affidabile. Il capitolo definisce anzitutto la decisione e le sue etichette. Descrive poi i tre dataset pubblici da cui le etichette provengono e il modo in cui vengono trasformati in training set, dev set e test set. Le ultime sezioni mostrano come l'affidabilità dei test set sia stata verificata senza nuove annotazioni, che cosa emerge da un'analisi esplorativa del dataset costruito e come sia stato escluso il leakage tra i dati di addestramento e quelli di test.

5.1 Task ed etichette

L'input è costituito da un documento recuperato e da una risposta, scritta da un modello linguistico e suddivisa in frasi. Per ogni frase il modello stabilisce se, *rispetto al documento*, essa è *supportata*, *contraddetta* o *non menzionata*. Una frase è supportata quando il documento dice, o implica direttamente, tutto ciò che essa afferma. È contraddetta quando una sua parte è in conflitto con il documento. È non menzionata quando aggiunge informazioni che il documento non contiene, ma nemmeno contraddice. Quando una frase contiene sia una parte in conflitto sia una parte non verificabile, viene etichettata come contraddetta, perché la contraddizione è l'errore più grave. Conta soltanto il documento: un'affermazione vera nel mondo reale ma assente dal documento resta non menzionata.

Il numero di decisioni richieste da una risposta coincide con il numero delle sue frasi valutate, indicato con K. Questa grandezza è centrale nella tesi. Un decision model legge documento e risposta una sola volta, quindi ogni decisione aggiuntiva gli costa quasi nulla; un giudice generativo, invece, deve produrre testo per ogni frase. Per questo il confronto con i giudici generativi è condotto in funzione di K, e i dati devono comprendere risposte lunghe oltre che brevi.

Per ogni frase il modello restituisce tre probabilità, e ogni altra grandezza è derivata da queste, invece di essere appresa separatamente. Il punteggio binario di groundedness, usato per il confronto con i benchmark binari pubblicati, è $P(\text{supported})$. Le segnalazioni dipendono invece dalla nozione di errore adottata. Secondo una policy *strict* è un errore tutto ciò che il documento non afferma, e una frase viene segnalata quando $P(\text{supported})$ scende sotto una soglia τ. Secondo una policy *lenient* sono errori soltanto le contraddizioni, e una frase viene segnalata solo quando è giudicata contraddetta. In questo modo l'utente sceglie la nozione di errore e la soglia al momento dell'inferenza, senza riaddestrare il modello. La Sezione 4.4 riporta la regola esatta con cui il verificatore produce le etichette che restituisce.

5.2 Dataset di partenza

Il dataset è costruito di proposito a partire dal minor numero possibile di fonti: una per l'addestramento e tre per il test (Tabella 5.1). Con meno fonti la definizione delle etichette e il formato del testo restano unici, e si riducono i punti in cui la costruzione può introdurre errori.

Tabella 5.1: I dataset di partenza: licenza, ruolo in GroundOne e tipo delle etichette originali.

Dataset	Licenza	Ruolo	Etichette
RAGTruth [8]	MIT	training set, dev set, test T1	span di caratteri relativi a quattro tipi di allucinazione nelle risposte di LLM
LLM-AggreFact [12, 88]	CC BY-ND 4.0	solo test T2	binarie (supportata / non supportata) per affermazione
VitaminC [79]	CC BY-SA 3.0	solo test T3	supports / refutes / not enough information

RAGTruth. RAGTruth [8] contiene 17,790 risposte scritte da sei modelli linguistici (GPT-3.5, GPT-4, Llama-2 7B, 13B e 70B, e Mistral-7B) in tre task di RAG, che si distinguono per il tipo di contesto. Nel question answering il contesto è un insieme di passaggi recuperati per una domanda di MS MARCO [99]. Nella summarization è un articolo di cronaca, tratto da CNN/DailyMail [100, 101] o da notizie recenti. Nella generazione data-to-text, in cui il modello descrive in prosa dati strutturati, il contesto è la scheda di un'attività commerciale di Yelp in formato

JSON [102]. In ciascuna risposta, due annotatori formati hanno individuato gli span di caratteri non supportati dal contesto[1], con una terza revisione nei casi di disaccordo. A ogni span hanno poi assegnato uno di quattro tipi: *conflitto*, evidente o sottile, e *informazione infondata*, evidente o sottile. L'articolo riporta un accordo tra annotatori del 91.8 % a livello di risposta e del 78.8 % a livello di span. RAGTruth è l'unico dataset pubblico che unisca risposte naturali di più frasi, prodotte da modelli diversi, alla distinzione tra contenuto contraddetto e contenuto non supportato, ed è per questo l'unica fonte di addestramento. Lo split ufficiale separa 450 fonti di test dalle restanti, destinate all'addestramento.

LLM-AggreFact. LLM-AggreFact [12, 88] aggrega undici dataset di fact checking annotati manualmente, ciascuno composto da coppie (documento, affermazione). I dataset coprono tre ambiti: la summarization (AggreFact-CNN e XSum [103], TofuEval [104], con i due sottoinsiemi MediaS e MeetB), il question answering basato su retrieval o con risposte in forma estesa (RAGTruth [8], ClaimVerify [105], ExpertQA [106], LFQA [107]) e il fact checking vero e proprio (WiCE [108], REVEAL [109], FactCheck-GPT [110]). Il benchmark ha una leaderboard pubblica, che rende i risultati confrontabili con quelli dei sistemi pubblicati. La sua licenza, però, vieta di usare i dati per addestrare modelli [111]. In questa tesi il dataset serve quindi soltanto alla valutazione: nessun addestramento, calibrazione o selezione avviene su alcuno dei suoi split.

VitaminC. VitaminC [79, 112] è un dataset di verifica dei fatti costruito a partire da revisioni di Wikipedia che cambiano un fatto. Per ogni revisione, annotatori formati hanno scritto affermazioni supportate da una versione della frase e confutate dall'altra. Quando non era possibile formulare una contraddizione, la coppia è stata etichettata come priva di informazioni sufficienti. Le due versioni dell'evidenza sono quasi identiche, quindi per decidere il modello deve guardare al fatto preciso, e non alla sovrapposizione lessicale. Su un campione ri-annotato di 2,000 coppie gli autori riportano un κ di Fleiss pari a 0.71[2]. In questa tesi VitaminC colma una lacuna precisa. RAGTruth non pubblica l'accordo tra annotatori sul *tipo* di allucinazione, quindi da solo non può dimostrare che il modello apprenda il confine tra contraddetta e non menzionata. VitaminC fornisce proprio questo confine, con etichette di affidabilità nota e in un dominio diverso.

[1] Uno *span* è un tratto contiguo di testo, individuato dalla posizione del suo primo e del suo ultimo carattere. Un'annotazione a livello di span marca la porzione esatta della risposta che contiene l'errore, invece di giudicare la risposta nel suo insieme.

[2] Il κ di Fleiss misura l'accordo tra più annotatori al netto dell'accordo che si otterrebbe per puro caso: vale 1 quando l'accordo è perfetto e 0 quando non supera quello casuale.

Diversi altri candidati sono stati esaminati e scartati. I corpora di NLI come ANLI [78] hanno premesse brevi, lontane dai contesti tipici della RAG, e la stessa distinzione a tre classi su testo fattuale è già coperta da VitaminC. I dati di addestramento sintetici o soltanto binari, come quelli rilasciati con MiniCheck [12], non hanno etichette a tre classi. Infine, sono stati esclusi i dataset con etichette generate da un modello linguistico e quelli che riutilizzano risposte di test di RAGTruth: in questo modo le etichette restano umane e i test set restano indipendenti.

5.3 Costruzione

Segmentazione in frasi. Le risposte contengono spesso elenchi e intestazioni, quindi vengono prima spezzate in corrispondenza degli a capo. Ogni riga viene poi divisa in frasi con PySBD [113], un segmentatore basato su regole che gestisce abbreviazioni e numeri decimali. I simboli di elenco iniziali, come "3." o "*", vengono rimossi, perché riguardano l'impaginazione e non il contenuto. Lo stesso segmentatore è usato in addestramento, in valutazione e in inferenza, così che il modello veda sempre frasi tagliate allo stesso modo. La Figura 5.1 riassume l'intero processo di costruzione, dalle fonti ai test set congelati.

Frasi senza contenuto verificabile. Alcune frasi non affermano nulla sul documento: saluti, intestazioni che introducono un elenco, righe che contengono soltanto una citazione, come "(Passage 2)", e formule di rifiuto. Queste frasi restano nella risposta numerata come contesto, ma non costituiscono decisioni e non hanno etichetta. Una breve regola le individua, e riguarda l'8.2 % di tutte le frasi di RAGTruth, in gran parte intestazioni di elenchi. La regola considera soltanto la formattazione, cioè simboli di elenco, righe di sola citazione e formule conversazionali, e non modifica mai un'etichetta. La sua copertura è stata verificata rispetto alla segmentazione indipendente di LLM-AggreFact, descritta nella Sezione 5.5.

Etichette. In RAGTruth l'etichetta di una frase valutata dipende dagli span con cui la frase si sovrappone. La frase è contraddetta se si sovrappone a uno span di conflitto, non menzionata se si sovrappone soltanto a span di informazione infondata, e supportata in tutti gli altri casi. Soltanto 27 dei 14,226 span annotati (0.19 %) non toccano alcuna frase valutata, quindi la conversione non perde quasi nessuna annotazione. Per LLM-AggreFact, invece, l'etichetta binaria è mantenuta così come rilasciata. Le etichette di VitaminC sono convertite direttamente: *supports* diventa supportata, *refutes* contraddetta e *not enough information* non menzionata. La Figura 5.2 riassume la regola applicata a RAGTruth.

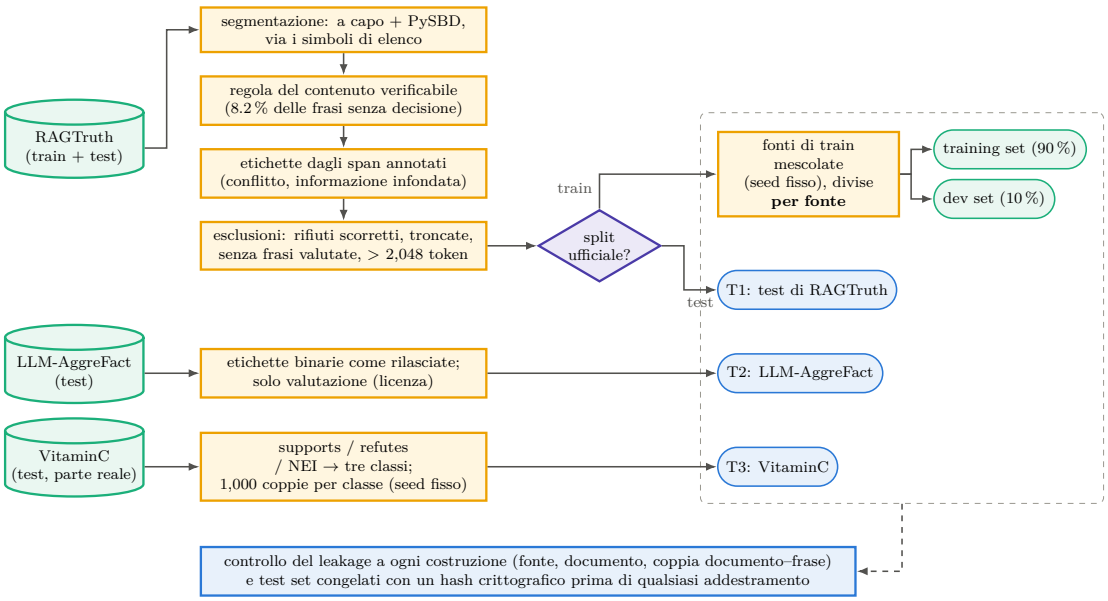


Figura 5.1: Costruzione del dataset. Le risposte di RAGTruth sono segmentate, filtrate dalle frasi senza contenuto verificabile, etichettate a partire dagli span e ripulite dalle esclusioni; lo split ufficiale di test diventa T1, mentre le fonti di addestramento sono divise per fonte in training set e dev set. LLM-AggreFact e VitaminC forniscono soltanto test set (T2 e T3). Un controllo automatico del leakage accompagna ogni costruzione, e i test set sono congelati con un hash prima di qualsiasi addestramento.

Esclusioni e split. Vengono rimosse tre categorie di risposte: quelle che RAGTruth segnala come rifiuti scorretti o come troncate, quelle senza alcuna frase valutata e quelle in cui documento e risposta superano insieme i 2,048 token. Quest'ultimo limite di lunghezza è imposto dalla memoria del portatile da 16 GB. Lo split di test ufficiale di RAGTruth diventa il test set T1. Le fonti di addestramento, invece, vengono mescolate con un seed fisso e divise in due parti: il 10 % forma il dev set e il resto il training set. Poiché la divisione avviene per fonte, le risposte relative alla stessa fonte non compaiono mai in entrambi gli insiemi. Vengono inoltre rimosse sei risposte di addestramento relative a un articolo di cronaca presente anche nel test set di LLM-AggreFact. Infine, la parte reale (non sintetica) dello split di test di VitaminC è sbilanciata tra le classi, quindi T3 è formato campionando da essa, con un seed fisso, 1,000 coppie per classe.

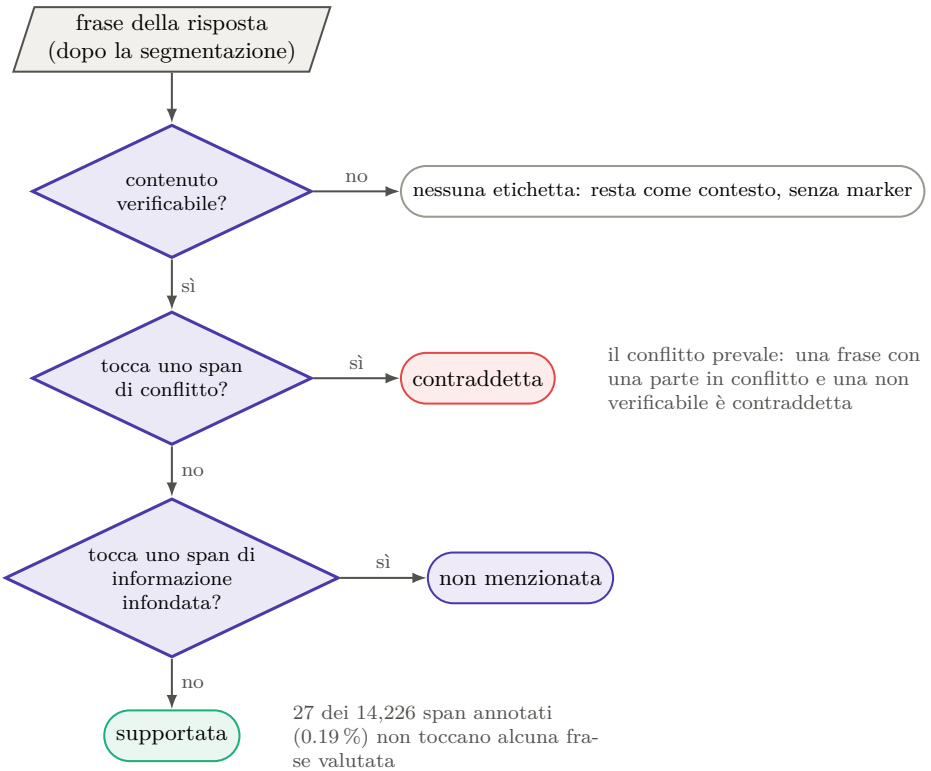


Figura 5.2: Regola con cui le annotazioni a livello di span di RAGTruth diventano etichette di frase. Le frasi senza contenuto verificabile restano nel prompt come contesto ma non sono decisioni; fra le altre, il conflitto prevale sull'informazione infondata.

5.4 Il dataset risultante

La Tabella 5.2 riassume i cinque file del dataset. Il training set contiene 13,203 risposte e 90,795 frasi etichettate. Ogni record corrisponde a una risposta e contiene un'etichetta per ciascuna frase: in questo modo gli stessi dati possono alimentare modi diversi di leggere le K decisioni di una risposta.

RAGTruth è fortemente sbilanciato: circa l'88 % delle frasi etichettate è supportato, e le contraddizioni sono appena il 3–4 % (Figura 5.3). Questa è la frequenza naturale degli errori nelle risposte dei modelli, e viene preservata. Il test set riflette così ciò che un verificatore incontra nella pratica, mentre lo sbilanciamento viene gestito in fase di addestramento. T3 compensa offrendo un test bilanciato fra le tre classi.

Le risposte contengono in media 6.9 frasi valutate nel training set e 6.5 in T1, con un massimo di 28 (Figura 5.4). Per analizzare i risultati in funzione della

Tabella 5.2: Il dataset GroundOne: numero di record e di decisioni per split e per classe. Una decisione è una frase etichettata. T2 ha etichette binarie, quindi le frasi contraddette e non menzionate vi sono riunite nella classe non supportata.

Split	Fonte	Record	Decisioni	Supportate	Contraddette	Non menzionate
Train	RAGTruth	13,203	90,795	79,898	4,039	6,858
Dev	RAGTruth	1,468	9,996	8,827	410	759
T1	RAGTruth test	2,647	17,284	15,745	584	955
T2	LLM-AggreFact test	29,320	29,320	22,832	6,488 non supportate	
T3	VitaminC test	3,000	3,000	1,000	1,000	1,000

lunghezza, T1 è diviso in quattro fasce di K: 756 risposte con 1–4 frasi, 1,288 con 5–8, 497 con 9–13 e 106 con 14 o più. Quando una cella dei risultati contiene meno di 30 frasi di una classe minoritaria, come accade per le frasi contraddette nel question answering, il valore viene riportato ma segnalato come indicativo.

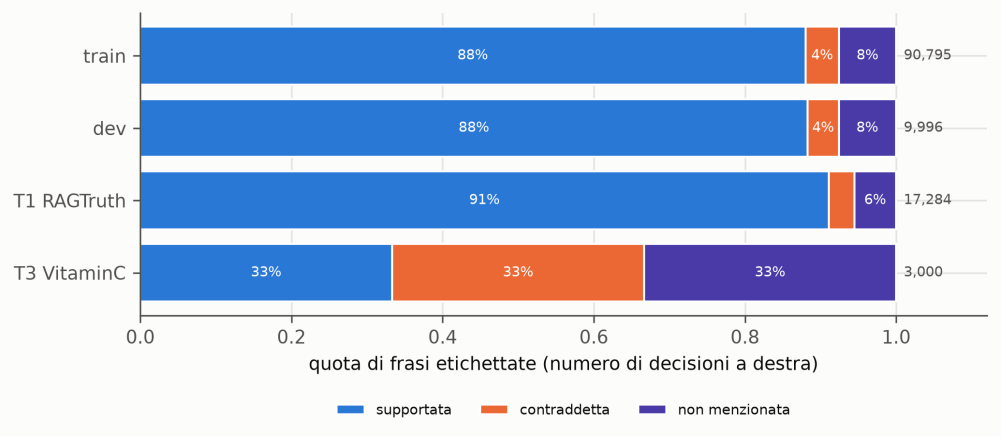


Figura 5.3: Distribuzione delle classi fra le frasi etichettate di ciascuno split. Negli split di RAGTruth (train, dev e T1) circa l'88–91 % delle frasi è supportato, mentre T3 è campionato con 1,000 coppie per classe.

5.5 Affidabilità dei test set

I test set sono stati verificati senza nuove annotazioni, ricorrendo a evidenze indipendenti dalla costruzione descritta sopra (Tabella 5.3). Tutte le etichette di test provengono dagli annotatori umani dei dataset pubblicati. Le due scelte che potevano introdurre errori sono la segmentazione e la conversione dagli span alle etichette di frase. Entrambe sono state confrontate con LLM-AggreFact, i cui autori

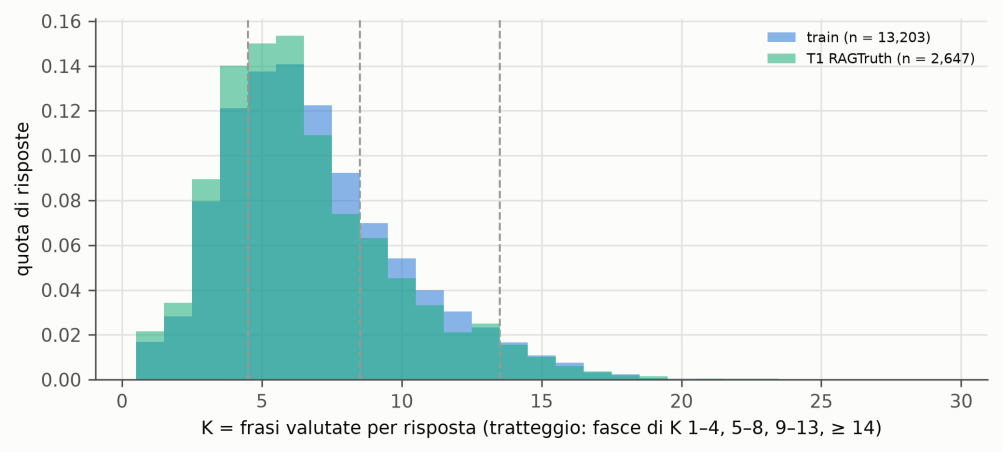


Figura 5.4: Distribuzione di K, il numero di frasi valutate per risposta, nel training set e in T1. Le linee tratteggiate delimitano le fasce di K usate per riportare i risultati.

hanno convertito in modo indipendente le stesse risposte di test di RAGTruth in affermazioni a livello di frase. Il segmentatore di questa tesi ritrova esattamente il 96.3 % delle loro affermazioni, e solo l'8.5 % delle frasi valutate manca dal loro insieme. Si tratta per lo più di frasi eliminate dal loro filtro delle affermazioni "non meritevoli di verifica", che non è documentato. Sulle 15,492 frasi presenti in entrambi gli insiemi, le etichette binarie concordano nel 98.75 % dei casi. Tutti i 193 disaccordi, inoltre, vanno nella stessa direzione: la regola basata sugli span marca come non supportata una frase che LLM-AggreFact considera supportata. Infine, una baseline basata su encoder, valutata su T2 senza modifiche, riproduce il punteggio pubblicato dai suoi autori (balanced accuracy di 74.0 misurata, contro 73.5 pubblicata), il che convalida il codice di valutazione.

Questi controlli convalidano il versante binario di T1, cioè la distinzione tra frasi supportate e non supportate. La distinzione tra contraddetta e non menzionata poggia invece sugli annotatori di RAGTruth, il cui accordo sul tipo di allucinazione non è pubblicato. Inoltre, una recente ri-annotazione dello split di test di RAGTruth ha trovato 1.68 volte più risposte con allucinazioni rispetto alle etichette originali [114], il che suggerisce che T1 sottostimi le due classi minoritarie. Per questi motivi i risultati a tre classi su T1 vengono sempre letti insieme a quelli su T3.

Completati questi controlli, e prima di addestrare qualsiasi modello, i tre file di test sono stati congelati con hash crittografici[3]. Da quel momento il costruttore del

[3]Un *hash* crittografico è un'impronta di lunghezza fissa calcolata dal contenuto di un file. Anche

Tabella 5.3: Controlli di affidabilità eseguiti prima del congelamento dei test set, con il criterio di superamento e l'esito di ciascuno.

Controllo	Criterio	Esito
Provenienza delle etichette	ogni etichetta di test proviene da annotatori umani	superato
Segmentazione	$\geq$ 90 % delle affermazioni di LLM-AggreFact recuperate, $\leq$ 10 % delle frasi assenti da esso	96.3 % / 8.5 %
Derivazione delle etichette	accordo $\geq$ 97 % con le etichette binarie di LLM-AggreFact	98.75 %
Baseline di riferimento	MiniCheck-RoBERTa su T2 entro 1.5 punti dal punteggio pubblicato	74.0 contro 73.5
Leakage	nessuna fonte, documento o coppia (documento, frase) in comune tra addestramento e test	superato

dataset si rifiuta di sovrascriverli con contenuti diversi, e ricostruendo il dataset dalle fonti grezze si riottengono esattamente i file congelati.

5.6 Analisi esplorativa

Un'analisi esplorativa delle fonti e del dataset costruito ha messo in luce quattro proprietà che condizionano la lettura dei risultati.

Gli errori dipendono dal task e dal modello. Le risposte data-to-text contengono allucinazioni con una frequenza da due a quattro volte maggiore rispetto a quelle degli altri due task. Anche il generatore conta: le risposte di Llama-2 e Mistral concentrano la maggior parte delle allucinazioni, mentre quelle di GPT-3.5 e GPT-4 sono quasi sempre fedeli al documento (Figura 5.5). Per questo i risultati sono riportati task per task: una media unica rifletterebbe soprattutto il data-to-text.

Le risposte di test sono più pulite di quelle di addestramento. In ogni task e per ogni generatore, la quota di frasi contraddette e non menzionate è più bassa in T1 che nel training set. Nel question answering, per esempio, è contraddetto lo

una modifica minima del file produce un'impronta completamente diversa, quindi confrontare le impronte basta a rilevare qualunque cambiamento.

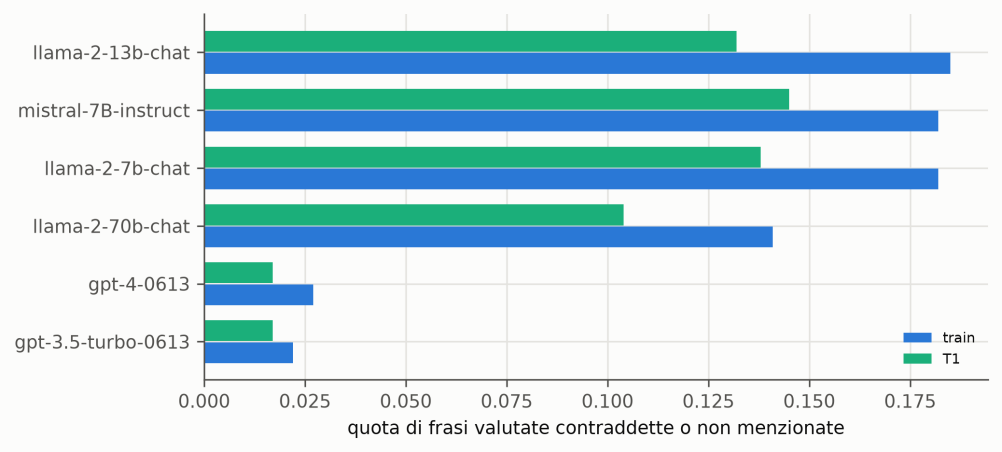


Figura 5.5: Quota di frasi contraddette o non menzionate in base al modello che ha scritto la risposta, nel training set e in T1.

0.6 % delle frasi di T1, contro l'1.6 % di quelle del training set. Di conseguenza, le metriche che dipendono dalla frequenza di base delle classi, come la precisione o l'F1 sulle classi minoritarie[4], non sono confrontabili tra dev set e test set, perché il dev set, estratto dal training set, ne conserva la distribuzione. Le metriche principali sono quindi la balanced accuracy, la recall macro e l'AUROC, che dalla frequenza di base non dipendono.

Le contraddizioni sono per lo più evidenti. Circa l'85 % delle frasi contraddette del training set deriva da span annotati come conflitti *evidenti*, per esempio numeri o nomi sbagliati, mentre soltanto 147 derivano da conflitti *sottili*. La classe contraddetta è quindi composta soprattutto da contraddizioni palesi, e le contraddizioni sottili non possono essere valutate separatamente.

Esistono indizi superficiali, e definiscono le baseline. Due indizi semplici predicono le etichette senza verificare la frase rispetto al documento. Il primo è la sovrapposizione lessicale, cioè la quota delle parole di contenuto della frase che compaiono anche nel documento; le parole di contenuto sono quelle che restano escludendo le parole funzionali, come articoli, preposizioni e verbi ausiliari. In tutti e tre i test set questa quota è più alta per le frasi supportate che per quelle contraddette e non menzionate (Figura 5.6). Usata da sola come punteggio, separa

[4]La *recall* di una classe è la quota dei suoi esempi che il modello riconosce. La *precisione* è la quota delle predizioni di quella classe che risultano corrette, e per questo cambia con la frequenza della classe anche quando il modello resta lo stesso. L'F1 è la media armonica delle due.

le frasi supportate da quelle non supportate con un AUROC di 0.74 su T1 e di 0.70 su T3. Il secondo indizio è la formulazione della frase. Nel data-to-text, un classificatore che vede soltanto la frase raggiunge una balanced accuracy a tre classi di 0.66, perché nelle descrizioni di Yelp alcuni dettagli vengono inventati spesso, come il numero di recensioni o la disponibilità di Wi-Fi o del parcheggio con valet. Nel question answering e nella summarization, invece, lo stesso classificatore resta al livello del caso (0.36 e 0.35), e lo stesso accade su T3 (0.33), dove la costruzione contrastiva del dataset elimina l'indizio. Un verificatore è utile solo nella misura in cui batte questi indizi, per cui entrambi sono riportati come baseline, task per task.

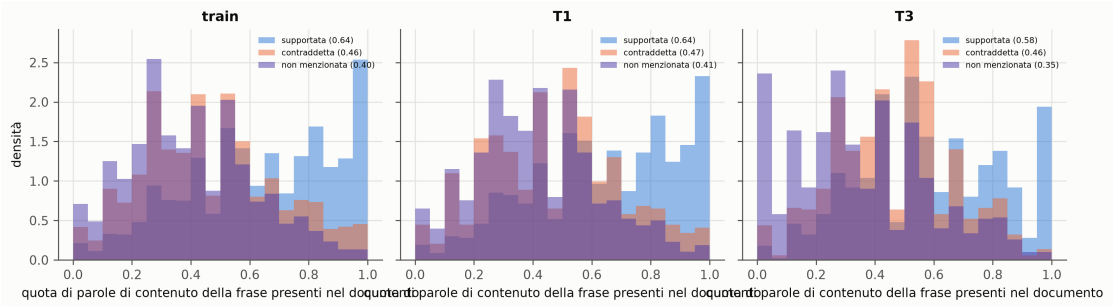


Figura 5.6: Sovrapposizione lessicale tra una frase e il suo documento, cioè quota delle parole di contenuto della frase presenti nel documento, per classe, nel training set, in T1 e in T3. Tra parentesi è indicata la media.

Infine, le frasi non supportate non si concentrano alla fine della risposta. La prima frase è raramente non supportata, mentre la quota di frasi non supportate raggiunge il massimo tra la terza e la quinta posizione e resta tra l'8 e l'11 % nelle posizioni successive (Figura 5.7). Anche le risposte lunghe, quindi, contengono errori verso la fine, e l'accuratezza per valori elevati di K resta rilevante.

5.7 Controllo del leakage

Gli split sono definiti per fonte e mai per frase, così che tutte le risposte relative a una stessa fonte cadano nello stesso split. A ogni costruzione del dataset viene inoltre eseguito un controllo automatico, che fallisce in due casi: quando una fonte, un documento o una coppia (documento, frase) del training set o del dev set compare in uno qualsiasi dei test set, e quando dev set e training set condividono una fonte. Il controllo risulta superato. Segnala inoltre le frasi che compaiono identiche sia nei dati di addestramento sia in quelli di test, associate a documenti diversi. Si tratta di 788 frasi template, come "The business has a rating of 4.0 stars.", attese nel data-to-text e che non rivelano alcuna etichetta di test. Resta un aspetto che non è possibile verificare: se i dataset pubblici usati qui facessero

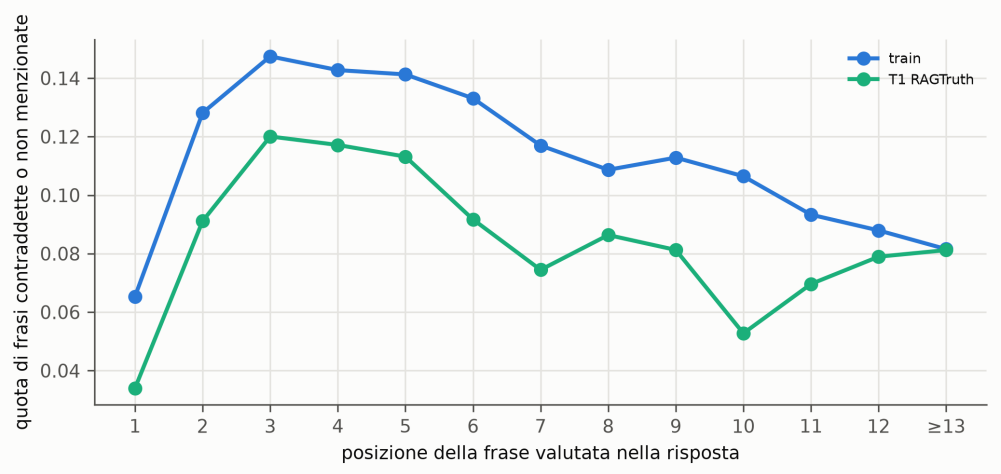


Figura 5.7: Quota di frasi contraddette o non menzionate in base alla posizione della frase nella risposta.

parte dei dati di pre-training di Qwen2.5. È un limite comune alla maggior parte dei lavori su benchmark pubblici.

Capitolo 6

Impostazione sperimentale

Questo capitolo presenta i modelli addestrati, i sistemi con cui vengono confrontati, le metriche, il protocollo statistico e il modo in cui sono misurati tempo ed energia. Tutti gli esperimenti si svolgono su un'unica macchina: un MacBook Pro con chip Apple M4 e 16 GB di memoria unificata, con CPU e GPU a 10 core ciascuna e una banda di memoria di 120 GB/s [43]. Ogni risultato riflette così ciò che un verificatore può fare sullo stesso dispositivo su cui gira un assistente locale. Anche tutte le baseline, compresi gli encoder e i due decision model generalisti, sono eseguite su questo portatile, e nessun risultato dipende da un servizio remoto.

6.1 Modelli

Entrambi i modelli GroundOne partono dalla versione di Qwen2.5 addestrata a seguire istruzioni (instruction-tuned) [23], convertita in MLX con pesi a 8 bit, e sono addestrati con la procedura descritta nella Sezione 4.3. I due modelli differiscono soltanto in tre aspetti: la dimensione, il numero di risposte di addestramento e la composizione dei micro-batch, che dipende dalla memoria disponibile.

Il modello 0.5B è addestrato sull'intero training set di 13,203 risposte. Un'epoca di 802 passi di ottimizzazione ha richiesto 5.5 ore di tempo wall-clock[1] e ha occupato al massimo 2.6 GB di memoria; ogni 150 passi il modello è stato valutato sull'intero dev set. Il modello 3B, invece, è circa sei volte più lento per token. Per questo viene addestrato nell'arco di una notte su un sottoinsieme casuale di 6,500 risposte, pari a circa 45,000 decisioni: 403 passi in 12.25 ore, con un picco di 5.9 GB. Una valutazione completa sul dev set richiederebbe circa un'ora, quindi il 3B viene

[1]Il tempo *wall-clock* è il tempo reale trascorso dall'inizio alla fine di un'operazione, come lo misurerebbe un orologio, attese comprese. Si distingue dal tempo di CPU, che conta soltanto il tempo in cui il processore lavora effettivamente.

valutato ogni 80 passi su un sottoinsieme fisso di 200 risposte. In entrambi i casi si conserva il checkpoint con la balanced accuracy a tre classi più alta sul dev set, che viene poi calibrato come descritto nella Sezione 4.4; per il 3B si tratta del passo 320.

Il seed determina l'inizializzazione degli adapter e l'ordine dei batch, ed è noto che l'esito del fine-tuning varia con entrambi questi fattori [96]. Per stimare quanto i risultati dipendano dal seed, la procedura del 0.5B viene quindi ripetuta con un secondo seed. Questa seconda esecuzione è stata portata a termine ed è valutata su tutti e tre i test set esattamente come la prima. Un'ulteriore esecuzione con un terzo seed, invece, è collassata presto su una predizione costante ed è stata interrotta a circa il 60 % dell'epoca. Essendo incompleta, è esclusa dalle tabelle e viene discussa nella Sezione 7.2.

6.2 Baseline

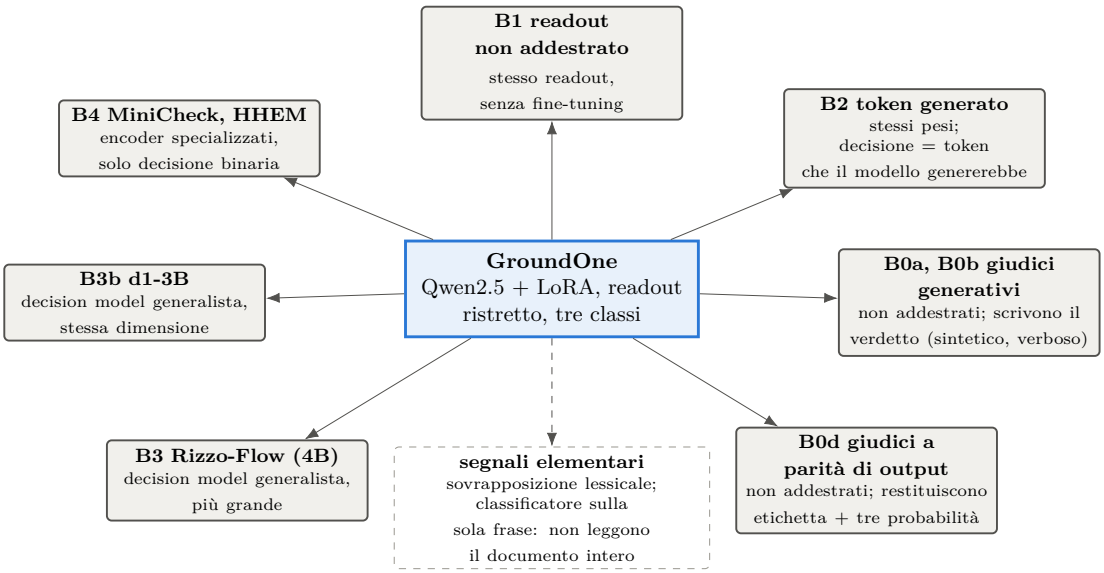
Le baseline servono a isolare l'origine di un eventuale vantaggio di GroundOne, e ciascuna risponde a una domanda precisa. La Figura 6.1 riassume che cosa ciascuna baseline cambia rispetto a GroundOne.

Segnali che non leggono mai il documento nella sua interezza. Due predittori elementari fissano il livello minimo che qualsiasi verificatore deve superare. Il primo assegna a una frase un punteggio pari alla frazione delle sue parole di contenuto che compaiono anche nel documento. Il secondo è un classificatore bag-of-words[2] addestrato sulle sole frasi del training set, senza il documento. Entrambi sono descritti nella Sezione 5.6.

Readout senza addestramento (B1). Agli stessi modelli Qwen2.5, senza fine-tuning, si pone per ogni frase la domanda a tre classi, e si leggono i logit delle lettere di risposta. Un modello non addestrato non ha alcun motivo di interpretare il marker `Verdict:` come una domanda. Per questo la baseline adotta il formato con una domanda esplicita per ogni frase (Sezione 4.2). In questo modo misura ciò che il meccanismo del decision model offre prima di qualunque specializzazione.

Giudici generativi (B0a, B0b). I modelli non addestrati sono impiegati anche nel modo più diffuso nella pratica, cioè come giudici. Ricevono il documento, la risposta numerata e l'elenco delle frasi da verificare, e scrivono un verdetto per

[2]Un classificatore *bag-of-words* rappresenta un testo con le parole, o le brevi sequenze di parole, che contiene e con le loro frequenze, ignorandone l'ordine.



ogni baseline isola un aspetto: l'addestramento (B1), il readout (B2), la generazione del verdetto (B0a, B0b, B0d), la specializzazione (B3, B3b), l'architettura (B4) e il ruolo del documento (segnali elementari)

Figura 6.1: Mappa delle baseline. Ciascuna differisce da GroundOne per un aspetto preciso e risponde quindi a una domanda diversa: quanto conta l'addestramento (B1), quanto il readout ristretto rispetto al token generato (B2), quanto il modo di produrre il verdetto (giudici B0a, B0b, B0d), quanto la specializzazione rispetto a un decision model generalista (B3, B3b), quanto l'architettura (encoder B4) e quanto basta senza leggere il documento (segnali elementari).

ogni frase con greedy decoding[3]. Il giudice *sintetico* produce una riga "`<number>: <letter>`" per frase. Il giudice *verboso*, invece, scrive per ogni frase una breve frase di ragionamento seguita dal verdetto. Quando un verdetto non può essere interpretato, si ha un errore di formato: nel calcolo dell'accuratezza viene contato come errore, e viene inoltre riportato a parte.

Giudici a parità di output (B0d). I giudici sintetico e verboso restituiscono meno di GroundOne: una lettera per frase e nessuna probabilità. Non consentono quindi né la calibrazione, né una soglia regolabile, né la scelta di una policy di decisione. Per questo a due ulteriori giudici si chiede esattamente ciò che produce GroundOne, cioè un'etichetta e tre probabilità per frase. Il primo scrive etichetta e probabilità in un oggetto JSON, esprimendo le probabilità come testo: è quanto

[3]Nel *greedy decoding* il modello sceglie a ogni passo il token più probabile, senza campionamento: a parità di input, quindi, l'output è sempre lo stesso.

uno sviluppatore otterrebbe con il solo prompt. Il secondo è il giudice sintetico, le cui tre probabilità sono lette dai logit nella posizione della lettera che genera. Ottiene così lo stesso contenuto al costo del giudice sintetico, ed è l'avversario generativo più forte. Entrambi sono calibrati con la stessa trasformazione usata per GroundOne, una temperatura e un bias, stimati per ciascun giudice su 200 risposte del dev set: in questo modo le loro probabilità ricevono lo stesso trattamento. Il prompt JSON è stato scelto su dodici risposte del dev set, badando unicamente alla validità del formato. I modelli copiavano alla lettera un esempio numerico presente nel prompt, che è stato perciò sostituito da segnaposto. Il parsing è permissivo, a vantaggio dei giudici: di un oggetto troncato si conservano le voci complete, e un'etichetta mancante si ricava dalle probabilità. Il decoding, infine, non è vincolato da una grammatica [87], così che il protocollo resti quello degli altri giudici. Questa scelta pesa soprattutto sugli errori di formato del modello più piccolo.

Readout e generazione a confronto (B2). I modelli GroundOne addestrati sono eseguiti con lo stesso prompt e lo stesso singolo forward pass, ma con una diversa regola di decisione. Il verdetto a ogni marker non è scelto fra le sole tre lettere, ma è il token con la probabilità più alta sull'intero vocabolario, cioè quello che produrrebbe la generazione greedy di un solo token. Le tre lettere, con o senza spazio iniziale, corrispondono alle tre classi, e qualunque altro token è un errore di formato. Questo controllo lascia invariati pesi, prompt e costo, e cambia soltanto un aspetto: se la decisione sia ristretta o meno alle opzioni ammissibili.

Un Decision Model generalista: Rizzo-Flow (B3). Il progetto open Rizzo-Flow [16] realizza l'idea del decision model con un modello da 4B parametri, sottoposto a fine-tuning su decisioni tipizzate generiche. Gira in un proprio server, basato su llama.cpp [2], e viene interrogato via HTTP. Il documento e la risposta numerata gli vengono passati come stato condiviso, cioè come contesto comune a tutte le domande, insieme a una domanda a scelta fra tre opzioni per ogni frase. Si usa il modello quantizzato a 4 bit, perché quello a 8 bit non entra in memoria insieme al sistema operativo.

Un Decision Model generalista della stessa dimensione: d1-3B (B3b). d1-3B di Liquid AI [18] è un decision model open da 3.1B parametri, post-addestrato per decisioni tipizzate su un'ampia varietà di task. Legge la decisione da lettere di risposta ristrette e condivide lo stato tra le domande. Appartiene quindi alla stessa famiglia di readout di GroundOne e ha la dimensione del suo modello più grande, ma, a differenza di GroundOne, è generalista. Il confronto con d1-3B isola così il valore della specializzazione rispetto a un decision model già addestrato a decidere, anziché rispetto a un modello non addestrato. Il modello viene eseguito zero-shot

con il proprio codice di inferenza, in PyTorch sulla GPU del portatile e senza accesso alla rete. Riceve lo stesso stato e le stesse domande di Rizzo-Flow, con al più sei domande per chiamata sullo stesso stato. Su LLM-AggreFact i documenti sono suddivisi esattamente come per GroundOne, e per ogni frase si conserva la probabilità di supporto più alta fra i frammenti. Ai fini della riproducibilità conta un dettaglio pratico. Sulla GPU Apple, PyTorch conserva i kernel compilati, cioè i programmi eseguiti dalla GPU, per ogni nuova lunghezza di input, e non ne rilascia mai la memoria. Per questo la valutazione si svolge in processi worker che vengono riavviati quando la memoria occupata dal driver della GPU supera 11 GB; i riavvii non hanno alcun effetto sugli output.

Encoder specializzati (B4). MiniCheck-RoBERTa [12] e HHEM-2.1-open [13] sono i verificatori di piccole dimensioni più forti oggi disponibili. Valutano ogni coppia (documento, frase) in modo indipendente e restituiscono un'unica probabilità di supporto. Come in MiniCheck, i documenti lunghi vengono suddivisi in frammenti di circa 400 parole, e per ogni frase si conserva la probabilità di supporto più alta fra i frammenti. Gli encoder non hanno un output a tre classi, quindi sono confrontati soltanto sulla decisione binaria. Girano in PyTorch sulla GPU del portatile.

Risultati pubblicati. Per LLM-AggreFact si riportano anche i punteggi di sistemi più grandi presenti nella leaderboard pubblica, come Qwen2.5-7B usato zero-shot. Questi punteggi sono ripresi così come pubblicati, senza rieseguire i sistemi.

I sistemi più lenti sono valutati su sottocampioni stratificati dei test set[4]. Rizzo-Flow è valutato su 300 risposte di T1, 600 coppie di T3 e 1,100 elementi di T2, di cui ne ha valutati 1,093, perché sette eccedevano il suo contesto. Il modello d1-3B è valutato su un sottocampione casuale di 4,000 elementi di T2, il 3B non addestrato su 600 risposte di T1 e i giudici generativi sulle 200 risposte del protocollo di latenza descritto più avanti. Sono invece valutati sui test set completi GroundOne, il 0.5B non addestrato, gli encoder e, su T1 e T3, d1-3B. Ogni confronto con un sistema sottocampionato è condotto sugli elementi valutati da entrambi i sistemi.

6.3 Metriche

La quota di frasi allucinate è diversa nel dev set e in T1 (Sezione 5.6), quindi le metriche primarie sono scelte in modo da non dipendere dal tasso di base delle

[4]Un sottocampione è *stratificato* quando viene estratto separatamente da ciascun sottogruppo (strato) dell'insieme di partenza, con un numero di elementi fissato per ogni strato, invece che dall'insieme intero in modo puramente casuale.

classi. Per la decisione a tre classi la metrica principale è la balanced accuracy, cioè la recall media sulle tre classi. Accanto a essa si riportano la recall di ciascuna classe e la frazione di predizioni assegnate a ciascuna classe, che rivela un modello collassato su un'unica risposta. Per la decisione binaria si riportano due metriche: la balanced accuracy con soglia 0.5, come nella leaderboard di LLM-AggreFact, e l'AUROC di $P(\text{supported})$, che non dipende da alcuna soglia. La calibrazione è misurata con l'ECE definito nella Sezione 4.4, con il Brier score e con la negative log-likelihood. La selective prediction è misurata con le curve risk–coverage e con l'area sottesa a esse (AURC) [95]. Tutte le metriche su T1 sono riportate anche per task e per fascia di K.

6.4 Protocollo statistico

Ogni differenza tra due sistemi è accompagnata da un intervallo di confidenza al 95 %, ottenuto con un paired bootstrap con $B = 2{,}000$ resampling [115, 116, 117]. Il bootstrap ricalcola la differenza su molti campioni estratti con reimmissione dal test set, e la dispersione dei valori ottenuti ne misura l'incertezza. L'unità di resampling, però, non è la frase, ma il gruppo impiegato nel controllo del leakage: il documento sorgente in RAGTruth, il documento in LLM-AggreFact e la pagina di Wikipedia in VitaminC. Le frasi di una stessa risposta, e le risposte relative a uno stesso documento, sono infatti fortemente correlate, e ricampionarle in modo indipendente porterebbe a intervalli troppo stretti. Su LLM-AggreFact i gruppi sono ricampionati all'interno di ciascuno degli undici dataset, perché la media macro assegna a ogni dataset lo stesso peso. Il bootstrap è *paired* perché i due sistemi di un confronto sono valutati sugli stessi resampling. Il p-value riportato è bilaterale: è il doppio della quota di resampling in cui la differenza ha segno opposto a quello osservato, con un minimo di $1/B$, dove B è il numero di resampling. La Figura 6.2 schematizza la procedura.

Due limiti del protocollo vanno dichiarati in anticipo. Il primo è che si eseguono circa cinquanta confronti senza alcuna correzione per confronti multipli[5]. Per questo le conclusioni si fondano su intervalli ben lontani dallo zero, e nessuna differenza con un p-value prossimo a 0.05 viene considerata acquisita. Il secondo è che il resampling dei dati non coglie la variabilità dovuta all'addestramento, che le due esecuzioni del 0.5B documentano a parte (Sezione 7.2).

[5]Quando si eseguono molti test statistici, cresce la probabilità che alcune differenze risultino significative per puro caso. Le correzioni per confronti multipli, come quella di Bonferroni, compensano questo effetto rendendo più severa la soglia di ciascun test.

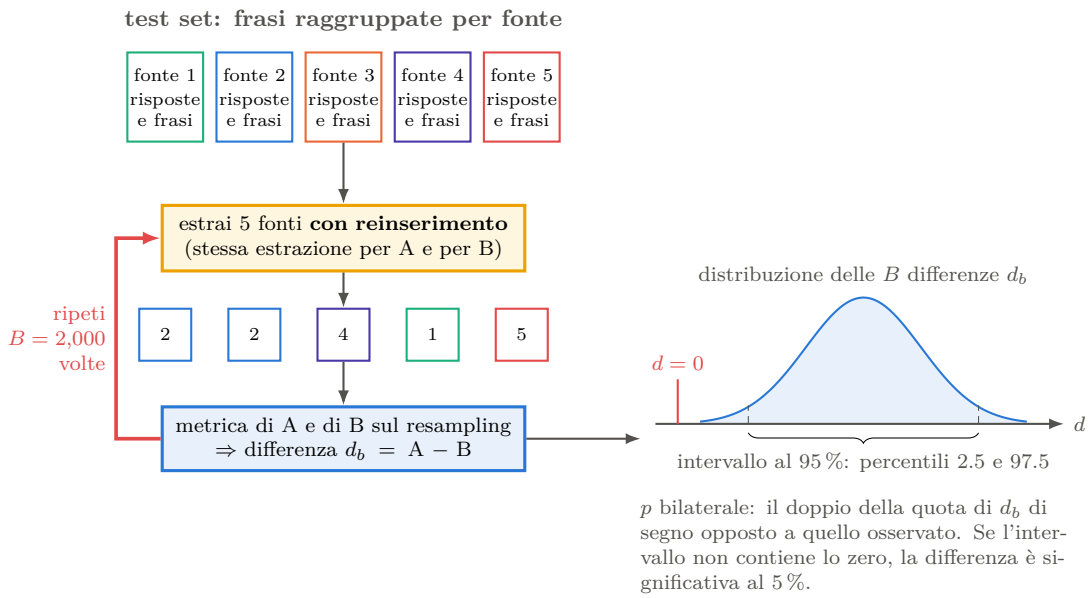


Figura 6.2: Bootstrap appaiato a cluster. Le frasi del test set sono raggruppate per fonte; a ogni ripetizione si estraggono le fonti con reinserimento, si calcola la metrica dei due sistemi sulle stesse fonti estratte e se ne registra la differenza. Dopo $B = 2{,}000$ ripetizioni, i percentili 2.5 e 97.5 delle differenze danno l'intervallo al 95 %, e la quota di differenze di segno opposto a quello osservato dà il p-value.

6.5 Latenza, energia e hardware

La latenza è misurata su 200 risposte di T1, 50 per ciascuna fascia di K, così che le risposte lunghe siano rappresentate quanto quelle brevi. GroundOne, il readout con una domanda per frase e tutti i giudici generativi girano in-process su MLX, cioè all'interno dello stesso processo che li invoca, e verificano una risposta alla volta, come farebbe un verificatore integrato in un assistente locale. Le prime tre risposte servono da warm-up e sono escluse dalla misura. Per GroundOne e per il readout con una domanda per frase ogni risposta è cronometrata tre volte e si conserva la mediana. I giudici generativi, invece, sono cronometrati una sola volta, perché il greedy decoding è deterministico e ogni esecuzione dura molto di più. Il tempo riportato è il tempo wall-clock della chiamata al modello per una risposta, e da esso si ricava il tempo per decisione.

I tempi sono stati registrati anche per gli encoder, per Rizzo-Flow e per d1-3B, ma in condizioni che non permettono un confronto equo. Gli encoder giravano in PyTorch e valutavano ogni frase separatamente, rileggendo ogni volta il documento, il che li penalizza. Rizzo-Flow girava in un altro motore di inferenza, con pesi a 4

bit e dietro un'interfaccia HTTP. Il modello d1-3B, infine, girava in PyTorch con il proprio codice e con i riavvii dei worker descritti sopra. Per questi sistemi, quindi, non si trae alcuna conclusione sulla latenza.

L'energia è misurata con `powermetrics`, lo strumento di macOS che riporta ogni 200 millisecondi la potenza assorbita da CPU, GPU e Neural Engine, la NPU dei processori Apple. Per ciascun sistema si misura dapprima, per 60 secondi, la potenza a riposo con il modello già caricato. Si verificano poi, una sola volta, 100 risposte del sottoinsieme di latenza, 25 per fascia di K. L'energia di una verifica è la potenza media durante l'esecuzione, al netto della potenza a riposo, moltiplicata per la durata dell'esecuzione e divisa per il numero di risposte. La misura comprende le unità di calcolo del chip, ma non la memoria, lo schermo o l'archiviazione. È stata effettuata di notte, con la macchina altrimenti inattiva.

Tre misure riguardano direttamente la località. La prima verifica l'indipendenza dalla rete: una valutazione completa di T1 viene ripetuta con le librerie in modalità offline e all'interno di una sandbox di macOS che nega al processo qualsiasi accesso alla rete, e le sue predizioni vengono confrontate con quelle dell'esecuzione originale. La seconda misura l'effetto della quantizzazione: lo stesso adapter del modello 0.5B viene applicato ai pesi originali a 16 bit (bf16) e valutato su T1 e T3. La terza misura la co-residenza: in un unico processo vengono caricati il generatore previste per l'applicazione dimostrativa, Qwen2.5-3B-Instruct quantizzato a 4 bit, e un verificatore GroundOne; per 30 documenti di T1, dieci per task, il generatore scrive una risposta di al più 300 token e GroundOne la verifica. Si registrano il picco di memoria allocata da MLX e i tempi di generazione e di verifica. Il documento è fornito direttamente, come in RAGTruth, per cui il retriever non è incluso. Anche queste misure girano interamente nella sandbox priva di rete.

In ogni esperimento è in esecuzione un solo modello alla volta, e una guardia di memoria (memory guard) sorveglia ogni job lungo, registrando per tutta la sua durata la memoria libera e lo swap. I risultati vengono scritti una sola volta e mai sovrascritti, e ogni esecuzione conserva la propria configurazione. In questo modo ogni numero del Capitolo 7 può essere ricondotto all'esecuzione che lo ha prodotto.

Capitolo 7

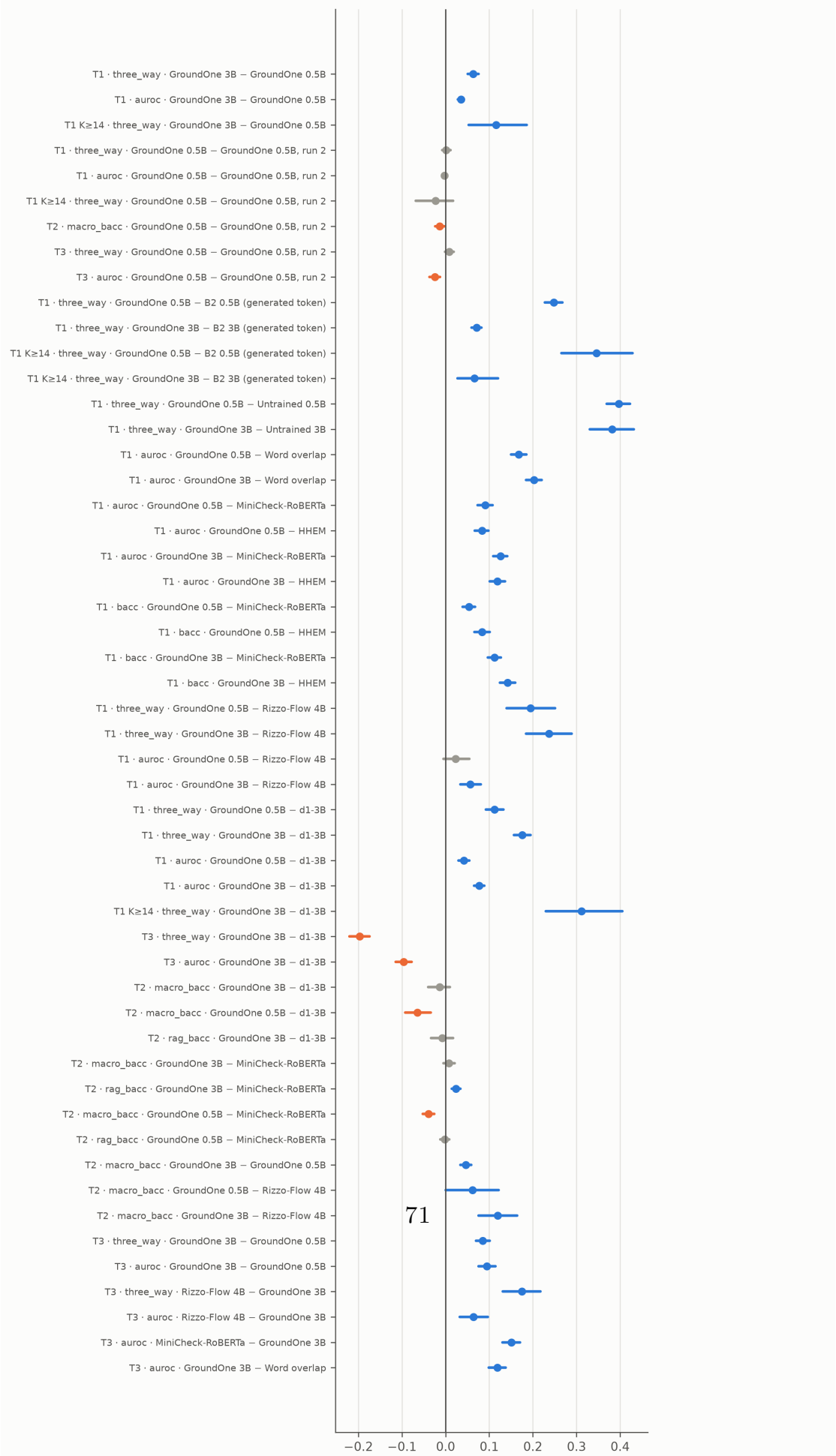
Risultati

Questo capitolo riporta i risultati sperimentali. Si parte da come GroundOne apprende il task e da quanto il suo addestramento sia stabile. Seguono i risultati sui tre test set congelati, il confronto con le baseline e il contributo che il readout del decision model dà di per sé. Il capitolo esamina poi la qualità delle probabilità, il tempo e l'energia necessari per verificare una risposta e, infine, gli errori che restano. Gli intervalli, riportati tra parentesi nella forma "da a a b", sono intervalli di confidenza al 95 % ottenuti con un bootstrap a cluster, calcolati come descritto nella Sezione 6.4. La Figura 7.1 raccoglie in un unico grafico le differenze appaiate discusse nel capitolo.

7.1 Addestramento

Entrambi i modelli hanno appreso il task. Prima dell'addestramento, un modello che riceve il formato con i marker non sa che cosa il marker gli chieda, e i suoi verdetti sono casuali: il 0.5B risponde "contraddetta" per l'88 % delle frasi del dev set e raggiunge una balanced accuracy a tre classi di 0.32. Nelle esecuzioni completate, invece, la loss di addestramento è scesa già nei primi cento passi sotto il valore di un modello che conosca soltanto le frequenze delle classi ($\ln 3 \approx 1.10$), e il punteggio sul dev set è poi cresciuto con regolarità (Figura 7.2).

Il 0.5B è stato addestrato su tutte le 13,203 risposte e ha raggiunto il punteggio migliore al passo 600 di 802, con una balanced accuracy media sui tre task di 0.633; da quel punto in poi il punteggio si è stabilizzato. Il 3B, addestrato per 12.25 ore su 6,500 risposte, ha raggiunto 0.705 al passo 320 di 403, misurato sul proprio sottoinsieme del dev set. Sull'intero dev set la balanced accuracy a tre classi è di 0.731 per il 0.5B e di 0.784 per il 3B. In entrambe le esecuzioni, inoltre, tutte e tre le classi sono rimaste presenti fra le predizioni per tutta la durata dell'addestramento.



differenza A − B con intervallo bootstrap al 95 % (blu: meglio A, arancione: meglio B, grigio: non significativa)

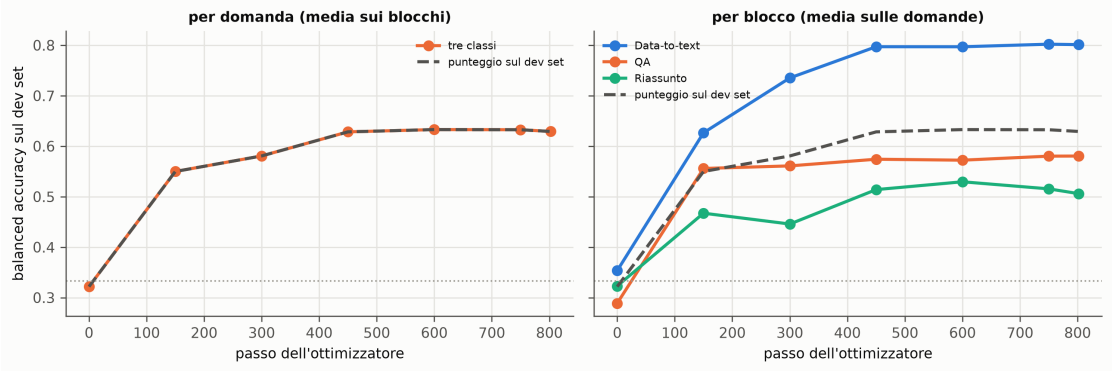


Figura 7.2: Balanced accuracy a tre classi sul dev set durante l'addestramento del modello 0.5B: valore complessivo (a sinistra) e valore per task (a destra). La linea punteggiata indica il livello casuale. Il 3B mostra lo stesso andamento sul proprio sottoinsieme di 200 risposte del dev set.

7.2 Variabilità tra seed

Per capire quanto i risultati dipendano dal seed, la procedura del 0.5B è stata ripetuta con un secondo seed, che cambia l'inizializzazione degli adapter e l'ordine dei batch. Quando l'addestramento converge, converge allo stesso punto (Figura 7.3). Su T1 la seconda esecuzione raggiunge una balanced accuracy a tre classi di 0.767, contro 0.769 della prima, e un AUROC di 0.907 contro 0.905. Su LLM-AggreFact la balanced accuracy media macro è 71.5 contro 70.1, e su VitaminC la balanced accuracy a tre classi è 0.434 contro 0.442. Su T1 la differenza appaiata tra le due esecuzioni, calcolata come prima meno seconda, è +0.001 (da −0.008 a +0.011) e non è significativa.

Le differenze emergono nei dettagli. Anzitutto, gli errori si distribuiscono in modo diverso tra le classi: la recall delle frasi supportate e quella delle frasi non menzionate si spostano di circa 0.04, in direzioni opposte. Inoltre, le porzioni più piccole dei dati oscillano di più. La balanced accuracy della fascia con quattordici o più decisioni varia di 0.023, l'AUROC su VitaminC di 0.025, e i singoli dataset di LLM-AggreFact variano fino a dieci punti, come WiCE (67.9 contro 78.0).

Da questi risultati seguono due conseguenze. La prima riguarda gli intervalli: il bootstrap ricampiona i dati di test, e quindi non coglie la variabilità dovuta all'addestramento. Su LLM-AggreFact e su VitaminC, infatti, le due esecuzioni della stessa procedura differiscono più di quanto i loro intervalli consentirebbero (prima meno seconda esecuzione: −0.013 sulla media macro di T2, −0.025 sull'AUROC di T3). Per questo non vengono interpretate le differenze tra sistemi inferiori alla dispersione tra seed, cioè a circa 1.5 punti sulla media macro di LLM-AggreFact e a 0.025 di AUROC su VitaminC. Lo stesso vale per i singoli dataset di LLM-AggreFact

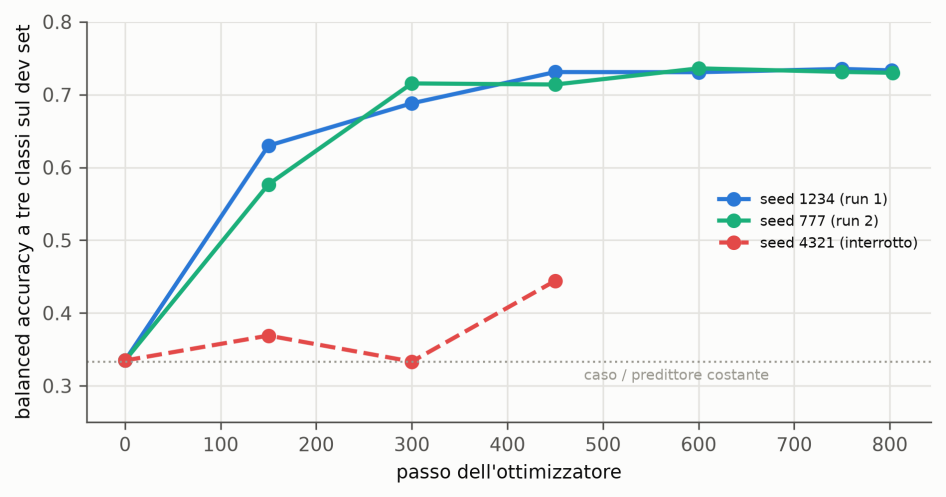


Figura 7.3: Balanced accuracy a tre classi sul dev set durante l'addestramento del modello 0.5B, con la stessa procedura e tre seed diversi. Due esecuzioni convergono allo stesso livello; la terza è collassata su un predittore costante ed è stata interrotta.

quando lo scarto non supera circa cinque punti.

La seconda conseguenza è che l'addestramento non sempre converge. Un'esecuzione con un terzo seed è rimasta vicina al livello casuale. Il suo punteggio sul dev set era 0.369 al passo 150 e 0.333 al passo 300, quando il modello prediceva "supportata" per ogni frase, e la loss di addestramento è rimasta prossima a quella di un modello privo di informazione. L'esecuzione è stata interrotta al passo 470 di 801, quando il punteggio era risalito a 0.444. Non è quindi possibile sapere se si sarebbe ripresa entro la fine dell'epoca e, poiché incompleta, è esclusa dalle tabelle. La procedura ha due caratteristiche note per rendere fragile il fine-tuning: non applica gradient clipping, e l'ottimizzatore lavora senza bias correction. L'assenza della bias correction è l'impostazione predefinita del framework ed è stata indicata come una delle cause principali dei fine-tuning non riusciti [98, 97]. Un solo collasso su tre esecuzioni non basta a stimare un tasso di fallimento, ma mostra che la procedura non è del tutto robusta rispetto all'inizializzazione.

7.3 Accuratezza sul test set RAG (T1)

Sulle 17,284 frasi di T1 il modello 0.5B raggiunge una balanced accuracy a tre classi di 0.769 e il 3B di 0.832 (Tabella 7.1). La differenza tra i due, +0.063 (da +0.052 a +0.076), è l'effetto più netto della dimensione del modello. Sulla decisione binaria il 0.5B ottiene una balanced accuracy di 0.815 e un AUROC di 0.905, mentre il 3B ottiene rispettivamente 0.874 e 0.940. Entrambi superano di gran lunga i

segnali che non confrontano mai una frase con il suo documento: la sovrapposizione lessicale arriva a un AUROC di 0.737, e il classificatore che vede soltanto la frase arriva a una balanced accuracy a tre classi di 0.570.

Tabella 7.1: GroundOne su T1, sull'intero test set, per task e per numero di decisioni per risposta (K). Per ciascun modello sono riportate la balanced accuracy a tre classi, la recall delle frasi contraddette e l'AUROC di P(supported).

		GroundOne 0.5B			GroundOne 3B		
Sottoinsieme	Frasi	Bal. acc.	Contr.	AUROC	Bal. acc.	Contr.	AUROC
Tutte	17,284	0.769	0.68	0.905	0.832	0.76	0.940
Question answering	4,876	0.589	0.03	0.916	0.699	0.30	0.948
Summarization	4,538	0.544	0.13	0.775	0.629	0.30	0.873
Data-to-text	7,870	0.812	0.84	0.929	0.873	0.90	0.955
$K = 1$–4	2,434	0.606	0.18	0.857	0.677	0.25	0.921
$K = 5$–8	8,012	0.758	0.67	0.899	0.829	0.78	0.936
$K = 9$–13	5,201	0.824	0.82	0.926	0.863	0.86	0.945
$K \geq 14$	1,637	0.723	0.50	0.908	0.839	0.68	0.957

Dalla tabella emergono tre aspetti. Il primo è che l'accuratezza non peggiora con la lunghezza della risposta. Le risposte con quattordici o più decisioni sono tra quelle gestite meglio, con una balanced accuracy di 0.723 per il 0.5B e di 0.839 per il 3B. La fascia più difficile è invece quella delle risposte più brevi, che comprende soprattutto risposte di question answering e di summarization. Il dato è importante perché è proprio sulle risposte lunghe che un decision model fa risparmiare più tempo; su queste risposte il 3B guadagna sul 0.5B +0.116 (da +0.050 a +0.180).

Il secondo aspetto è che le contraddizioni vengono riconosciute soprattutto nel data-to-text, il task in cui RAGTruth le concentra. Nel question answering e nella summarization il 0.5B distingue bene le frasi supportate da quelle non menzionate, ma non individua quasi nessuna contraddizione: la recall delle frasi contraddette è 0.03 e 0.13. Il 3B la porta a 0.30 in entrambi i task, quindi la dimensione del modello compensa in parte la scarsità di questi esempi nei dati di addestramento. Il terzo aspetto è che question answering e summarization offrono la prova più chiara che i modelli leggono davvero il documento. In questi due task il classificatore che vede soltanto la frase resta al livello casuale (0.36 e 0.35), mentre i due modelli raggiungono 0.59–0.70 nel question answering e 0.54–0.63 nella summarization.

Le stesse predizioni si possono valutare anche a livello di risposta intera. È il criterio con cui il Decision Index della comunità valuta i decision model generalisti su RAGTruth: una risposta è allucinata se contiene un qualsiasi contenuto non supportato, e la metrica è l'F1 di questa classe. Un sistema che risponda sempre "allucinata" raggiunge già un F1 di 0.52 [22]. Per giudicare una risposta, GroundOne

prende il minimo della P(supported) calibrata sulle sue frasi e lo confronta con una soglia stimata sul dev set. Con questa regola, sulle 2,647 risposte di T1 raggiunge un F1 di 0.733 con il 0.5B e di 0.784 con il 3B. Nella leaderboard, invece, nessun decision model generalista sotto il miliardo di parametri supera 0.55, Jev raggiunge 0.765 e il modello migliore, da 27B parametri, 0.803. Il confronto è solo indicativo, perché GroundOne è stato addestrato su RAGTruth, mentre i modelli della leaderboard sono valutati zero-shot, con un prompt diverso e su 53 risposte in più. Colloca comunque uno specialista da 0.5B molto al di sopra dei decision model generalisti della stessa dimensione.

7.4 Confronto con le baseline

La Figura 7.5 mette a confronto tutti i sistemi su T1. Gli encoder specializzati raggiungono un AUROC di 0.814 (MiniCheck-RoBERTa) e di 0.821 (HHEM): più della sovrapposizione lessicale, ma meno di entrambi i modelli GroundOne. Il 0.5B supera MiniCheck-RoBERTa di +0.091 (da +0.074 a +0.107) e HHEM di +0.083 (da +0.066 a +0.099); il 3B li supera rispettivamente di +0.126 e +0.119. Il confronto per task, però, ridimensiona questo risultato. Il vantaggio di GroundOne viene in gran parte dal data-to-text, dove lo avvantaggiano l'addestramento nel dominio e la struttura ricorrente dei template, e dove gli encoder sono più deboli (AUROC 0.71–0.75). Nella summarization, invece, MiniCheck-RoBERTa supera il modello 0.5B (0.872 contro 0.775) ed è alla pari con il 3B (0.873). La Figura 7.4 riporta l'AUROC per task di tutti i sistemi.

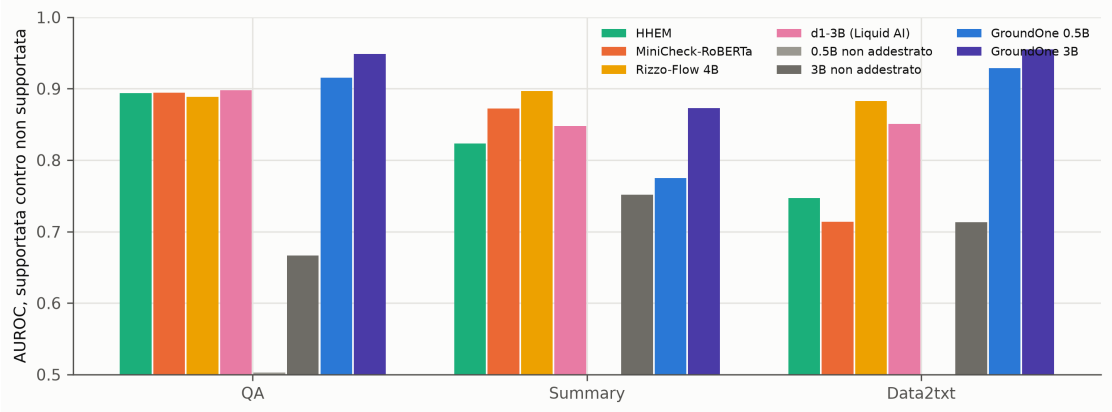


Figura 7.4: AUROC di P(supported) su T1 per task (question answering, riassunto, data-to-text) e per sistema. Il vantaggio di GroundOne si concentra nel data-to-text; nella summarization MiniCheck-RoBERTa supera il 0.5B ed è alla pari con il 3B.

I modelli non addestrati mostrano quanta parte dell'accuratezza si debba all'addestramento. Il 0.5B non addestrato è al livello casuale, perché non risponde mai "supportata". Il 3B non addestrato, a cui si pone la stessa domanda, raggiunge una balanced accuracy a tre classi di 0.437, contro 0.818 dello stesso modello dopo l'addestramento, sulle stesse risposte. Il contributo del readout ristretto, al di là dell'addestramento, è misurato separatamente nella Sezione 7.6.

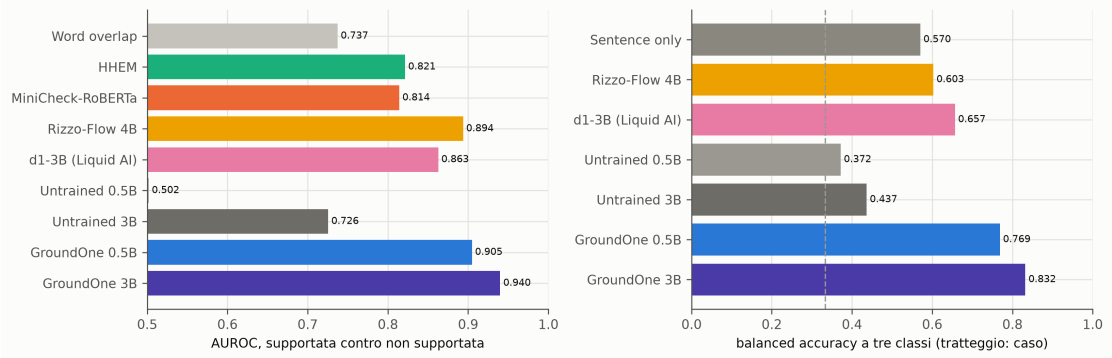


Figura 7.5: T1: AUROC di $P(\text{supported})$ per ogni sistema (a sinistra) e balanced accuracy a tre classi per i sistemi che hanno un output a tre classi (a destra). Rizzo-Flow è valutato su 300 risposte e il 3B non addestrato su 600; tutti gli altri sistemi, compreso d1-3B, sono valutati sull'intero test set.

Il confronto più istruttivo è quello con Rizzo-Flow, il decision model generalista più grande: un modello da 4B sottoposto a fine-tuning per decisioni generiche. Sulle stesse 300 risposte di T1, Rizzo-Flow raggiunge una balanced accuracy a tre classi di 0.603, contro 0.798 del GroundOne 0.5B e 0.840 del 3B. Le differenze sono ampie: +0.195 (da +0.136 a +0.249) per il 0.5B e +0.237 (da +0.184 a +0.285) per il 3B. In AUROC, invece, Rizzo-Flow (0.894) è vicino al 0.5B, che su queste risposte ottiene 0.916, e la differenza non è significativa. Rizzo-Flow, infatti, ordina bene le frasi, ma predice "supportata" per il 90 % di esse. Il 3B le ordina meglio di Rizzo-Flow (+0.056, da +0.034 a +0.079). Uno specialista otto volte più piccolo, quindi, supera il generalista nella decisione per cui è stato progettato e lo eguaglia nella capacità di ordinare le frasi.

d1-3B, un decision model generalista della stessa dimensione del 3B, offre la verifica più severa del valore della specializzazione. È infatti già addestrato a prendere decisioni tipizzate, e si differenzia da GroundOne soprattutto per i dati su cui è stato addestrato. Sull'intero T1 raggiunge una balanced accuracy a tre classi di 0.657 e un AUROC di 0.863. Entrambi i modelli GroundOne fanno nettamente meglio. Il 0.5B, sei volte più piccolo, lo supera di +0.112 (da +0.092 a +0.133) in balanced accuracy a tre classi e di +0.042 in AUROC; il 3B lo supera di +0.175 (da +0.157 a +0.195) e di +0.077. Il divario è massimo sulle risposte lunghe: quando

la risposta contiene quattordici o più decisioni, d1-3B scende a 0.527, mentre il GroundOne 3B resta a 0.839 (+0.312, da +0.230 a +0.405). Inoltre, d1-3B riconosce soltanto metà delle frasi non menzionate (recall 0.49). Nel dominio delle risposte RAG, dunque, la specializzazione vale molto più di un addestramento generico a prendere decisioni.

Su LLM-AggreFact (T2) il modello 0.5B raggiunge una balanced accuracy media macro di 70.1 sugli undici dataset, contro 64.2 dello stesso modello usato zero-shot e 72.8 del modello 7B zero-shot riportato nella leaderboard pubblica. Resta invece 3.9 punti sotto MiniCheck-RoBERTa (74.0), una differenza significativa. Sui quattro dataset più vicini alla retrieval-augmented generation i due sistemi sono alla pari (74.4 per GroundOne 0.5B, 74.6 per MiniCheck-RoBERTa). Nella colonna di RAGTruth GroundOne è in vantaggio, 82.1 contro 76.5, ma il confronto lo favorisce, perché è stato addestrato sulle risposte del training set di RAGTruth. Al di fuori del RAG, invece, il 0.5B è inferiore all'encoder di sette-nove punti su AggreFact-CNN, AggreFact-XSum, FactCheck-GPT e REVEAL: è il costo atteso di un addestramento sui soli dati RAG.

Il modello 3B, valutato sull'intero test set, colma gran parte di questo divario. La sua balanced accuracy media macro è 74.8: è alla pari con quella di MiniCheck-RoBERTa (74.0; differenza appaiata +0.8 punti, intervallo da −0.5 a +2.1) e superiore al punteggio zero-shot pubblicato per il modello 7B (72.8), per il quale non si può calcolare un intervallo. Sui dataset RAG raggiunge 76.9, 2.4 punti sopra l'encoder (intervallo da +1.4 a +3.3). Il suo guadagno rispetto al 0.5B, +4.7 punti (da +3.4 a +5.9), è diffuso tra i dataset, con i miglioramenti maggiori su REVEAL, WiCE, FactCheck-GPT e XSum: la dimensione del modello migliora quindi il trasferimento al di fuori del dominio di addestramento. Il 3B resta nettamente sotto l'encoder soltanto nella summarization di notizie (AggreFact-CNN, 59.3 contro 67.1). Rizzo-Flow raggiunge 61.5 sui 1,093 elementi del suo sottocampione, e su questi elementi il GroundOne 3B fa meglio di 12.0 punti (da +7.4 a +16.5).

Su questo benchmark, tuttavia, i sistemi più forti restano sopra GroundOne. Una valutazione indipendente del decision model closed Jev, sullo stesso test set e con la stessa metrica, riporta 78.6, e 76.9 per un modello open da 27B letto attraverso le lettere di risposta. I prompt sono diversi da quelli usati qui, e non è possibile calcolare alcun intervallo [20]. Su un benchmark ampio di groundedness, quindi, uno specialista da 3B addestrato soltanto su risposte RAG eguaglia il miglior encoder di piccole dimensioni, ma resta sotto lo stato dell'arte closed e sotto modelli open molto più grandi. Il generalista d1-3B conferma il quadro quando la dimensione è la stessa. Su un sottocampione casuale di 4,000 elementi raggiunge 75.7, contro 74.3 del GroundOne 3B sugli stessi elementi, una differenza non significativa (−1.3, da −4.0 a +1.0), mentre supera il 0.5B di 6.5 punti. Rispetto al GroundOne 3B, d1-3B è più debole nella colonna di RAGTruth (78.9 contro 87.3) e nettamente più forte su WiCE e sulla summarization di dialoghi.

7.5 Trasferimento a un dominio diverso (T3)

Su VitaminC il quadro si rovescia. Sull'intero test set il 0.5B raggiunge una balanced accuracy a tre classi di 0.442 e il 3B di 0.527, con un AUROC rispettivamente di 0.722 e di 0.817, mentre MiniCheck-RoBERTa arriva a un AUROC di 0.968. Sulle 600 coppie valutate con Rizzo-Flow, il modello generalista ottiene una balanced accuracy a tre classi di 0.712 e un AUROC di 0.909, contro 0.442 e 0.744 del 0.5B e 0.537 e 0.845 del 3B. La Figura 7.6 mostra il confronto completo su T3.

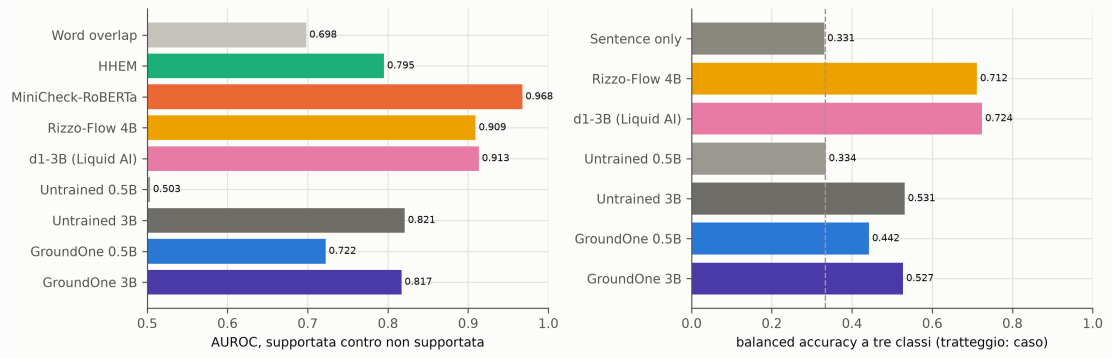


Figura 7.6: Confronto dei sistemi su T3 (VitaminC): a sinistra l'AUROC di $P(\text{supported})$, a destra la balanced accuracy a tre classi per i sistemi con un output a tre classi. Rizzo-Flow è valutato su un sottocampione di 600 coppie; il tratteggio indica il livello casuale. Per MiniCheck-RoBERTa VitaminC fa parte della genealogia dei dati di addestramento.

Due fatti aiutano a interpretare questi numeri. Il primo è che MiniCheck-RoBERTa parte dal modello AlignScore, i cui dati di addestramento comprendono VitaminC [11]. Per l'encoder, quindi, T3 appartiene alla distribuzione di addestramento, mentre per GroundOne misura il trasferimento da RAGTruth. Il secondo è che il guadagno del 3B sul 0.5B in T3 si deve alla dimensione del modello più che all'addestramento su dati RAG: il 3B non addestrato raggiunge già 0.531 e un AUROC di 0.821. d1-3B, il decision model generalista della stessa dimensione, rende esplicito il contrasto. Sull'intero T3 raggiunge 0.724 e un AUROC di 0.913, e supera il GroundOne 3B di 0.197 e di 0.096: l'esatto opposto di quanto accade su T1. La specializzazione su risposte RAG migliora quindi in modo sostanziale il task RAG, ma da sola non basta a produrre un fact checker generale. La Sezione 7.12 mostra quali casi sfuggono.

7.6 Readout e generazione a confronto

Il controllo B2 isola il contributo del readout. Il confronto mantiene invariati i pesi ottenuti con il fine-tuning, il prompt e il singolo forward pass, e cambia soltanto il modo in cui si ricava il verdetto a ogni marker. Il verdetto è letto dalle tre lettere di risposta, come fa GroundOne, oppure è il token che emetterebbe la generazione greedy (Figura 7.7). Per il 0.5B il token generato dà una balanced accuracy a tre classi di 0.521, contro 0.769 del readout, con una differenza di +0.248 (da +0.228 a +0.269) a favore del readout. In un quarto dei marker (24.7 %) il modello non emetterebbe affatto una lettera, ma per lo più un semplice spazio, cioè l'inizio di una risposta più lunga. Per il 3B, invece, il token generato è quasi sempre una lettera (1.2 % di errori di formato) e concorda con il readout nel 92 % dei marker; eppure il readout resta migliore, con 0.832 contro 0.761 (+0.071, da +0.059 a +0.083). Quando la risposta contiene quattordici o più decisioni, il divario è massimo per il modello piccolo: +0.346.

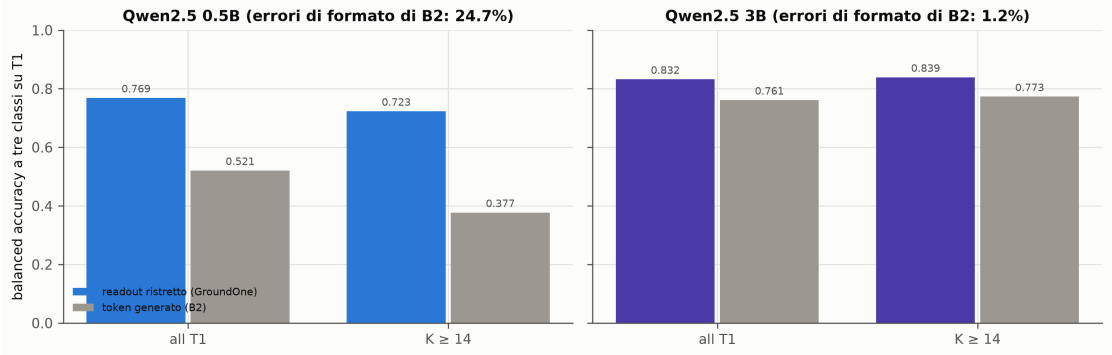


Figura 7.7: Controllo B2 sull'intero T1. Con gli stessi pesi e lo stesso forward pass, il verdetto è letto dalle tre lettere di risposta (GroundOne) oppure preso come il token che emetterebbe la generazione greedy; gli errori di formato sono contati come errori.

Le due opzioni hanno la stessa latenza, perché entrambe richiedono un solo forward pass: a cambiare è soltanto la regola di decisione. Restringere la scelta alle opzioni ammissibili elimina per costruzione gli errori di formato. Inoltre, poiché il modello è addestrato soltanto a ordinare le tre lettere, la scelta ristretta fornisce anche la decisione più accurata (Sezione 4.2). Questa restrizione conta tanto più quanto più il modello è piccolo. Si ripete spesso che leggere i logit delle risposte e generare un singolo token di risposta siano operazioni equivalenti: per un verificatore sottoposto a fine-tuning l'equivalenza vale per il costo, ma non per l'accuratezza.

7.7 Calibrazione e selective prediction

Prima della calibrazione, le probabilità sottostimano il supporto. Nel reliability diagram (Figura 7.9) la curva delle probabilità grezze giace sopra la diagonale, perché i pesi delle classi addestrano il modello come se le classi fossero bilanciate. L'ECE binario su T1 è 0.30 per il 0.5B e 0.18 per il 3B. La temperatura e il bias stimati sul dev set lo portano a 0.019 per entrambi i modelli, e a 0.020 per la seconda esecuzione del 0.5B. Riducono inoltre la negative log-likelihood da 0.55 a 0.21 per il 0.5B e da 0.36 a 0.17 per il 3B, lasciando l'AUROC praticamente invariato. La mappa stimata sul dev set si trasferisce quindi al test set dello stesso dominio. La Figura 7.8 mostra lo stesso diagramma per il 3B.

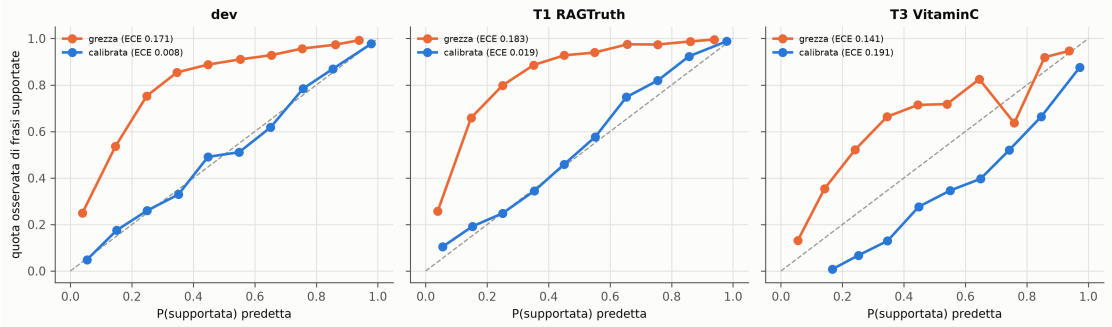


Figura 7.8: Reliability diagram di $P(\text{supported})$ del 3B, prima e dopo la calibrazione stimata sul dev set, per il dev set, T1 e T3; ogni punto è una fascia di probabilità con almeno 20 frasi. Come per il 0.5B, la calibrazione si trasferisce a T1 ma non a T3.

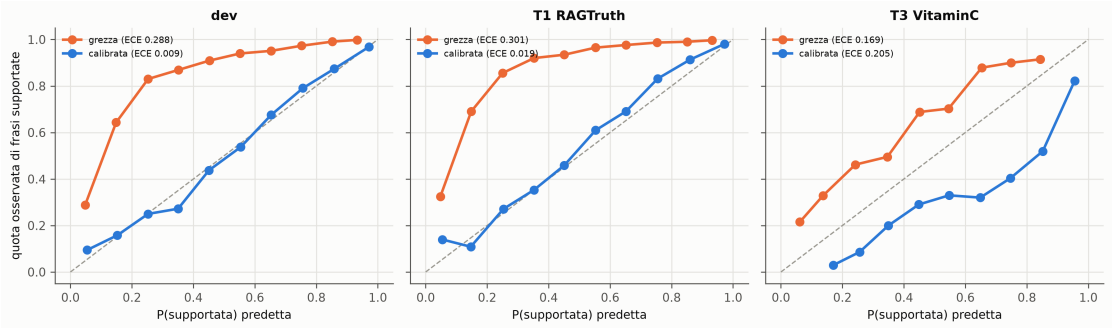


Figura 7.9: Reliability diagram di $P(\text{supported})$ del 0.5B, prima della calibrazione (grezza) e dopo la calibrazione stimata sul dev set, per il dev set, T1 e T3. Ogni punto rappresenta una fascia di probabilità che contiene almeno 20 frasi.

La calibrazione ha però una conseguenza da tenere presente. Il bias ripristina la frequenza reale di ciascuna classe, per cui la classe più probabile dopo la calibrazione diventa più spesso "supportata". Per il 0.5B su T1 l'accuratezza sale così da 0.82 a 0.93, ma la balanced accuracy a tre classi scende da 0.77 a 0.63. Quando i tre tipi di errore hanno lo stesso peso, quindi, scegliere la classe calibrata più probabile non è una buona regola di decisione. La regola con cui vengono restituite le etichette, descritta nella Sezione 4.4, risolve il problema. La regola usa una soglia τ stimata sul dev set, pari a 0.935 per il 0.5B, a 0.895 per la sua seconda esecuzione e a 0.925 per il 3B. Con queste soglie raggiunge su T1 una balanced accuracy a tre classi di 0.758, 0.769 e 0.830, entro 0.01 dalla decisione grezza valutata in tutto il capitolo, e mantiene le etichette coerenti con le probabilità calibrate. Le soglie sono alte perché il 91 % delle frasi di RAGTruth è supportato: una frase viene restituita come supportata soltanto quando il modello è molto sicuro.

Usate come confidenze, le probabilità calibrate rendono efficace la selective prediction. Rinviare le frasi su cui il modello è meno sicuro riduce rapidamente il tasso di errore della decisione binaria: su T1 l'area sotto la curva risk–coverage (AURC) è 0.015 per il 0.5B e 0.010 per il 3B (Figura 7.10).

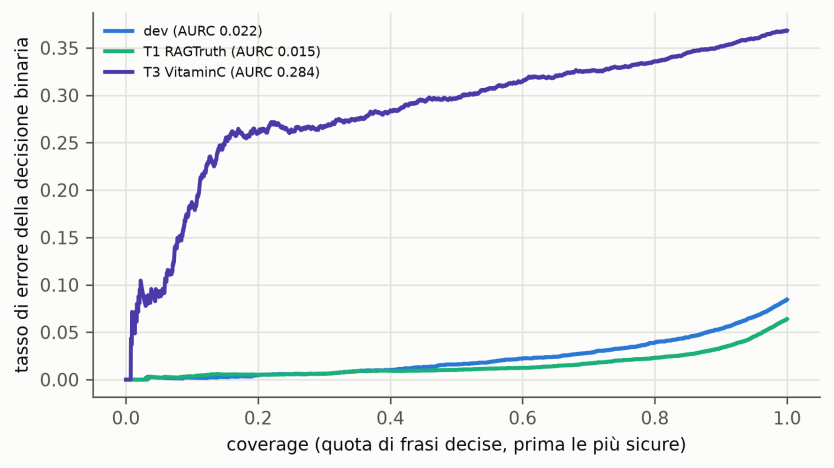


Figura 7.10: Curve risk–coverage della decisione binaria del 0.5B con probabilità calibrate: tasso di errore al crescere del numero di frasi meno sicure che vengono rinviate.

Su VitaminC, invece, la calibrazione non si trasferisce. L'ECE binario sale da 0.17 a 0.21 per il 0.5B e da 0.14 a 0.19 per il 3B, e la regola di restituzione delle etichette perde 0.02–0.04 rispetto alla decisione grezza. Il bias, infatti, codifica il tasso di base di RAGTruth, mentre VitaminC è bilanciato per costruzione. Una calibrazione vale per la distribuzione su cui è stata stimata, per cui ogni impiego reale richiede un piccolo campione etichettato dei propri dati.

7.8 Probabilità a parità di output

I giudici a parità di output servono a stabilire se un giudice generativo della stessa dimensione, a cui si chiede lo stesso contenuto di GroundOne, possa eguagliare la qualità delle sue probabilità. La risposta è negativa (Figura 7.11). Il confronto usa le 200 risposte del sottoinsieme di latenza, con tutti i sistemi calibrati allo stesso modo sul dev set. Per il 0.5B e per il 3B rispettivamente, l'AUROC di P(supported) è 0.94 e 0.97 per GroundOne, 0.48 e 0.68 per il giudice le cui probabilità sono lette dai logit, e 0.47 e 0.58 per il giudice che le scrive. L'area sotto la curva risk–coverage è 0.009 e 0.005 per GroundOne, contro valori da 0.048 a 0.063 per i giudici, cioè da cinque a dodici volte maggiori. L'accuratezza delle decisioni segue lo stesso andamento: 0.835 e 0.900 per GroundOne, al massimo 0.41 per qualsiasi giudice.

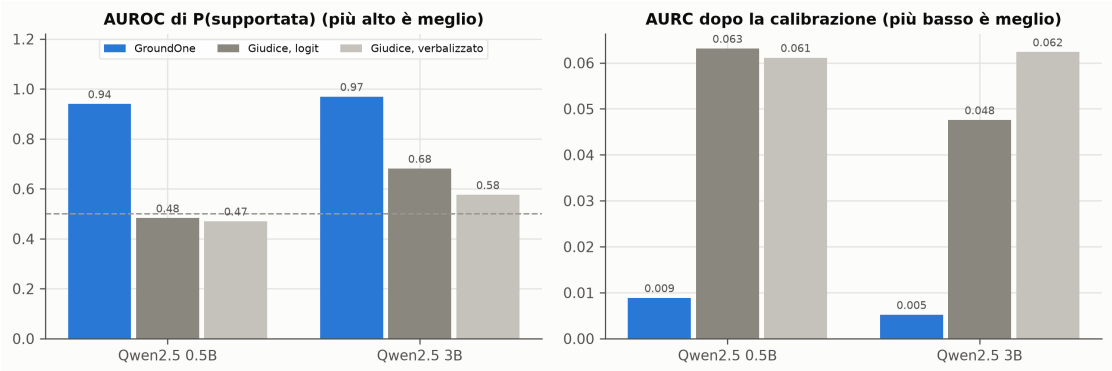


Figura 7.11: Confronto a parità di output sulle 200 risposte del sottoinsieme di latenza. Ai giudici generativi della stessa dimensione si chiede l'output di GroundOne, con le probabilità lette dai logit della lettera generata oppure scritte in JSON; tutti i sistemi sono calibrati sul dev set con la stessa mappa. Sono considerate soltanto le frasi con un verdetto valido.

Nella pratica contano altre due osservazioni. La prima è che le probabilità scritte a parole non portano quasi alcuna informazione. Il 3B scrive un vettore con tutta la massa su una sola classe per il 93 % delle frasi, e usa appena sedici vettori distinti. Il 0.5B scrive invece pareggi non informativi, come probabilità uguali per tutte le classi: se i pareggi privi di etichetta vengono contati come errori, la sua balanced accuracy scende da 0.234 a 0.081. Il dato concorda con l'osservazione che la confidenza verbalizzata dei modelli piccoli è grossolana e poco informativa [94, 93]. La seconda osservazione è che l'ECE, da solo, nasconderebbe tutto ciò. Dopo la calibrazione ogni sistema, giudici compresi, ottiene un ECE binario compreso tra 0.03 e 0.05, perché una mappa di calibrazione può allineare la confidenza media al tasso di base anche quando le probabilità non separano le

classi. La capacità discriminativa va quindi misurata a parte, con l'AUROC e con la curva risk–coverage.

Chiedere le probabilità, infine, ha un costo. Il giudice che le legge dai logit è veloce quanto il giudice sintetico, perché genera le stesse lettere. Il giudice che scrive l'oggetto JSON, invece, impiega con il 3B 17.0 secondi per una risposta con quattordici o più decisioni, quanto il giudice verboso, e non è più accurato del giudice sintetico.

7.9 Latenza in funzione di K

La Figura 7.12 riporta la misura centrale della tesi: il tempo necessario per verificare una risposta al crescere del numero di decisioni, misurato sulle stesse 200 risposte di T1. Tra i sistemi eseguiti in-process su MLX, GroundOne è il più veloce con entrambe le dimensioni e in ogni fascia di K, e il suo vantaggio aumenta con K. Quando la risposta contiene quattordici o più decisioni, il GroundOne 0.5B impiega 0.41 secondi, contro 0.80 del giudice sintetico e 2.51 del giudice verboso della stessa dimensione. È quindi 1.95 volte più veloce del primo (da 1.82 a 2.14) e 6.16 volte più veloce del secondo (da 5.72 a 6.67). Nella stessa fascia il GroundOne 3B impiega 2.57 secondi, contro 4.44 e 15.86 secondi dei due giudici 3B, ed è quindi 1.73 volte (da 1.65 a 1.82) e 6.18 volte (da 5.88 a 6.65) più veloce.

Leggere tutti i verdetti in un unico forward pass è anche più rapido che porre una domanda per frase in un batch: ai valori più alti di K il guadagno è di un fattore 1.4–1.5. L'altra leva è la dimensione del modello: il GroundOne 0.5B verifica una risposta circa sei volte più in fretta del 3B, con un tempo mediano di 0.33 secondi contro 1.99.

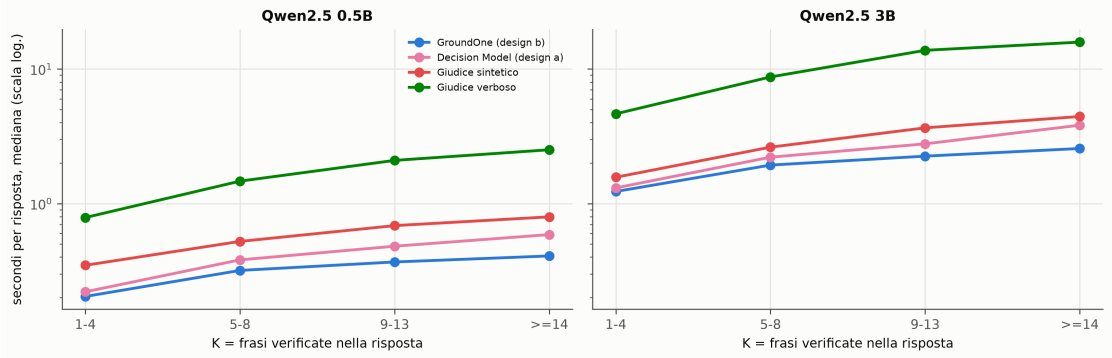


Figura 7.12: Tempo mediano per risposta in funzione di K sulle stesse 200 risposte di T1 (MLX in-process, una risposta alla volta, M4 con 16 GB). Sono confrontati GroundOne, il decision model con una domanda per frase e i giudici generativi sintetico e verboso. Scala logaritmica.

La velocità non è l'unica differenza (Figura 7.13). I giudici generativi non addestrati sono anche molto meno accurati, e non sempre rispettano il formato di output. Il giudice verboso 0.5B produce un verdetto interpretabile soltanto per il 3 % delle frasi, perché per lo più ricopia la risposta, mentre il giudice sintetico 0.5B fallisce sul 23 % delle frasi. I giudici 3B rispettano il formato molto meglio (0.1 % di errori di formato per il giudice sintetico e 7.6 % per quello verboso formato), ma si fermano a una balanced accuracy a tre classi di circa 0.40, contro 0.90 del GroundOne 3B sulle stesse risposte. Il giudice verboso 3B, infine, scrive in media 248 token per risposta, che vanno tutti generati prima di poter usare il verdetto; GroundOne, invece, non ne genera nessuno.

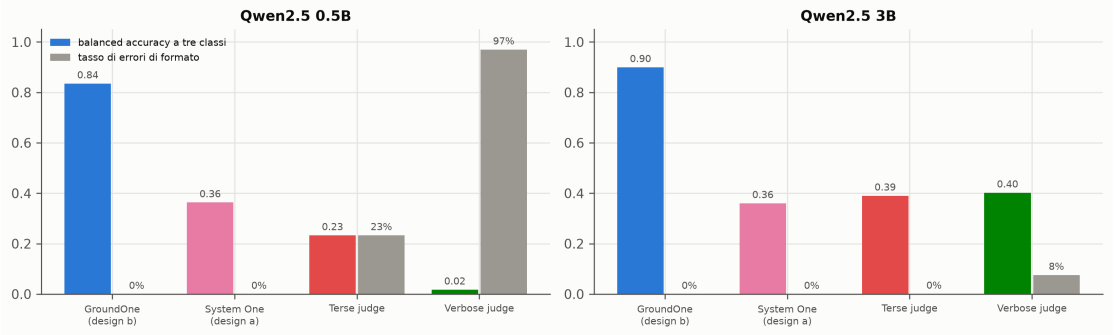


Figura 7.13: Giudici generativi sul sottoinsieme di latenza: balanced accuracy a tre classi, con gli errori di formato contati come errori, e quota di frasi il cui verdetto non è stato possibile interpretare.

7.10 Energia per verifica

La misura dell'energia aggiunge una precisazione che la sola latenza nasconde (Figura 7.14). GroundOne è il sistema che consuma meno energia con entrambe le dimensioni: 5.5 joule per risposta con il 0.5B e 33 con il 3B, contro 8.9 e 43 joule del giudice sintetico e 22.5 e 112 joule del giudice verboso. Il suo vantaggio sull'energia, tuttavia, è minore di quello sul tempo. Rispetto al giudice sintetico, GroundOne consuma 1.6 volte meno energia con il 0.5B e 1.3 volte meno con il 3B; rispetto al giudice verboso, 4.1 e 3.4 volte meno.

Il motivo è la potenza assorbita durante il calcolo. GroundOne trascorre il suo tempo in un unico, lungo prefill, che tiene la GPU pienamente occupata e assorbe circa 16 W oltre la potenza a riposo. Il decoding autoregressivo dei giudici, invece, è limitato dalla banda di memoria e assorbe 11–14 W, ma per un tempo molto più lungo. È il comportamento atteso dalle due fasi dell'inferenza dei transformer [40],

e implica che uno speed-up non vada mai letto come un risparmio di energia equivalente [66, 67].

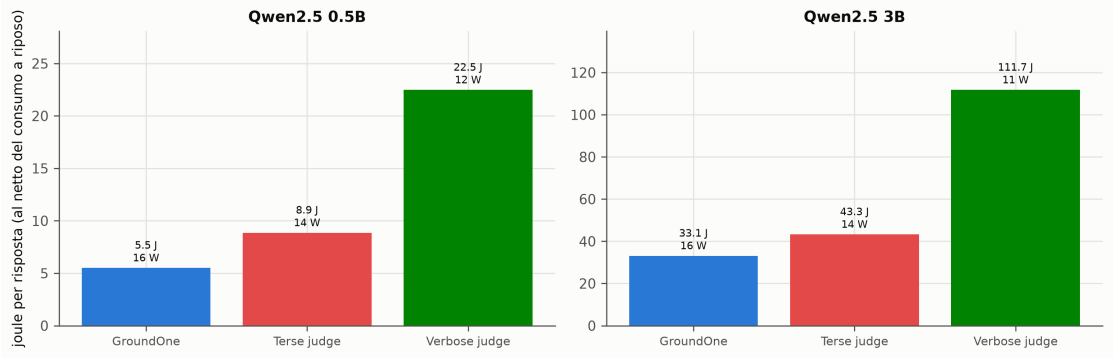


Figura 7.14: Energia per risposta verificata su 100 risposte del sottoinsieme di latenza, misurata con powermetrics su CPU, GPU e Neural Engine al netto della potenza a riposo, insieme alla potenza netta media durante l'esecuzione.

In termini assoluti i valori sono piccoli. Un consumo di 5.5 joule corrisponde a circa 0.0015 wattora: con l'energia di un wattora, quindi, il GroundOne 0.5B potrebbe verificare circa seicento risposte, interamente sul portatile.

7.11 Località: esecuzione offline, quantizzazione e co-residenza

Tre misure completano il quadro del costo sul dispositivo (Sezione 6.5). La prima riguarda l'indipendenza dalla rete. La valutazione completa di T1 con il modello 0.5B, ripetuta all'interno di una sandbox che nega al processo qualsiasi accesso alla rete, produce predizioni identiche bit per bit a quelle dell'esecuzione originale su tutte le 17,284 frasi. Caricamento del modello, inferenza e valutazione non richiedono quindi alcuna connessione. La prova è più forte di una macchina disconnessa, perché il processo non può raggiungere la rete nemmeno quando la macchina è collegata.

La seconda riguarda la quantizzazione. Quando lo stesso adapter del 0.5B viene applicato ai pesi originali in bf16 invece che a quelli a 8 bit, le probabilità cambiano in media di 0.011 su T1 e di 0.007 su T3, e la classe predetta coincide nel 98.3 % e nel 98.9 % delle frasi. Le metriche restano invariate: la balanced accuracy a tre classi è 0.769 con la base a 8 bit e 0.768 con quella in bf16 su T1, 0.442 in entrambi i casi su T3, e l'AUROC su T1 è 0.905 contro 0.904. La base a 8 bit, che dimezza la memoria dei pesi, non costa quindi accuratezza misurabile. La misura riguarda

però la sensibilità alla precisione della base di un adapter addestrato sulla base a 8 bit, non un modello addestrato interamente in bf16.

La terza riguarda la co-residenza con il generatore. Nello stesso processo, il generatore Qwen2.5-3B-Instruct a 4 bit e GroundOne verificano 30 risposte generate da documenti di T1, con una mediana di 169 token generati e di sette frasi valutate per risposta (Tabella 7.2). Con il verificatore 3B il picco di memoria allocata da MLX è di 6.1 GB, con il 0.5B di 3.3 GB: entrambi ben al di sotto dei 16 GB della macchina. La verifica occupa il 30 % del tempo complessivo per risposta con il 3B e il 6.7 % con il 0.5B. Il modello piccolo aggiunge quindi una frazione modesta al tempo della generazione, mentre il 3B lo allunga di circa un terzo in cambio di una maggiore accuratezza. Durante l'esecuzione con il 3B, con le normali applicazioni del desktop aperte, la memoria libera dell'intero sistema è scesa per qualche istante a circa 1.6 GB: la coppia sta comodamente nel processo, ma su un portatile da 16 GB lascia poco margine a un uso intenso del desktop. Il retriever non è incluso in questa misura.

Tabella 7.2: Co-residenza di generatore e verificatore in un unico processo: Qwen2.5-3B-Instruct a 4 bit come generatore, 30 documenti di T1, generazione greedy fino a 300 token. Memoria allocata da MLX; tempi mediani per risposta.

Verificatore	Memoria dopo il caricamento	Picco di memoria	Generazione	Verifica	Quota della verifica
GroundOne 3B	5.0 GB	6.1 GB	4.75 s	1.68 s	30 %
GroundOne 0.5B	2.3 GB	3.3 GB	4.65 s	0.28 s	6.7 %

7.12 Analisi degli errori

Il risultato debole su VitaminC e le contraddizioni mancate su T1 hanno una causa comune, che la sovrapposizione lessicale mette in evidenza. Su VitaminC, quando un'affermazione supportata condivide poche parole con la propria evidenza (sovrapposizione inferiore a 0.4), il 0.5B la riconosce soltanto nel 2 % dei casi. Quando la sovrapposizione supera 0.7, invece, la riconosce nel 64 % dei casi. Inoltre, il 0.5B non riconosce mai una contraddizione in un'affermazione priva di cifre. I modelli, quindi, si appoggiano in parte alla sovrapposizione lessicale: un'affermazione parafrasata sembra "non menzionata", mentre una piccola modifica a una frase copiata, spesso un numero, passa inosservata.

La stessa debolezza si ritrova su T1, anche se con un peso minore. Quando una frase condivide con il documento poche o alcune parole, la balanced accuracy a tre classi del 0.5B è 0.77. Per le frasi che riprendono dal documento più del

70 % delle proprie parole, invece, scende a 0.54, con una recall di 0.24 per le frasi contraddette e di 0.43 per quelle non menzionate. Il 3B riduce questo punto cieco senza eliminarlo: sulle stesse frasi la sua recall è 0.36 e 0.56, e la sua balanced accuracy 0.63. Le allucinazioni nascoste in frasi che per il resto ricopiano la fonte restano così il limite principale di entrambi i modelli. VitaminC non è più difficile in assoluto: è composto in gran parte proprio da casi di questo tipo, cioè da supporto parafrasato e da modifiche minime, che in RAGTruth sono meno numerosi.

L'Appendice A, infine, mostra l'output di ogni sistema su una stessa risposta di esempio.

Capitolo 8

Discussione, limiti e minacce alla validità

Questo capitolo interpreta i risultati alla luce delle cinque domande di ricerca. Indica poi ciò che gli esperimenti non possono dimostrare e quali scelte potrebbero averli distorti.

8.1 Quando conviene un Decision Model specialista

Specializzazione (RQ1). A rendere GroundOne accurato è l'addestramento sulla decisione. I modelli non addestrati restano vicini al caso, mentre gli stessi modelli, una volta addestrati, raggiungono sul test set di RAGTruth una balanced accuracy a tre classi di 0.769 (0.5B) e 0.832 (3B). Superano così gli encoder specializzati in AUROC, e Rizzo-Flow, un decision model generalista otto volte più grande, in balanced accuracy a tre classi. Il vantaggio, però, si concentra dove si trovano i dati di addestramento. Su LLM-AggreFact il 3B si porta alla pari con il miglior encoder di piccole dimensioni, senza superarlo, e resta sotto il decision model chiuso Jev e un modello open da 27B, misurati in modo indipendente; il 0.5B rimane al di sotto dell'encoder. Su VitaminC, infine, il 3B addestrato non fa meglio di quello non addestrato.

Il confronto con d1-3B, un decision model generalista della stessa dimensione, rende esplicito il compromesso. d1-3B è già addestrato a decidere, ma su un'ampia miscela di task. Su RAGTruth è molto lontano da GroundOne, soprattutto sulle risposte lunghe; su LLM-AggreFact è alla pari con il 3B; su VitaminC è nettamente superiore. Uno specialista è dunque una scelta solida per il dominio su cui è stato addestrato, ma uno strumento debole per la verifica dei fatti in generale. All'interno

del dominio la specializzazione garantisce accuratezza e robustezza sulle risposte lunghe, a scapito del trasferimento; fuori dal dominio, invece, sono la dimensione del modello oppure un'ampia miscela di addestramento a migliorare il trasferimento.

Readout e generazione a confronto (RQ2). Il meccanismo del decision model contribuisce in due modi distinti. Il primo riguarda il costo: un solo forward pass sulla risposta sostituisce la generazione di K verdetti, per cui il tempo risparmiato cresce con la lunghezza della risposta. Quando la risposta contiene quattordici o più frasi, il risparmio arriva a un fattore di circa due rispetto a un giudice sintetico e di circa sei rispetto a uno verboso. Il secondo contributo, che questa tesi non aveva previsto, riguarda l'accuratezza. Con gli stessi pesi e lo stesso costo, scegliere tra le tre lettere ammissibili è più accurato che prendere il token che il modello genererebbe, con un guadagno di balanced accuracy a tre classi di 0.25 per il 0.5B e di 0.07 per il 3B. Un decision model sottoposto a fine-tuning si comprende quindi meglio come un classificatore a tre classi innestato su un modello linguistico che come un generatore che si ferma dopo un token. La distinzione pesa soprattutto per i modelli piccoli, cioè proprio per quelli che un portatile può permettersi. Il confronto con i giudici generativi, infine, misura anche l'effetto dell'addestramento: poiché i giudici non sono addestrati, le ampie differenze di accuratezza rispetto a GroundOne vanno lette come l'effetto congiunto della specializzazione e del readout. La Figura 8.1 scompone il guadagno su T1 nei due passaggi, l'addestramento e il readout ristretto.

Calibrazione (RQ3). Una temperatura e un bias stimati sul dev set allineano le probabilità alle frequenze osservate sul test set dello stesso dominio, e le probabilità calibrate rendono efficace la selective prediction. Due indicazioni valgono al di là di GroundOne. La prima è che l'Expected Calibration Error, da solo, non basta a ordinare i sistemi. Dopo la stessa calibrazione, giudici le cui probabilità separano a malapena le classi raggiungono lo stesso ECE di GroundOne, e soltanto l'AUROC e la risk–coverage curve ne rivelano la differenza. La seconda è che decisioni e probabilità vanno ricavate in modo coerente. Se si sceglie la classe calibrata più probabile, la decisione collassa verso la classe maggioritaria; una soglia stimata sul dev set, invece, recupera la decisione bilanciata e mantiene l'etichetta coerente con la probabilità mostrata all'utente. Come mostrano i risultati su VitaminC, infine, la calibrazione vale soltanto per la distribuzione su cui è stata stimata.

Costo sul dispositivo (RQ4). Il verificatore sta comodamente nel portatile da 16 GB. Durante l'addestramento la memoria occupata raggiunge un picco di 2.6 GB con il 0.5B e di 5.9 GB con il 3B. Verificare una risposta lunga richiede 0.41 secondi e 5.5 joule con il 0.5B, 2.57 secondi e 33 joule con il 3B. Il 0.5B rientra in un budget interattivo; il 3B, invece, su una risposta tipica è più lento di quanto

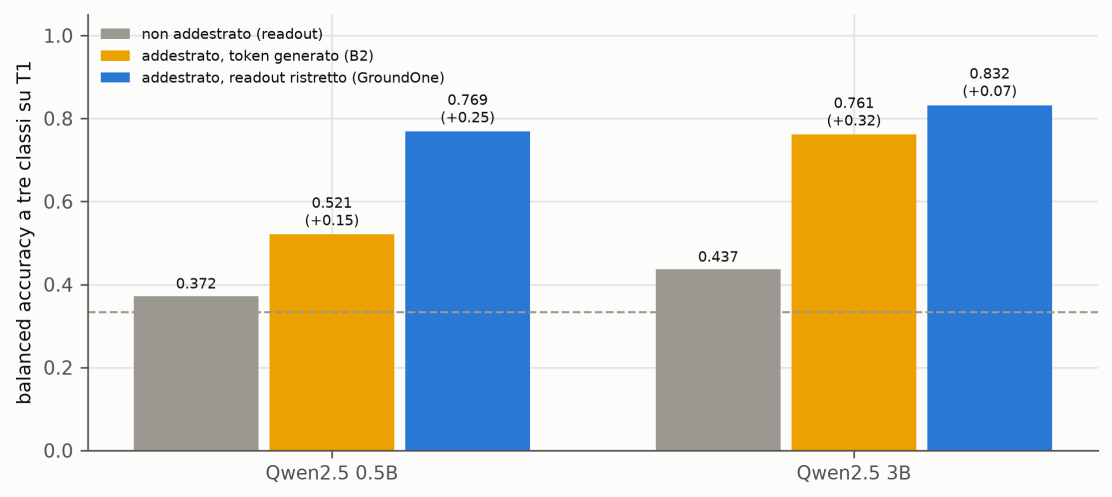


Figura 8.1: Da dove viene l'accuratezza di GroundOne su T1: lo stesso modello base non addestrato (readout, design a), addestrato ma con la decisione presa dal token che genererebbe (controllo B2, errori di formato contati come errori) e addestrato con il readout ristretto. Tra parentesi il guadagno rispetto al passaggio precedente. Il 3B non addestrato è valutato su 600 risposte; il tratteggio indica il livello casuale.

un uso interattivo richiederebbe, e scambia tempo con accuratezza. Le misure di energia precisano la portata dei risultati sulla velocità. Un singolo prefill lungo assorbe più potenza del decoding token per token, per cui l'energia risparmiata è inferiore al tempo risparmiato, e la maggiore velocità non va letta come un risparmio equivalente di batteria. Il verificatore convive inoltre con un generatore nello stesso processo: insieme a un generatore 3B a 4 bit, il picco di memoria è di 6.1 GB con il 3B e di 3.3 GB con il 0.5B. L'intera valutazione gira senza alcun accesso alla rete, con risultati identici bit per bit, e i pesi a 8 bit non costano accuratezza rispetto a quelli in bf16 (Sezione 7.11). Resta da misurare il retriever, che verrà valutato nell'applicazione dimostrativa.

Anche il costo economico va letto con misura (Tabella 8.1). Verificare 10,000 risposte con GroundOne richiede un'energia che, al prezzo medio europeo dell'elettricità per le famiglie [118], vale meno di un centesimo di euro con il 0.5B e circa tre centesimi con il 3B. Affidare la stessa verifica a un giudice in cloud, ai prezzi di listino dei fornitori [119, 120, 121], costa invece da circa un dollaro con i modelli più economici a oltre duecento con quelli di fascia più alta. Il rapporto è quindi di due-quattro ordini di grandezza, ma in valore assoluto i giudici economici restano poco costosi, e il confronto non considera né l'acquisto dell'hardware né la qualità dei giudici in cloud, che questa tesi non ha misurato. L'argomento decisivo

a favore della verifica locale resta quindi l'indipendenza: i documenti non lasciano la macchina e il servizio non dipende da un fornitore esterno.

Tabella 8.1: Costo indicativo della verifica di 10,000 risposte RAG. Giudici in cloud: prezzi di listino del livello standard, consultati il 19 ottobre 2026, con circa 900 token in ingresso per risposta (documento e risposta mediani di T1 più le istruzioni) e i token in uscita misurati per i giudici 3B di questa tesi (36 per il giudice sintetico, 248 per quello verboso). GroundOne: energia netta misurata del chip moltiplicata per il prezzo medio dell'elettricità per le famiglie nell'UE (0.29 €/kWh, secondo semestre 2025). Sono esclusi il costo dell'hardware e gli sconti per caching o elaborazione in batch.

Sistema	Prezzo ingresso / uscita (USD per Mtoken)	Giudice sintetico	Giudice verboso
Claude Haiku 5.5, gpt-6-luna	0.10 / 0.50	1.1 USD	2.2 USD
Gemini 3.5 Flash-Lite	0.30 / 2.50	3.6 USD	8.9 USD
Gemini 3.8 Flash	0.75 / 3.75	8.2 USD	16 USD
Claude Sonnet 5.5, gpt-6.1-sol, Gemini 4 Argon	2.00 / 10.00	22 USD	43 USD
Claude Opus 5.5	4.00 / 20.00	43 USD	86 USD
gpt-6-astra	10.00 / 50.00	109 USD	215 USD
GroundOne 0.5B (locale)	0.015 kWh, circa 0.004 €		
GroundOne 3B (locale)	0.092 kWh, circa 0.03 €		

Oltre un punteggio binario (RQ5). L'output a tre classi distingue bene le frasi contraddette da quelle non menzionate nel data-to-text, il task in cui RAGTruth concentra le proprie contraddizioni. Le distingue molto meno nel question answering e nella summarization, dove nei dati di addestramento le contraddizioni sono rare. Il 3B migliora sensibilmente su questo fronte, e ciò suggerisce che al modello piccolo manchino, più che un'architettura diversa, contraddizioni più varie in addestramento. Le policy strict e lenient e la soglia derivano tutte dalle stesse tre probabilità. L'utente può quindi cambiare la nozione di errore senza riaddestrare il modello, cosa che nessun verificatore binario consente. La Figura 8.2 riassume queste indicazioni in forma di albero decisionale.

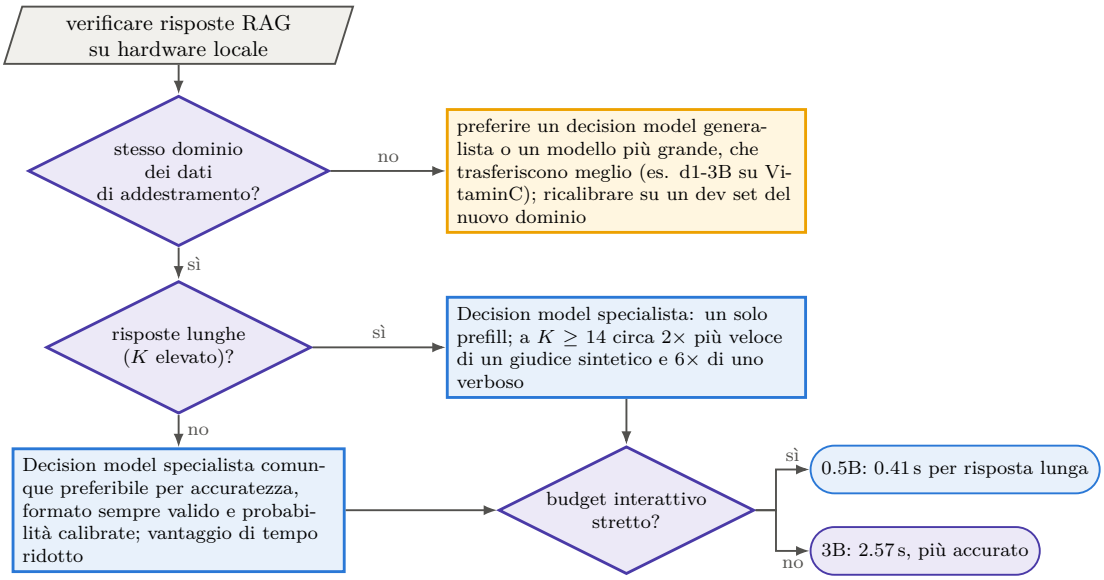


Figura 8.2: Quando conviene un decision model specialista, secondo i risultati di questa tesi. È una sintesi qualitativa: i tempi si riferiscono al portatile M4 e alle risposte con 14 o più frasi, e il trasferimento fuori dominio è quello osservato su LLM-AggreFact e VitaminC.

8.2 Limiti

Una sola piattaforma e una sola famiglia di modelli. Tutti i risultati provengono da un unico portatile, un unico runtime e un'unica famiglia di modelli. Latenza ed energia dipendono dall'hardware e dall'implementazione: ciò che dovrebbe valere anche altrove sono i rapporti tra i metodi, non i valori assoluti. La latenza di GroundOne non viene confrontata con quella degli encoder né con quella dei decision model generalisti, perché questi sono stati eseguiti in runtime diversi e in condizioni che renderebbero il confronto iniquo.

Stabilità dell'addestramento. Due run completi del 0.5B concordano su tutte le metriche aggregate, ma un terzo run è collassato nelle prime fasi ed è stato interrotto. La ricetta di addestramento non prevede gradient clipping e usa AdamW senza bias correction, due cause note di fine-tuning instabile [98, 97]. L'una o l'altra potrebbe spiegare il collasso, ma nessuna delle due ipotesi è stata verificata. Il 3B, inoltre, è stato addestrato una sola volta, su metà delle risposte di addestramento, e la sua variabilità al cambiare del seed resta ignota.

Dati di addestramento. RAGTruth è l'unica fonte di addestramento. Le sue contraddizioni sono concentrate nel data-to-text e, al di fuori di questo task, vengono apprese solo in parte. I modelli, inoltre, si affidano in parte alla sovrapposizione lessicale. Quando il supporto è espresso con parole diverse, la frase appare non supportata; quando una frase copia la fonte salvo una piccola modifica, la modifica passa spesso inosservata. Questo punto cieco si riduce al crescere del modello, ma resta il limite principale di entrambi.

Ambito della decisione. La verifica avviene frase per frase e soltanto rispetto al documento recuperato. Una frase vera nel mondo ma assente dal documento conta quindi come non menzionata: è il comportamento corretto per la groundedness, ma non per la verifica dei fatti. Le frasi prive di contenuto verificabile vengono escluse da una regola, e la segmentazione stessa può dividere o accorpare le affermazioni in modo diverso da un lettore umano. Infine, il verdetto su una frase dipende soltanto dal documento e dalle frasi che la precedono.

Portata statistica. Gli intervalli bootstrap descrivono la variabilità del test set, non quella dell'addestramento. Inoltre, circa cinquanta confronti sono stati sottoposti a test senza correzione per confronti multipli. Per questo le conclusioni poggiano su effetti ben lontani dallo zero, e non si rivendicano differenze su LLM-AggreFact e VitaminC inferiori alla dispersione tra seed. Diverse baseline, infine, sono state valutate su sottocampioni, il che ne allarga gli intervalli.

8.3 Minacce alla validità

Etichette. La distinzione tra frase contraddetta e non menzionata si basa sugli annotatori di RAGTruth, il cui accordo sul tipo di allucinazione non è pubblicato. Inoltre, una recente riannotazione della partizione di test di RAGTruth ha individuato un numero di risposte allucinate considerevolmente superiore a quello indicato dalle etichette originali [114]. Il test set potrebbe quindi sottostimare le classi minoritarie. Per questo i risultati a tre classi su RAGTruth vanno sempre letti insieme a quelli su VitaminC, le cui etichette hanno un accordo tra annotatori misurato. LLM-AggreFact, infine, unifica le etichette di undici dataset di qualità diversa, e alcune delle sue colonne sono notoriamente rumorose.

Vantaggio nel dominio. GroundOne è addestrato su RAGTruth. I suoi risultati sul test set di RAGTruth e sulla colonna RAGTruth di LLM-AggreFact lo avvantaggiano quindi rispetto a sistemi addestrati altrove. Per contro, VitaminC fa parte dei dati da cui discende l'addestramento di MiniCheck-RoBERTa. Ogni confronto

è riportato con la relativa avvertenza, e le conclusioni poggiano sull'andamento complessivo nei tre test set anziché su uno solo.

Configurazione delle baseline. I giudici generativi sono sensibili al prompt. I loro prompt sono stati fissati prima di esaminare i risultati di test, e quello del giudice a parità di output è stato scelto sul dev set unicamente in base alla validità del formato. Il parsing dell'output è stato permissivo, a vantaggio dei giudici. Il decoding non era vincolato da una grammatica, e questo incide soprattutto sugli errori di formato del modello più piccolo. Non sono stati valutati né un giudice sottoposto a fine-tuning sugli stessi dati né un giudice più grande: il confronto misura dunque ciò che offre, senza addestramento, un giudice della stessa dimensione. Rizzo-Flow è stato eseguito con pesi a 4 bit, perché il modello a 8 bit non trovava posto in memoria accanto al sistema operativo, e questo potrebbe sottostimarne l'accuratezza. d1-3B, infine, è stato eseguito zero-shot e, su LLM-AggreFact, soltanto su un sottocampione. Ha ricevuto le domande usate per tutte le baseline di tipo decision model, che potrebbero non essere la formulazione a esso più congeniale.

Contaminazione. I modelli base sono stati preaddestrati su ampi corpora web che potrebbero includere parti dei test set pubblici[1]. Gli stessi modelli base, però, usati senza addestramento come baseline, raggiungono un'accuratezza vicina al caso sul task a tre classi. La contaminazione non può quindi spiegare i guadagni dovuti all'addestramento; per i valori assoluti, invece, non può essere esclusa del tutto.

Una sola macchina. Tutte le misure di tempo e di energia provengono da un unico portatile, un Apple M4 con 16 GB, e da un unico runtime, MLX. Il modello di costo del Capitolo 2 prevede che il vantaggio di un decision model valga su ogni classe di hardware locale, perché poggia sulla differenza tra prefill e decoding, che produttori e studi indipendenti osservano ovunque. Anche i limiti inferiori al costo del decoding calcolati in quel capitolo per altre macchine discendono dalle specifiche pubblicate. Gli speed-up effettivi su una scheda grafica dedicata, su un DGX Spark o su uno smartphone, invece, non sono stati misurati. La loro entità dipende dall'equilibrio tra calcolo e banda di ciascuna macchina: il modello di costo sostiene la direzione del vantaggio, non la sua entità.

[1]Si parla di *contaminazione* dei dati quando esempi di un test set compaiono, anche solo in parte, nei dati di preaddestramento di un modello. Un modello contaminato può ricordare gli esempi anziché valutarli, e i suoi punteggi su quel test set risultano allora troppo ottimistici.

Capitolo 9

Conclusioni e sviluppi futuri

9.1 Sintesi dei risultati

Questa tesi si è chiesta se un modello linguistico compatto, specializzato in un'unica decisione e impiegato come decision model, possa verificare frase per frase, su un portatile, le risposte di un sistema RAG, e da dove provenga l'eventuale vantaggio. GroundOne, un modello Qwen2.5 sottoposto a fine-tuning che legge i verdetti di un'intera risposta in un solo forward pass, fornisce una risposta positiva, sebbene con alcune riserve.

La specializzazione funziona là dove era mirata. Sul test set di RAGTruth GroundOne raggiunge una balanced accuracy a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B. Supera di molto gli stessi modelli non addestrati, ordina le frasi supportate meglio degli encoder specializzati ed è nettamente superiore a Rizzo-Flow, un decision model generalista otto volte più grande. Su LLM-AggreFact il 3B è alla pari con il miglior encoder di piccole dimensioni. Su VitaminC, invece, il guadagno scompare: uno specialista non è uno strumento generale di verifica dei fatti. d1-3B, un decision model generalista della stessa dimensione addestrato a decidere su un'ampia miscela di task, conferma il compromesso: GroundOne lo supera nettamente su RAGTruth, lo eguaglia su LLM-AggreFact e resta molto indietro su VitaminC.

Il meccanismo del decision model ripaga su due fronti. Il primo è la velocità. Leggere tutti i verdetti in un solo passaggio rende la verifica delle risposte lunghe circa due volte più rapida di quella di un giudice generativo sintetico e sei volte più rapida di quella di un giudice verboso della stessa dimensione, con un output sempre valido. Il secondo è l'accuratezza: restringere la decisione alle risposte ammissibili è di per sé più accurato che lasciare allo stesso modello la generazione del verdetto. Inoltre, anche quando si chiede loro lo stesso output, i giudici generativi non riescono a fornire probabilità di qualità paragonabile. Le loro probabilità separano

a malapena le decisioni corrette da quelle errate, e quelle espresse a parole sono quasi sempre del tipo tutto-o-niente.

Le probabilità di GroundOne si possono calibrare con una temperatura e un bias stimati sul dev set. Una soglia scelta sugli stessi dati le converte poi in etichette coerenti sia con le probabilità sia con la decisione valutata. L'Expected Calibration Error, da solo, non distingue però le probabilità utili da quelle inutili: la capacità discriminativa va misurata a parte.

L'intero sistema gira su un portatile da 16 GB. Una risposta lunga viene verificata in 0.41 secondi e 5.5 joule con il 0.5B, e in 2.57 secondi e 33 joule con il 3B. Rispetto a un giudice generativo il risparmio di energia è reale, ma inferiore al risparmio di tempo, perché un singolo prefill lungo assorbe più potenza del decoding.

I limiti principali sono due. Il primo è la ristrettezza dei dati di addestramento, a causa della quale le contraddizioni al di fuori del data-to-text e le allucinazioni nascoste in frasi copiate vengono apprese solo in parte. Il secondo è una ricetta di addestramento non del tutto robusta rispetto alla scelta del seed.

9.2 Sviluppi futuri

Da questi limiti discendono direttamente diverse linee di lavoro. La ricetta di addestramento andrebbe resa robusta abilitando la bias correction e il gradient clipping, registrando le norme dei gradienti e ripetendo i run con più seed, anche per il 3B. Servirebbero poi dati di addestramento con contraddizioni più varie, con supporto parafrasato e con modifiche minime come quelle di VitaminC: dovrebbero intervenire sia sulla bassa recall al di fuori del data-to-text sia sul punto cieco delle frasi copiate. Infine, addestrare il readout sulla lettera che il modello scriverebbe spontaneamente dopo il marker potrebbe allineare readout e generazione, con effetti sull'accuratezza ancora da misurare.

Sul versante dei sistemi, il modello di costo del Capitolo 2 andrebbe verificato su altro hardware. Un porting del readout su llama.cpp permetterebbe di misurare GroundOne su una scheda grafica dedicata, su un DGX Spark o su una macchina AMD Strix Halo, e su uno smartphone, le cui unità di elaborazione neurale accelerano proprio il prefill su cui si regge un decision model. Per il pilastro della località resta inoltre da misurare la memoria del retriever accanto a generatore e verificatore, già misurati insieme in questa tesi. Anche i confronti di latenza con le altre famiglie di verificatori richiedono condizioni eque: gli encoder vanno eseguiti in batch, mentre GroundOne e i decision model generalisti vanno interrogati attraverso la stessa interfaccia HTTP.

La verifica potrebbe inoltre procedere durante la generazione della risposta, segnalando ogni frase non appena è completa: la soluzione basata sui marker rende naturale questo uso. Infine, un assistente locale costruito sull'interfaccia del servizio

permetterebbe agli utenti di scegliere la regola e la soglia più adatte ai propri documenti. È proprio in questo uso che le probabilità calibrate mostrano il loro valore.

Appendice A

Output di ogni sistema su una stessa risposta

Questa appendice mostra che cosa restituisce ciascun sistema per una stessa risposta di T1, in modo da collegare i numeri del Capitolo 7 a output concreti. La risposta (identificativo `rt-7784`) è di tipo data-to-text: un modello linguistico l'ha scritta a partire dalla descrizione strutturata di un ristorante, ossia un record JSON con nome, indirizzo, categorie, orari di apertura (da martedì a domenica, 11:00–21:00), attributi, una valutazione media di 4.5 stelle e due recensioni di clienti. È stata scelta perché tutti i sistemi l'hanno valutata e perché contiene tutte e tre le classi, non in base alle prestazioni di un qualche sistema. Le sei frasi e le relative etichette di riferimento sono:

1. *Mesa Verde is a popular business located in Santa Barbara, CA.* — supportata.
2. *It offers a variety of services including Garage Door Services, Restaurants, Live/Raw Food, . . . , Vegan, and Salad.* — supportata.
3. *The business is open seven days a week from 11 am to 9 pm.* — contraddetta (il lunedì il locale è chiuso).
4. *Mesa Verde has a rating of 4.5 stars on Google and has received multiple positive reviews.* — non menzionata (il record non indica la fonte della valutazione e contiene due recensioni, una delle quali negativa).
5. *One customer praised the restaurant's fresh, delicious, and inventive cuisine, while another was disappointed by the taste and price of the food.* — supportata.
6. *Despite this, many customers have had positive experiences at Mesa Verde and recommend it for its fusion and vegan food options.* — supportata.

A.1 Verdetti frase per frase

La Tabella A.1 riporta il verdetto di ciascun sistema. Per GroundOne l'etichetta è quella restituita dal servizio, ricavata dalle probabilità calibrate con la soglia stimata sul dev set ($\tau = 0.935$ per il 0.5B e 0.925 per il 3B), e il numero è la P(supported) calibrata. Per gli encoder il numero è la probabilità di supporto, con la decisione presa a 0.5. S, C e N stanno per supportata, contraddetta e non menzionata; un trattino indica un errore di formato.

Tabella A.1: Verdetti sulla risposta `rt-7784`. Gli errori rispetto al riferimento sono in grassetto.

Sistema	1	2	3	4	5	6
Riferimento	S	S	C	N	S	S
GroundOne 0.5B (restituita, P_{sup})	S 0.96	S 0.98	C 0.50	N 0.05	S 0.98	S 0.97
GroundOne 3B (restituita, P_{sup})	S 0.96	S 0.99	C 0.58	N 0.21	S 0.99	**N 0.85**
B2 0.5B (token generato)	S	S	C	N	S	–
B2 3B (token generato)	S	S	C	–	S	S
0.5B non addestrato (readout)	**C**	**C**	C	**C**	**C**	**C**
Giudice sintetico 0.5B	S	S	**S**	**S**	S	S
Giudice sintetico 3B	**C**	S	**S**	**S**	**N**	S
Giudice verboso 0.5B	–	–	–	–	–	–
Giudice verboso 3B	–	–	–	–	–	–
Giudice a parità di output 3B, logit	**C**	S	**S**	**S**	**N**	S
Giudice a parità di output 3B, probabilità espresse a parole	S	**C**	**N**	**S**	**C**	**N**
MiniCheck-RoBERTa (P_{sup})	0.78	0.90	0.09	0.03	**0.43**	**0.03**
HHEM-2.1-open (P_{sup})	0.75	0.95	0.01	0.09	0.73	**0.24**
Rizzo-Flow 4B	S	S	**S**	**S**	S	S

GroundOne 0.5B classifica correttamente tutte e sei le frasi. La contraddizione nella frase 3 viene colta con una P(supported) calibrata di 0.50: ben sotto la soglia, ma lontana dalla certezza, ed è esattamente ciò che una probabilità calibrata dovrebbe esprimere per una frase corretta in tutto tranne che in un giorno della settimana. Anche GroundOne 3B coglie entrambi gli errori, ma restituisce come non menzionata la frase 6, una generalizzazione tratta dalle due recensioni: la sua P(supported) di 0.85 è inferiore alla soglia elevata imposta dal tasso di base di RAGTruth, un caso in cui un utente che preferisce meno segnalazioni abbasserebbe la soglia. A parità di pesi, il token generato (B2) dà la lettera corretta su cinque frasi su sei, ma sulla restante produce uno spazio (0.5B) o la parola "We" (3B) al posto di un verdetto. Gli encoder binari separano bene i due errori, ma respingono anche le frasi 5 o 6, che riassumono le recensioni con parole diverse; Rizzo-Flow accetta tutte le frasi. I giudici generativi falliscono nei modi quantificati nel Capitolo 7, come mostrano gli output grezzi che seguono.

A.2 Output grezzi

GroundOne 0.5B (JSON restituito, abbreviato). Un unico forward pass su 1,193 token di input, nessun token generato, 0.47 secondi (2.6 secondi per il 3B):


```
{"design": "b", "threshold": 0.935, "n_decisions": 6,
 "usage": {"input_tokens": 1193, "output_tokens": 0},
 "sentences": [
  {"index": 2, "text": "The business is open seven days a week ...",
   "scored": true, "labels": {"three": "contradicted"},
   "answers": {"three": {"supported": 0.502, "contradicted": 0.462,
                         "not_mentioned": 0.036}},
   "flag_strict": true, "flag_lenient": true},
  {"index": 3, "text": "Mesa Verde has a rating of 4.5 stars ...",
   "scored": true, "labels": {"three": "not_mentioned"},
   "answers": {"three": {"supported": 0.051, "contradicted": 0.003,
                         "not_mentioned": 0.946}},
   "flag_strict": true, "flag_lenient": false}, ...]}
```


Giudice sintetico 3B. Una riga per frase, come richiesto, ma con la lettera sbagliata in quattro frasi:

```
1: B
2: A
3: A
4: A
5: C
6: A
```

Giudice verboso 3B (primi tre verdetti). Il ragionamento riformula la frase e cita il documento a sostegno anche della frase contraddetta e di quella non supportata, e tutti i verdetti sono "A". I verdetti compaiono su una riga a sé anziché sulla stessa riga richiesta dall'istruzione, per cui il protocollo li conta come errori di formato:

```
1: Mesa Verde is located in Santa Barbara, CA, as stated in the document.
Verdict: A

2: Mesa Verde offers a variety of services including Garage Door Services,
   [...], as mentioned in the categories provided.
Verdict: A

3: The business is open seven days a week from 11 am to 9 pm, as indicated
   by the hours provided.
Verdict: A
```

Giudice verboso 0.5B. Ricopia le sei frasi, numerate, senza alcun verdetto.

Giudice a parità di output 3B, probabilità espresse a parole. Le probabilità sono tutte one-hot e, per le frasi 3 e 6, l'etichetta dichiarata (C) contraddice le probabilità dichiarate, che assegnano tutta la massa ad A:


```
{"1": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "2": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "3": {"label": "C", "p": [1.0, 0.0, 0.0]},
 "4": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "5": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "6": {"label": "C", "p": [1.0, 0.0, 0.0]}}
```


Giudice a parità di output 0.5B, probabilità espresse a parole. Lo stesso vettore per ogni frase:


```
{"1": {"label": "A", "p": [0.3, 0.3, 0.3]},
 "2": {"label": "A", "p": [0.3, 0.3, 0.3]},
 ...
 "6": {"label": "A", "p": [0.3, 0.3, 0.3]}}
```

Bibliografia

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020 (cit. alle pp. 1, 35).

[2] Georgi Gerganov e ggml-org contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: `https://github.com/ggml-org/llama.cpp` (cit. alle pp. 1, 25, 29, 34, 65).

[3] Apple. *MLX: An array framework for Apple silicon*. 2023. URL: `https://github.com/ml-explore/mlx` (cit. alle pp. 1, 25, 48).

[4] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang et al. *Apple Intelligence Foundation Language Models*. 2024. arXiv: `2407.21075` (cit. alle pp. 1, 13, 16).

[5] Venkat Somala e Luke Emberson. *Frontier AI performance becomes accessible on consumer hardware within a year*. Epoch AI. 15 Ago. 2025. URL: `https://epoch.ai/data-insights/consumer-gpu-model-gap` (visitato il giorno 2026-10-11) (cit. alle pp. 1, 15).

[6] Stanford Institute for Human-Centered Artificial Intelligence. *The 2025 AI Index Report: Technical Performance*. 2025. URL: `https://hai.stanford.edu/ai-index/2025-ai-index-report/technical-performance` (visitato il giorno 2026-10-11) (cit. alle pp. 1, 15, 16).

[7] NYU Shanghai RITS. *OpenAI claims a Navier–Stokes proof, amid a dispute over credit*. 9 Set. 2026. URL: `https://rits.shanghai.nyu.edu/ai/openai-navier-stokes-proof-credit-dispute/` (visitato il giorno 2026-10-11) (cit. alle pp. 2, 11, 12).

[8] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu et al. "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. alle pp. 2, 4, 35, 51, 52).

[9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong et al. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ACM Transactions on Information Systems* (2025). DOI: 10.1145/3703155 (cit. alle pp. 2, 35).

[10] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin et al. “Efficiently Scaling Transformer Inference”. In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023 (cit. alle pp. 3, 17, 18, 43).

[11] Yuheng Zha, Yichi Yang, Ruichen Li e Zhiting Hu. “AlignScore: Evaluating Factual Consistency with a Unified Alignment Function”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. alle pp. 3, 36, 78).

[12] Liyan Tang, Philippe Laban e Greg Durrett. “MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024 (cit. alle pp. 3, 36, 51–53, 66).

[13] Vectara. *HHEM-2.1-Open: Hughes Hallucination Evaluation Model*. 2024. URL: https://huggingface.co/vectara/hallucination_evaluation_model (cit. alle pp. 3, 36, 66).

[14] TypeSafe. *Introducing System One Models and Jev*. Blog post. 2026. URL: https://typesafe.ai/blog/introducing-system-one-models-and-jev (cit. alle pp. 3, 32).

[15] TheoLeeCJ. *SemIf: open option-logit System One baseline*. 2026. URL: https://github.com/TheoLeeCJ/SemIf (cit. alle pp. 3, 4, 33).

[16] Simone Rizzo. *Rizzo Flow: typed decisions from an LLM, without generating a single token*. 2026. URL: https://github.com/Rizzo-AI-Academy/rizzo-flow (cit. alle pp. 3, 4, 33, 65).

[17] Hugging Face. *Models tagged decision-model*. URL: https://huggingface.co/models?other=decision-model (visitato il giorno 2026-10-10) (cit. alle pp. 4, 33).

[18] Liquid AI. *d1-3B: a decision model built on LFM2.5-VL-3B (model card, LiquidAI/d1-3B, revision 051bcc4)*. URL: https://huggingface.co/LiquidAI/d1-3B (visitato il giorno 2026-10-10) (cit. alle pp. 4, 34, 65).

[19] Cloudflare. *Clef: open decision models (model card, Cloudflare/clef)*. URL: https://huggingface.co/Cloudflare/clef (visitato il giorno 2026-10-10) (cit. alle pp. 4, 34).

[20] Tobias Deußer, Lorenz Sparrenberg e Rafet Sifa. *Evaluating and Benchmarking the System One Model Jev*. 2026. arXiv: 2609.37647 (cit. alle pp. 4, 33, 77).

[21] Gowthamkumar Nandakishore. *Laya as a Typed Probabilistic Assessor: An Independent Reproduction and a Preregistered Study of Calibration and Selective Escalation*. 2026. arXiv: 2609.33843 (cit. alle pp. 4, 33).

[22] multimodalart. *Jev Decision Index: a leaderboard of System One decision models (Hugging Face Space, v0.2.1)*. URL: https://huggingface.co/spaces/multimodalart/jev-decision-index (visitato il giorno 2026-10-10) (cit. alle pp. 4, 34, 74).

[23] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115 (cit. alle pp. 4, 14, 38, 45, 62).

[24] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang e Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022 (cit. alle pp. 4, 38, 45).

[25] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen e Kaibin Huang. "Mobile Edge Intelligence for Large Language Models: A Contemporary Survey". In: *IEEE Communications Surveys & Tutorials* 27.6 (2025), pp. 3820–3860. arXiv: 2407.18921 (cit. alle pp. 11, 13).

[26] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88. 2016. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (cit. a p. 11).

[27] Jeremy Kahn. *OpenAI says it cracked a Navier–Stokes math grand challenge; a mathematician raises accusations (Eye on AI)*. Fortune. 8 Set. 2026. URL: https://fortune.com/2026/09/08/openai-says-it-cracked-navier-stokes-math-grand-challenge-buckmaster-accusation-cheating-intimidation-tao-lament/ (visitato il giorno 2026-10-11) (cit. a p. 11).

[28] Larissa G. Capella. *'Why would you ruin your career?': OpenAI claims to have cracked one of math's greatest unsolved problems, but mathematicians allege it played dirty*. Live Science. 9 Set. 2026. URL: https://livescience.com/physics-mathematics/mathematics/why-would-you-ruin-your-career-openai-claims-to-have-cracked-one-of-maths-greatest-unsolved-problems-but-mathematicians-allege-it-played-dirty (visitato il giorno 2026-10-11) (cit. a p. 12).

[29] Apple. *Private Cloud Compute: A new frontier for AI privacy in the cloud*. 10 Giu. 2024. URL: https://security.apple.com/blog/private-cloud-compute/ (visitato il giorno 2026-10-11) (cit. alle pp. 12, 13).

[30] Senyao Li, Haozhao Wang, Wenchao Xu, Rui Zhang, Song Guo, Jingling Yuan, Xian Zhong, Tianwei Zhang e Ruixuan Li. *Collaborative Inference and Learning between Edge SLMs and Cloud LLMs: A Survey of Algorithms, Execution, and Open Challenges*. 2025. arXiv: 2507.16731 (cit. alle pp. 13, 24).

[31] Open Source Initiative. *The Open Source AI Definition, version 1.0*. 28 Ott. 2024. URL: https://opensource.org/ai/open-source-ai-definition (visitato il giorno 2026-10-11) (cit. a p. 14).

[32] Artificial Analysis. *Introducing the Artificial Analysis Openness Index*. 1 Dic. 2025. URL: https://artificialanalysis.ai/articles/announcing-artificial-analysis-openness-index (visitato il giorno 2026-10-11) (cit. a p. 14).

[33] Rishi Bommasani, Alice Hau, Kevin Klyman e Percy Liang. *Foundation models under the EU AI Act*. Stanford Center for Research on Foundation Models. 1 Ago. 2024. URL: https://crfm.stanford.edu/2024/08/01/eu-ai-act.html (visitato il giorno 2026-10-11) (cit. a p. 14).

[34] Epoch AI. *Open-weight models lag the closed frontier by about four months on the Epoch Capabilities Index*. 29 Mag. 2026. URL: https://epoch.ai/data-insights/open-closed-eci-gap (visitato il giorno 2026-10-11) (cit. a p. 15).

[35] Venkat Somala e Luke Emberson. *What's the best model you can run on a single consumer GPU?* Epoch AI. 9 Set. 2025. URL: https://epochai.substack.com/p/whats-the-best-model-you-can-run (visitato il giorno 2026-10-11) (cit. a p. 16).

[36] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane e Mengwei Xu. *Small Language Models: Survey, Measurements, and Insights*. 2024. arXiv: 2409.15790 (cit. a p. 16).

[37] Zechun Liu et al. "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases". In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. PMLR. 2024 (cit. alle pp. 16, 30).

[38] Google. *Introducing Gemma 3n: the developer guide*. 20 Mag. 2025. URL: https://developers.googleblog.com/en/introducing-gemma-3n/ (visitato il giorno 2026-10-11) (cit. a p. 16).

[39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 2023 (cit. a p. 17).

[40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah et al. “Splitwise: Efficient Generative LLM Inference Using Phase Splitting”. In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024 (cit. alle pp. 17, 23, 30, 43, 84).

[41] Samuel Williams, Andrew Waterman e David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785 (cit. alle pp. 17, 19).

[42] Zhihang Yuan et al. *LLM Inference Unveiled: Survey and Roofline Model Insights*. 2024. arXiv: 2402.16363 (cit. alle pp. 17, 18).

[43] Apple. *MacBook Pro (14-inch, M4, 2024) – Technical Specifications*. URL: https://support.apple.com/en-us/121552 (visitato il giorno 2026-10-10) (cit. alle pp. 18, 23, 62).

[44] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard et al. “LLM in a flash: Efficient Large Language Model Inference with Limited Memory”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. alle pp. 19, 27).

[45] Apple. *MacBook Pro – Technical Specifications (M5, M5 Pro and M5 Max)*. URL: https://www.apple.com/macbook-pro/specs/ (visitato il giorno 2026-10-11) (cit. a p. 23).

[46] Apple. *Mac mini – Technical Specifications*. URL: https://www.apple.com/mac-mini/specs/ (visitato il giorno 2026-10-11) (cit. alle pp. 22, 23).

[47] Apple. *Mac Studio (2025) – Technical Specifications*. URL: https://support.apple.com/en-us/122211 (visitato il giorno 2026-10-11) (cit. alle pp. 22, 23).

[48] NVIDIA. *DGX Spark User Guide: Hardware Overview*. 10 Set. 2026. URL: https://docs.nvidia.com/dgx/dgx-spark/hardware.html (visitato il giorno 2026-10-11) (cit. a p. 23).

[49] Mark Hachman. *Framework Desktop review*. PCWorld. 7 Ago. 2025. URL: https://www.pcworld.com/article/2866400/framework-desktop-review.html (visitato il giorno 2026-10-11) (cit. alle pp. 22, 23).

[50] NVIDIA. *Compare GeForce Graphics Cards: specifications*. URL: https://www.nvidia.com/en-us/geforce/graphics-cards/compare/ (visitato il giorno 2026-10-11) (cit. alle pp. 23, 24).

[51] EXO Labs. *NVIDIA DGX Spark and Mac Studio: disaggregated prefill and decode*. 2025. URL: https://blog.exolabs.net/nvidia-dgx-spark/ (visitato il giorno 2026-10-11) (cit. a p. 23).

[52] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia e Haibo Chen. *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. 2024. arXiv: 2406.06282 (cit. a p. 24).

[53] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu e Xuanzhe Liu. "Fast On-device LLM Inference with NPUs". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*. ACM, 2025. DOI: 10.1145/3669940.3707239 (cit. alle pp. 24, 25, 30).

[54] Apple Machine Learning Research. *Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU*. 19 Nov. 2025. URL: https://machinelearning.apple.com/research/exploring-llms-mlx-m5 (visitato il giorno 2026-10-11) (cit. a p. 25).

[55] Subhransu Das, Jiaming Cheng, Arnav Kumar, Sadia Afrose, Mingzhe Han, Michael Silagy, Shreya Palande, Brijesh Soni e Rajiv Ramnath. *Large Models for Small Devices: Edge AI Deployment*. 2026. arXiv: 2608.15693 (cit. a p. 25).

[56] Apple. *mlx-lm: LLMs with MLX*. 2024. URL: https://github.com/ml-explore/mlx-lm (cit. alle pp. 25, 29, 48).

[57] Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma e Hritvik Shrivastava. *Production-Grade Local LLM Inference on Apple Silicon: A Comparative Study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS*. 2025. arXiv: 2511.05502 (cit. a p. 25).

[58] Stefanos Laskaridis, Kleomenis Katevas, Lorenzo Minto e Hamed Haddadi. "MELTing Point: Mobile Evaluation of Language Transformers". In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*. 2024. arXiv: 2403.12844 (cit. alle pp. 27, 28, 30).

[59] Salvatore Sanfilippo. *DwarfStar (ds4): DeepSeek 4 Flash and PRO local inference engine for Metal, CUDA and ROCm*. URL: https://github.com/antirez/ds4 (visitato il giorno 2026-10-13) (cit. a p. 27).

[60] JustVugg. *colibri: run frontier MoE models on hardware you already own, experts streamed from disk*. URL: https://github.com/JustVugg/colibri (visitato il giorno 2026-10-13) (cit. alle pp. 27, 28).

[61] Tim Dettmers, Mike Lewis, Younes Belkada e Luke Zettlemoyer. "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022 (cit. alle pp. 28, 29).

[62] Elias Frantar, Saleh Ashkboos, Torsten Hoefler e Dan Alistarh. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. In: *International Conference on Learning Representations (ICLR)*. 2023 (cit. alle pp. 28, 29).

[63] Ji Lin et al. “AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration”. In: *Proceedings of Machine Learning and Systems (MLSys)*. 2024 (cit. alle pp. 28, 29).

[64] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman e Luke Zettlemoyer. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023 (cit. alle pp. 29, 38, 45).

[65] Yaniv Leviathan, Matan Kalman e Yossi Matias. “Fast Inference from Transformers via Speculative Decoding”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. PMLR. 2023 (cit. a p. 29).

[66] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li et al. “From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference”. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. 2023 (cit. alle pp. 30, 85).

[67] Alexandra Sasha Luccioni, Yacine Jernite e Emma Strubell. “Power Hungry Processing: Watts Driving the Cost of AI Deployment?” In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2024 (cit. alle pp. 30, 85).

[68] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré e Azalia Mirhoseini. *Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making*. Blog post, not peer-reviewed. 2026. URL: `https://contrastive-lm.notion.site` (cit. a p. 33).

[69] Convai Innovations. *Laya: a System One decision model (model card, convaiinnovations/laya)*. URL: `https://huggingface.co/convaiinnovations/laya` (visitato il giorno 2026-10-10) (cit. a p. 33).

[70] Seyedarmin Azizi, Erfan Baghaei Potraghloo e Massoud Pedram. *Labels Override Definitions in Jev-Style Typed Decision Models*. 2026. arXiv: `2610.02586` (cit. a p. 33).

[71] Cho Yin Yong. *DecideBench (dataset card, choyiny/decidebench)*. URL: `https://huggingface.co/datasets/choyiny/decidebench` (visitato il giorno 2026-10-10) (cit. a p. 33).

[72] Perplexity. *pplx-decider-v1.1-27b (model card)*. URL: https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b (visitato il giorno 2026-10-10) (cit. a p. 34).

[73] Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li et al. *Just Ask Jev: Reinforcement Learning for Calibrated Decisions as a Zero-Shot Detector of AI Alignment Failures*. 2026. arXiv: 2609.29429 (cit. a p. 34).

[74] Yubo Li, Yidi Miao, Ramayya Krishnan e Rema Padman. *JEV-as-a-Judge: Accept When Confident, Escalate When Unsure*. 2026. arXiv: 2609.26550 (cit. a p. 34).

[75] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel et al. ""My Answer is C": First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. arXiv: 2402.14499 (cit. alle pp. 34, 45).

[76] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou e Minlie Huang. "Large Language Models Are Not Robust Multiple Choice Selectors". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. alle pp. 34, 45).

[77] Adina Williams, Nikita Nangia e Samuel R. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *Proceedings of NAACL-HLT 2018*. 2018 (cit. a p. 35).

[78] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston e Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020 (cit. alle pp. 35, 53).

[79] Tal Schuster, Adam Fisch e Regina Barzilay. "Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 624–643. DOI: 10.18653/v1/2021.naacl-main.52. URL: https://aclanthology.org/2021.naacl-main.52 (cit. alle pp. 35, 51, 52).

[80] Philippe Laban, Tobias Schnabel, Paul N. Bennett e Marti A. Hearst. "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 163–177 (cit. a p. 35).

[81] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of EMNLP 2023*. 2023 (cit. a p. 35).

[82] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023 (cit. a p. 36).

[83] Shahul Es, Jithin James, Luis Espinosa-Anke e Steven Schockaert. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024 (cit. a p. 36).

[84] Jon Saad-Falcon, Omar Khattab, Christopher Potts e Matei Zaharia. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 36).

[85] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela e Rebecca Qian. *Lynx: An Open Source Hallucination Evaluation Model*. 2024. arXiv: 2407.08488 (cit. a p. 36).

[86] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil e Hannaneh Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. a p. 36).

[87] Brandon T. Willard e Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: 2307.09702 (cit. alle pp. 36, 65).

[88] Liyan Tang, Philippe Laban e Greg Durrett. *LLM-AggreFact Leaderboard*. 2024. URL: https://llm-aggrefact.github.io/ (visitato il giorno 2026-10-05) (cit. alle pp. 36, 51, 52).

[89] Chuan Guo, Geoff Pleiss, Yu Sun e Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017 (cit. alle pp. 36, 46, 47).

[90] Mahdi Pakdaman Naeini, Gregory F. Cooper e Milos Hauskrecht. "Obtaining Well Calibrated Probabilities Using Bayesian Binning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2015 (cit. alle pp. 36, 47).

[91] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein e Sameer Singh. "Calibrate Before Use: Improving Few-Shot Performance of Language Models". In: *International Conference on Machine Learning (ICML)*. 2021 (cit. alle pp. 36, 46).

[92] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan et al. *Language Models (Mostly) Know What They Know*. 2022. arXiv: 2207.05221 (cit. a p. 37).

[93] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback". In: *Proceedings of EMNLP 2023*. 2023 (cit. alle pp. 37, 82).

[94] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li et al. "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. alle pp. 37, 82).

[95] Yonatan Geifman e Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017 (cit. alle pp. 37, 67).

[96] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi e Noah Smith. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. 2020. arXiv: 2002.06305 (cit. alle pp. 39, 63).

[97] Marius Mosbach, Maksym Andriushchenko e Dietrich Klakow. "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. alle pp. 39, 73, 92).

[98] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013 (cit. alle pp. 39, 73, 92).

[99] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder e Li Deng. *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. CoCo Workshop at NIPS 2016. 2016. arXiv: 1611.09268 (cit. a p. 51).

[100] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman e Phil Blunsom. "Teaching Machines to Read and Comprehend". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015 (cit. a p. 51).

[101] Abigail See, Peter J. Liu e Christopher D. Manning. "Get To The Point: Summarization with Pointer-Generator Networks". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017 (cit. a p. 51).

[102] Yelp. *Yelp Open Dataset*. URL: https://business.yelp.com/data/resources/open-dataset/ (visitato il giorno 2026-10-05) (cit. a p. 52).

[103] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryściński, Justin Rousseau e Greg Durrett. "Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. a p. 52).

[104] Liyan Tang, Igor Shalyminov, Amy Wing-mei Wong et al. "TofuEval: Evaluating Hallucinations of LLMs on Topic-Focused Dialogue Summarization". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 52).

[105] Nelson F. Liu, Tianyi Zhang e Percy Liang. "Evaluating Verifiability in Generative Search Engines". In: *Findings of EMNLP 2023*. 2023 (cit. a p. 52).

[106] Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar e Dan Roth. "ExpertQA: Expert-Curated Questions and Attributed Answers". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 52).

[107] Hung-Ting Chen, Fangyuan Xu, Shane Arora e Eunsol Choi. "Understanding Retrieval Augmentation for Long-Form Question Answering". In: *Conference on Language Modeling (COLM)*. 2024 (cit. a p. 52).

[108] Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez e Greg Durrett. "WiCE: Real-World Entailment for Claims in Wikipedia". In: *Proceedings of EMNLP 2023*. 2023 (cit. a p. 52).

[109] Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni e Mor Geva. "A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. a p. 52).

[110] Yuxia Wang, Revanth Gangi Reddy, Zain Muhammad Mujahid et al. "Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers". In: *Findings of EMNLP 2024*. 2024 (cit. a p. 52).

[111] Liyan Tang. *LLM-AggreFact dataset card (lytang/LLM-AggreFact)*. URL: `https://huggingface.co/datasets/lytang/LLM-AggreFact` (visitato il giorno 2026-10-05) (cit. a p. 52).

[112] Tal Schuster. *VitaminC dataset card (tals/vitaminc)*. URL: `https://huggingface.co/datasets/tals/vitaminc` (visitato il giorno 2026-10-06) (cit. a p. 52).

[113] Nipun Sadvilkar e Mark Neumann. "PySBD: Pragmatic Sentence Boundary Disambiguation". In: *Proceedings of the Second Workshop for NLP Open Source Software (NLP-OSS)*. 2020, pp. 110–114. DOI: 10.18653/v1/2020.nlposs-1.15. URL: https://aclanthology.org/2020.nlposs-1.15 (cit. a p. 53).

[114] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand e Emir Muñoz. *Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG*. 2026. arXiv: 2603.27752. URL: https://arxiv.org/abs/2603.27752 (cit. alle pp. 57, 93).

[115] Philipp Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2004, pp. 388–395. URL: https://aclanthology.org/W04-3250 (cit. a p. 67).

[116] Taylor Berg-Kirkpatrick, David Burkett e Dan Klein. "An Empirical Investigation of Statistical Significance in NLP". In: *Proceedings of EMNLP-CoNLL 2012*. 2012, pp. 995–1005. URL: https://aclanthology.org/D12-1091 (cit. a p. 67).

[117] Rotem Dror, Gili Baumer, Segev Shlomov e Roi Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2018, pp. 1383–1392. URL: https://aclanthology.org/P18-1128 (cit. a p. 67).

[118] Eurostat. *Electricity price statistics*. Statistics Explained; household prices, second half of 2025. URL: https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Electricity_price_statistics (visitato il giorno 2026-10-19) (cit. a p. 90).

[119] Anthropic. *Claude API pricing*. URL: https://platform.claude.com/docs/en/about-claude/pricing (visitato il giorno 2026-10-19) (cit. a p. 90).

[120] OpenAI. *API pricing*. URL: https://developers.openai.com/api/docs/pricing (visitato il giorno 2026-10-19) (cit. a p. 90).

[121] Google. *Gemini API pricing*. URL: https://ai.google.dev/gemini-api/docs/pricing (visitato il giorno 2026-10-19) (cit. a p. 90).